

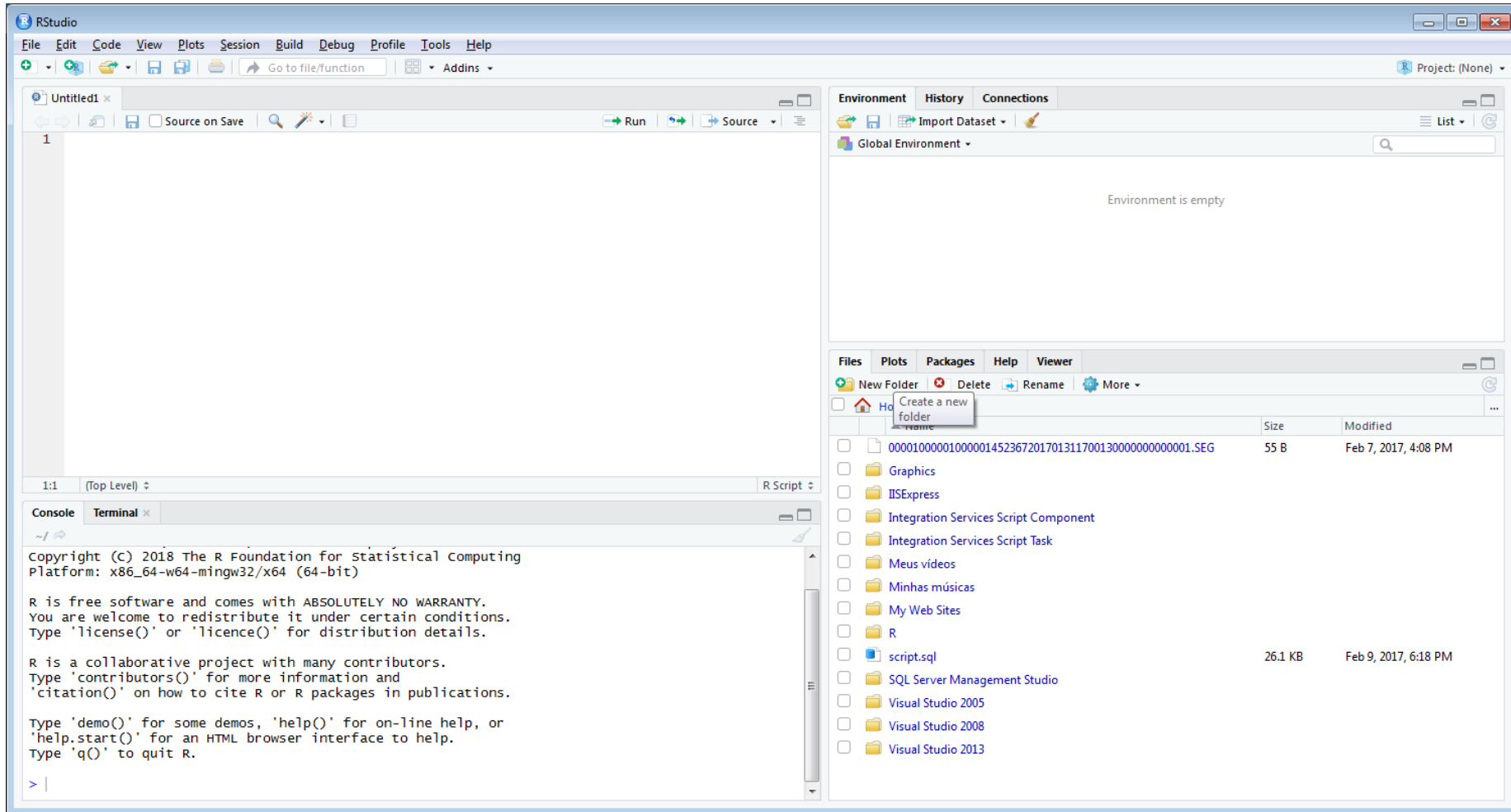
INF 1036

Probabilidade Computacional

Material Extra – Revisão R



R Studio



Console

- O símbolo “>” no console indica que o R está esperando um comando.
- Funcionamento básico:
 - entre com uma expressão no console e digite <Enter>.
 - a expressão é avaliada e o resultado apresentado na tela.
- Note que **[1]** sempre acompanha qualquer resultado.

```
> 1+2+3
[1] 6
> 1+2*3
[1] 7
> |
```

- Use os **parênteses** para calcular **expressões**, por exemplo:

```
> ((20 + 7)/3)^2
[1] 81
>
```

Console

- Para conseguir ajuda sobre um comando pode ser usada a função **help()**:

help(comando) ou ?comando

- Para instalar um pacote no seu ambiente você pode usar o comando **install.packages()** com o nome do pacote entre "" dentro do parênteses:

install.packages("nomepacote")

- Para usar o pacote em seu programa use o comando **library()** com o nome do pacote como parâmetro (sem ""):

library(nomepacote)

Algumas características do R

- R é **case-sensitive**: então **A** e **a** são símbolos diferentes e se referem a diferentes variáveis.
- Comandos diferentes são separados por ponto e vírgula ;.
- Comentários começam com #.
- Como a maioria das linguagens de programação, R permite atribuir valores a variáveis.

```
endereco = "Rua Lessa" ; numero = 36;  
Endereco = "Rua Lessa 10"
```

```
print(endereco) # Exemplo de uso da função print.
```

Constantes armazenadas no R

```
> pi
[1] 3.141593
>
> letters
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q"
   "r" "s" "t" "u" "v" "w" "x" "y" "z"
>
> LETTERS
[1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q"
   "R" "S" "T" "U" "V" "W" "X" "Y" "Z"
>
> month.abb
[1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "
Dec"
>
> month.name
[1] "January" "February" "March" "April" "May" "June"
   "July" "August" "September"
[10] "October" "November" "December"
```

Tipos de dados no R

- character (“aula”)
- numeric (1.2)
- integer (3L)
- logical (TRUE or FALSE)
- vector (tipos homogêneos)
- list (parecidos com vectors, mas heterogêneos)
- matrix
- missing values (NA)
- ...

Operadores matemáticos

- A lista abaixo apresenta alguns dos principais operadores matemáticos do R.

Operador	Descrição
+	Operador de adição
-	Operador de subtração
*	Operador de multiplicação
/	Operador de divisão
:	Operador de sequência
^	Operador exponencial
%%	Operador de módulo

```
x <- (2 + 3) * 2      # x assume o valor 10.
y <- 2 + 3 * 2        # y assume o valor 8.
z = x - y             # z assume o valor 2.
x <- (4 / 5) * (-2)   # x assume o valor -1.6.
a <- 1:10             # a é um vetor de 1 a 10.
y <- z^3              # y assume o valor 8.
z <- 15 %% 4          # z assume o valor 3, resto da
                     # divisão inteira de 15 por 4.
```

```
Se (b %% 2) retornar 0, então b é um número par.
Se (b %% 2) retornar 1, então b é um número impar.
```

Operadores relacionais e lógicos

- São operados binários para realização de testes entre duas variáveis (objetos). Estas operações retornam o valor TRUE (1) ou FALSE (0).

Operador	Descrição
<code>==</code>	Operador de igualdade
<code>></code>	Operador “maior que”
<code><</code>	Operador “menor que”
<code><=</code>	Operador “menor ou igual”
<code>!</code>	Operador “Não”
<code>&</code>	Operador lógico “E”
<code> </code>	Operador lógico “OU”

```
x <- 10
y <- 8

(x == y)           # retorna FALSE.
(x == (y + 2))     # retorna TRUE.
(x < y)            # retorna FALSE.
(x > y)            # retorna TRUE.
(x <= y)           # retorna FALSE.
(x != y)           # retorna TRUE.
```

Operadores relacionais e lógicos

- São operados binários para realização de testes entre duas variáveis (objetos). Estas operações retornam o valor TRUE (1) ou FALSE (0).

Operador	Descrição
<code>==</code>	Operador de igualdade
<code>></code>	Operador “maior que”
<code><</code>	Operador “menor que”
<code><=</code>	Operador “menor ou igual”
<code>!</code>	Operador “Não”
<code>&</code>	Operador lógico “E”
<code> </code>	Operador lógico “OU”

```
x <- 2
```

```
y <- 3
```

```
z <- 4
```

```
!(x == y)           # retorna TRUE.
```

```
!(x < y)            # retorna FALSE.
```

```
!TRUE               # retorna FALSE.
```

```
!FALSE              # retorna TRUE.
```

Operadores relacionais e lógicos

- São operados binários para realização de testes entre duas variáveis (objetos). Estas operações retornam o valor TRUE (1) ou FALSE (0).

Operador	Descrição
<code>==</code>	Operador de igualdade
<code>></code>	Operador “maior que”
<code><</code>	Operador “menor que”
<code><=</code>	Operador “menor ou igual”
<code>!</code>	Operador “Não”
<code>&</code>	Operador lógico “E”
<code> </code>	Operador lógico “OU”

```
x <- 2
```

```
y <- 3
```

```
z <- 4
```

```
(x < y) & (y < z) # retorna TRUE.
```

```
(x < y) & (x > 5) # retorna FALSE.
```

```
TRUE & TRUE      # retorna TRUE.
```

```
TRUE & FALSE     # retorna FALSE.
```

```
FALSE & TRUE     # retorna FALSE.
```

```
FALSE & FALSE    # retorna FALSE.
```

Operadores relacionais e lógicos

- São operados binários para realização de testes entre duas variáveis (objetos). Estas operações retornam o valor TRUE (1) ou FALSE (0).

Operador	Descrição
<code>==</code>	Operador de igualdade
<code>></code>	Operador “maior que”
<code><</code>	Operador “menor que”
<code><=</code>	Operador “menor ou igual”
<code>!</code>	Operador “Não”
<code>&</code>	Operador lógico “E”
<code> </code>	Operador lógico “OU”

```
x <- 2
y <- 3
z <- 4

(x < y) | (y < z) # retorna TRUE.
(x > y) | (x > 1) # retorna TRUE.
TRUE | TRUE      # retorna TRUE.
TRUE | FALSE     # retorna TRUE.
FALSE | TRUE     # retorna TRUE.
FALSE | FALSE    # retorna FALSE.
```

Algumas funções internas do R

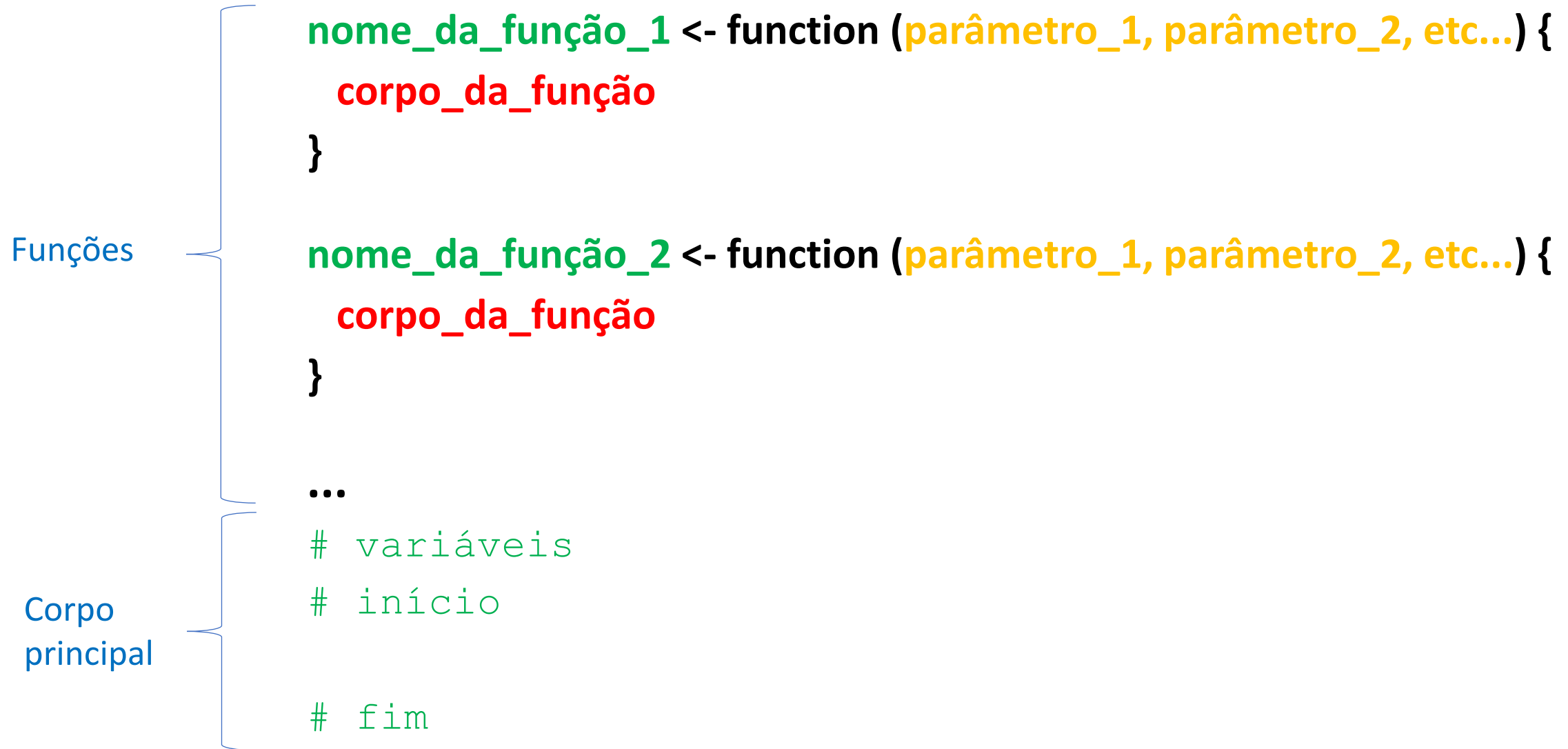
Função	Descrição
<code>sqrt()</code>	Raiz quadrada.
<code>abs()</code>	Valor absoluto.
<code>exp()</code>	Exponencial.
<code>log10()</code>	Logaritmo na base 10.
<code>log()</code>	Logaritmo na base e.
<code>sin()</code> <code>cos()</code> <code>tan()</code>	Funções trigonométricas.
<code>asin()</code> <code>acos()</code> <code>atan()</code>	Funções trigonométricas inversas.
<code>sort()</code>	Ordena os elementos de um vetor.
<code>read.csv()</code>	Lê um arquivo no padrão csv.
<code>max(x)</code>	Retorna o maior valor encontrado em x (vetor, etc).
<code>mean(x)</code>	Retorna a média dos elementos de x (vetor, etc).
<code>tolower()</code>	Reescreve uma string em letras minúsculas.
<code>hist()</code>	Cria um histograma.

Definindo funções

```
nome_da_função <- function (parâmetro_1, parâmetro_2, etc...) {  
  corpo_da_função  
}
```

- A **primeira linha** da definição de uma função chama-se **cabeçalho**.
- O **nome da função** deve seguir as mesmas regras de nomeação de variáveis, como, por exemplo, não começar com números.
- **Funções** podem ou não possuir **parâmetros**.
- O **corpo da função** é a sequência de instruções dentro da definição da função e normalmente possui:
 - declaração de variáveis;
 - lógica de processamento;
 - valor de retorno.
- Um comando **não obrigatório**, mas muito comum nas funções, é o **return()** que retorna o **valor de retorno** calculado pela função.

Organização básica de um script R



Como fica o escopo das variáveis?

- Uma **função não enxerga** (não tem acesso) às **variáveis de outra função**.
- A **passagem de dados** para, ou entre funções, é feita **através de parâmetros**.
- As funções **podem** ou **não retornar valores** e, no caso de retorno, estes ficam disponíveis para utilização de quem chamou a função somente após o término da execução da função.
- **Parâmetros e variáveis declaradas no corpo** da função **não existem fora** da função. Em termos computacionais, eles são criados **a cada vez** que a função é chamada, **deixando de existir** quando a função **termina sua execução**.
- Variáveis definidas como **globais** no corpo principal do programa (script) podem ser acessadas **dentro e fora das funções**.

Execução condicional vetorizada

- Como visto anteriormente, a linguagem R dispõe de execução condicional da seguinte forma:
 - **if** (condição) **expres_1** **else** **expres_2**.
- Onde o **resultado da condição** deve ser um valor lógico **TRUE** ou **FALSE**, e, se este for verdadeiro, será executada a **expres_1**; caso contrário, será executada a **expres_2**. O uso do condicional **else** não é obrigatório.
- Existe uma **versão vetorizada** da construção condicional **if/else**, que é a função **ifelse**, cuja sintaxe no R é:
 - **ifelse** (condição, **expres_1**, **expres_2**).
- A **expres_1** será executada, se a **condição** for verdadeira, e a **expres_2** será executada, caso contrário.

```
x <- 8
y <- ifelse (x >= 5, x + 3, x - 4)
print(y)
```

Estruturas de controle de fluxo

- A Linguagem R, como qualquer linguagem de programação, possui instruções destinadas a **alterar o curso sequencial de execução** dos programas.
- **Como já visto**, para executar parte do código **somente se** determinada **condição for verdadeira**, utilizamos a instrução *if*, incluindo a condição a ser testada entre os parênteses. Se houver algum **código alternativo** a ser executado no caso da **condição ser falsa**, podemos informar isso para o R com a instrução *else*.
- Agora, como fazer para:
 - contar de 1 a 50.
 - executar uma ou mais instruções 100 vezes.
 - executar uma ou mais instruções repetidas vezes até que um critério de parada seja alcançado.

Ciclos (estruturas de repetição ou loops)

- Ciclos são **processos iterativos** (repetitivo ou realizado diversas vezes) que permitem executar um **bloco de instruções** com base em uma **condição previamente estabelecida**.
- É importante, nestes casos, que as iterações tenham uma **condição finita**.
- Instruções como **while(condição)**, **for(condição)** e **repeat** permitem a criação de ciclos no R.

```
....  
while (<condição booleana>)  
  <bloco de instruções>  
...
```

```
....  
for (<variável de controle> in <conjunto>)  
  <bloco de instruções>  
...
```

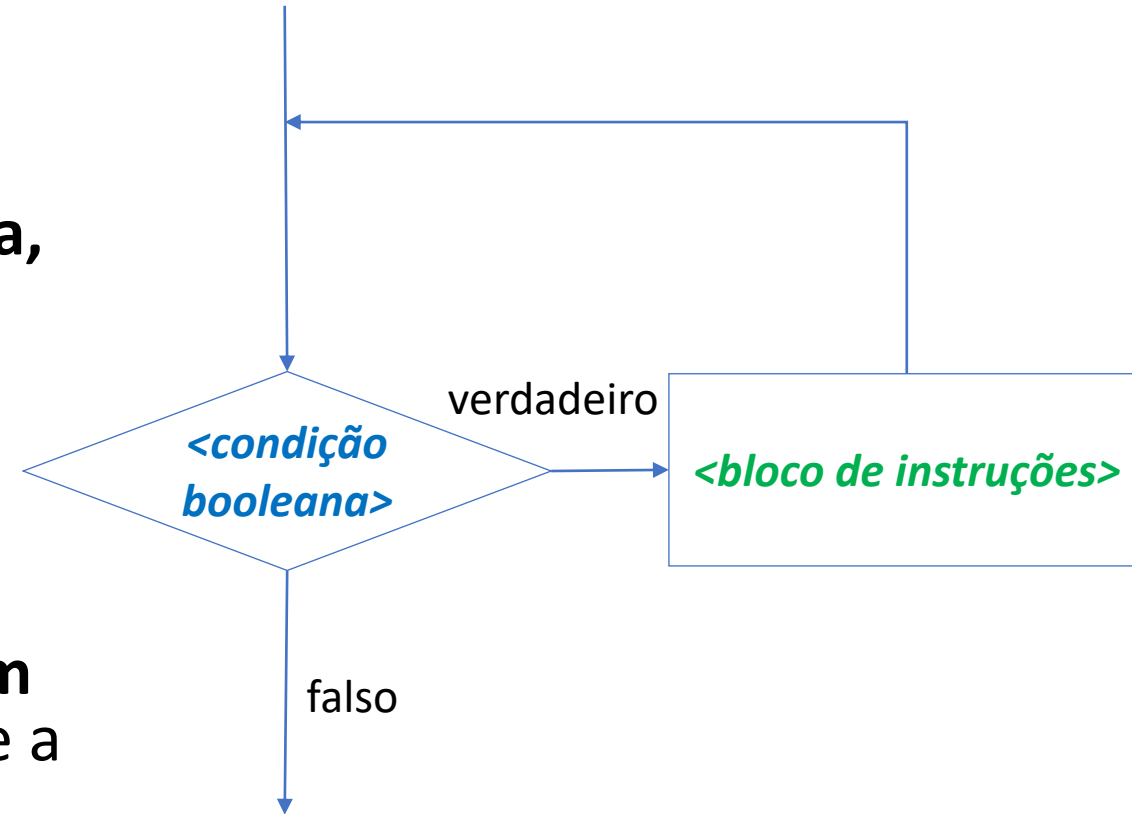
```
....  
repeat  
  <bloco de instruções>  
...
```

Instrução iterativa – *while*

- A sua semântica pode ser descrita por:
enquanto a condição booleana for verdadeira, repita o bloco de instruções.

```
....  
while (<condição booleana>)  
    <bloco de instruções>  
...
```

- As instruções no bloco dentro do **while** podem **nunca ser executadas**, bastando para isso que a condição seja **falsa da primeira vez** em que é avaliada.
- Um erro relativamente frequente consiste em escrever instruções iterativas que levam a **loops (ciclos) que nunca terminam**.

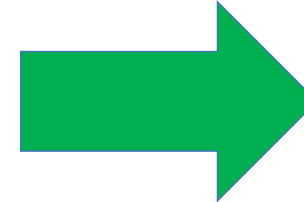


Instrução iterativa – *while*

- A sua semântica pode ser descrita por: **enquanto a condição booleana for verdadeira, repita o bloco de instruções.**

```
....  
while (<condição booleana>)  
  <bloco de instruções>  
...
```

```
x <- 0  
while (x <= 5) {  
  x <- x + 1  
  print (x)  
}
```



[1]	1
[1]	2
[1]	3
[1]	4
[1]	5
[1]	6

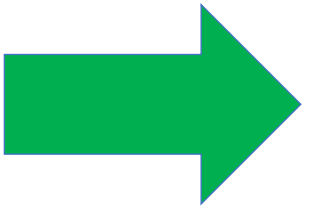
- As instruções no bloco dentro do **while** podem nunca ser executadas, bastando para isso que a condição seja **falsa da primeira vez** em que é avaliada.
- Um erro relativamente frequente consiste em escrever instruções iterativas que levam a **loops (ciclos) que nunca terminam**.

Instrução iterativa – *while*

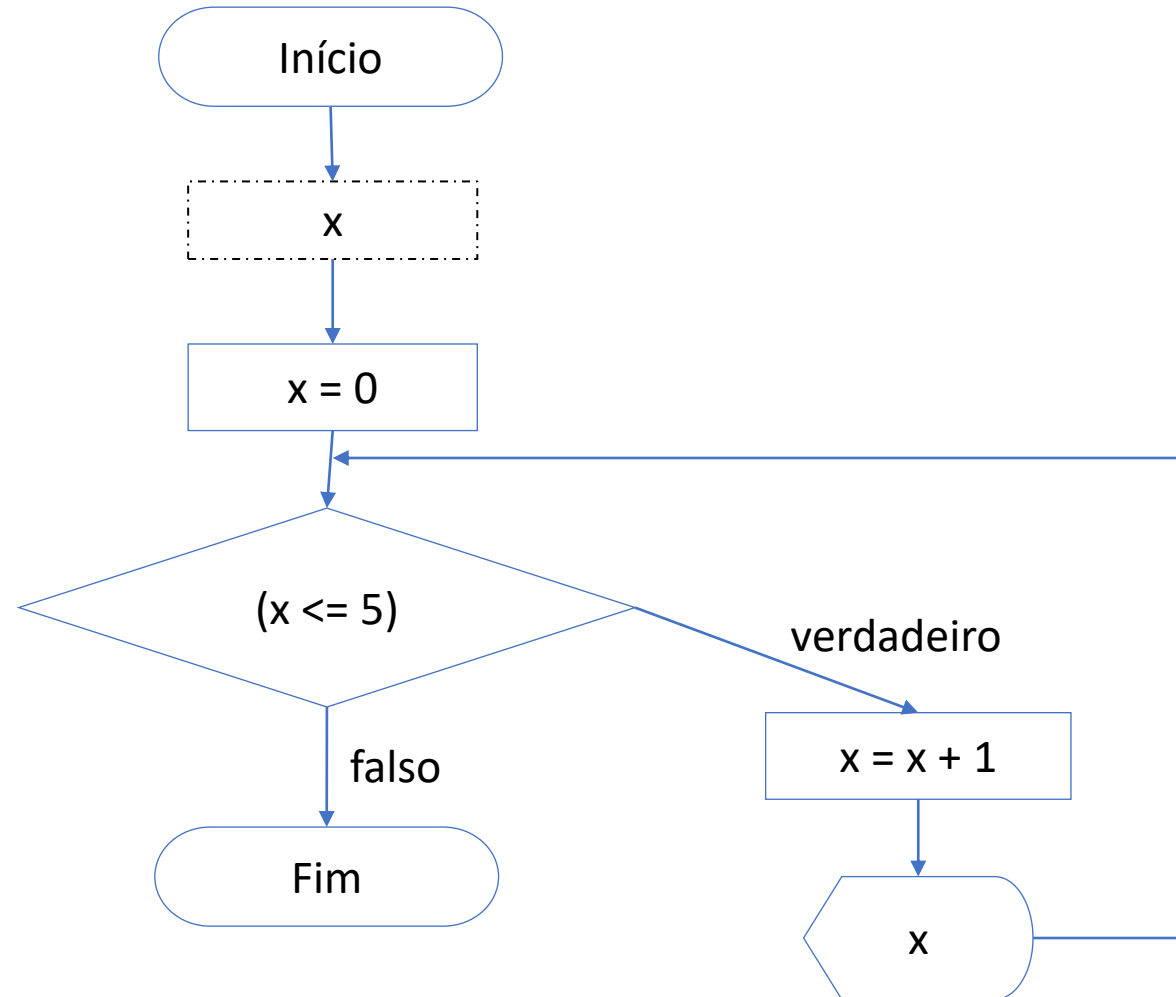
```
....  
while (<condição booleana>  
    <bloco de instruções>  
...  

```

```
x <- 0  
while (x <= 5) {  
    x <- x + 1  
    print (x)  
}
```



```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5  
[1] 6
```

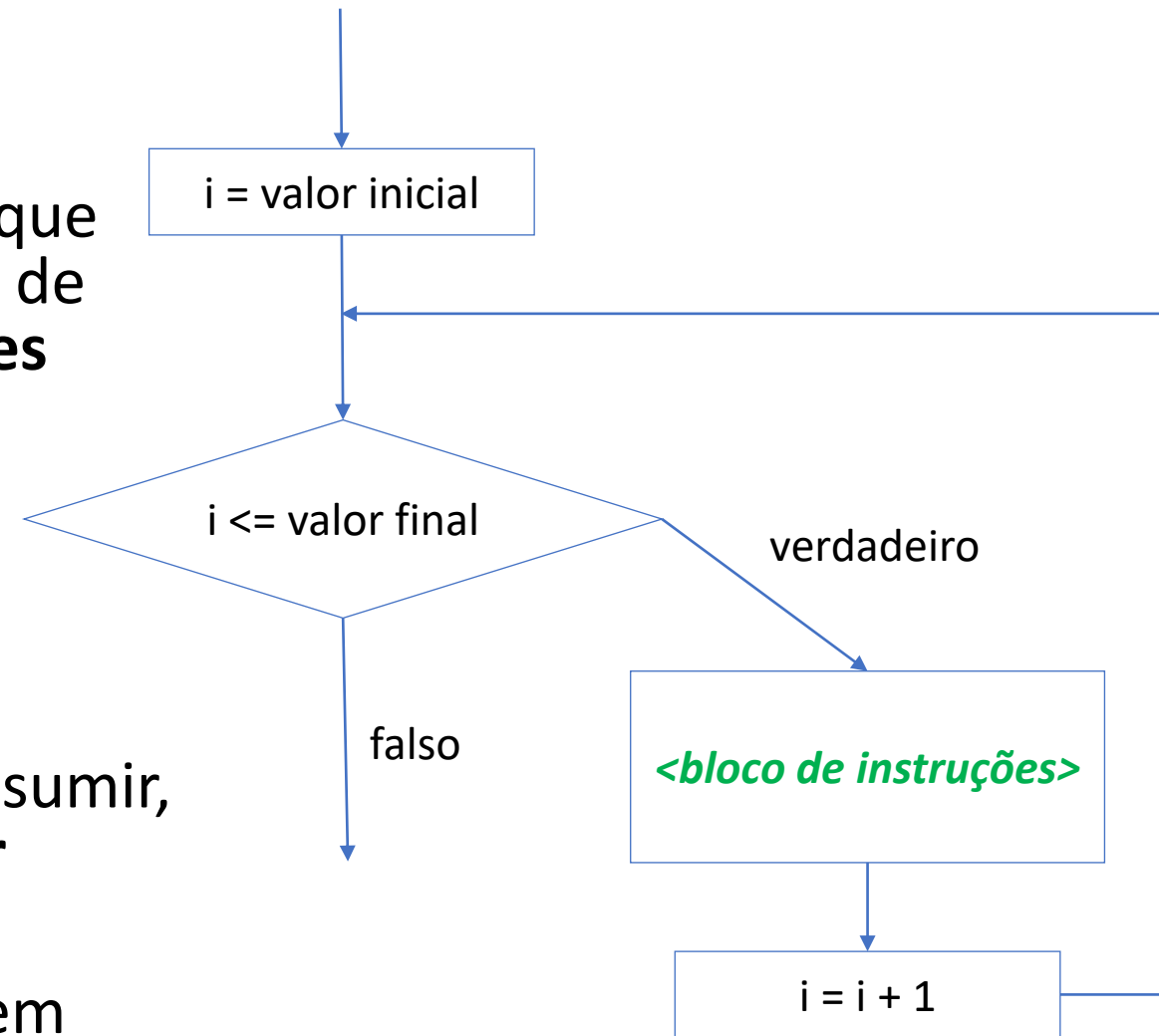


Instrução iterativa – *for*

- O **for** permite controlar o **número de vezes** que um bloco de instruções é **executado** através de uma **variável de controle** que assume **valores pré-definidos** em cada iteração do loop.

```
....  
for (<variável de controle> in <conjunto>)  
    <bloco de instruções>  
...
```

- No fluxo **i** é a **variável de controle** que irá assumir, a cada iteração e de forma sequencial (**valor inicial** a **valor final**), os valores do **conjunto**.
- Um erro **relativamente frequente** consiste em escrever instruções iterativas que levam a **loops (ciclos) que nunca terminam**.

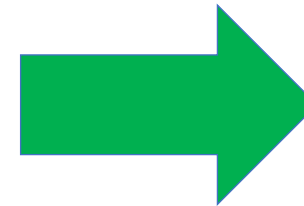


Instrução iterativa – *for*

- O **for** permite controlar o **número de vezes** que um bloco de instruções é **executado** através de uma **variável de controle** que assume **valores pré-definidos** em cada iteração do loop.

```
....  
for (<variável de controle> in <conjunto>)  
    <bloco de instruções>  
...
```

```
for (i in 1:6) {  
    print (i)  
}
```



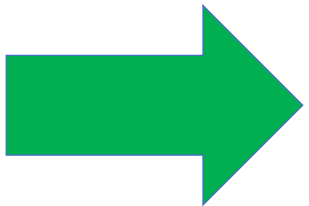
```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5  
[1] 6
```

- No exemplo **i** é a **variável de controle** que irá assumir, a cada iteração e de forma sequencial, os valores do **conjunto 1:6**.
- Um erro **relativamente frequente** consiste em escrever instruções iterativas que levam a **loops (ciclos) que nunca terminam**.

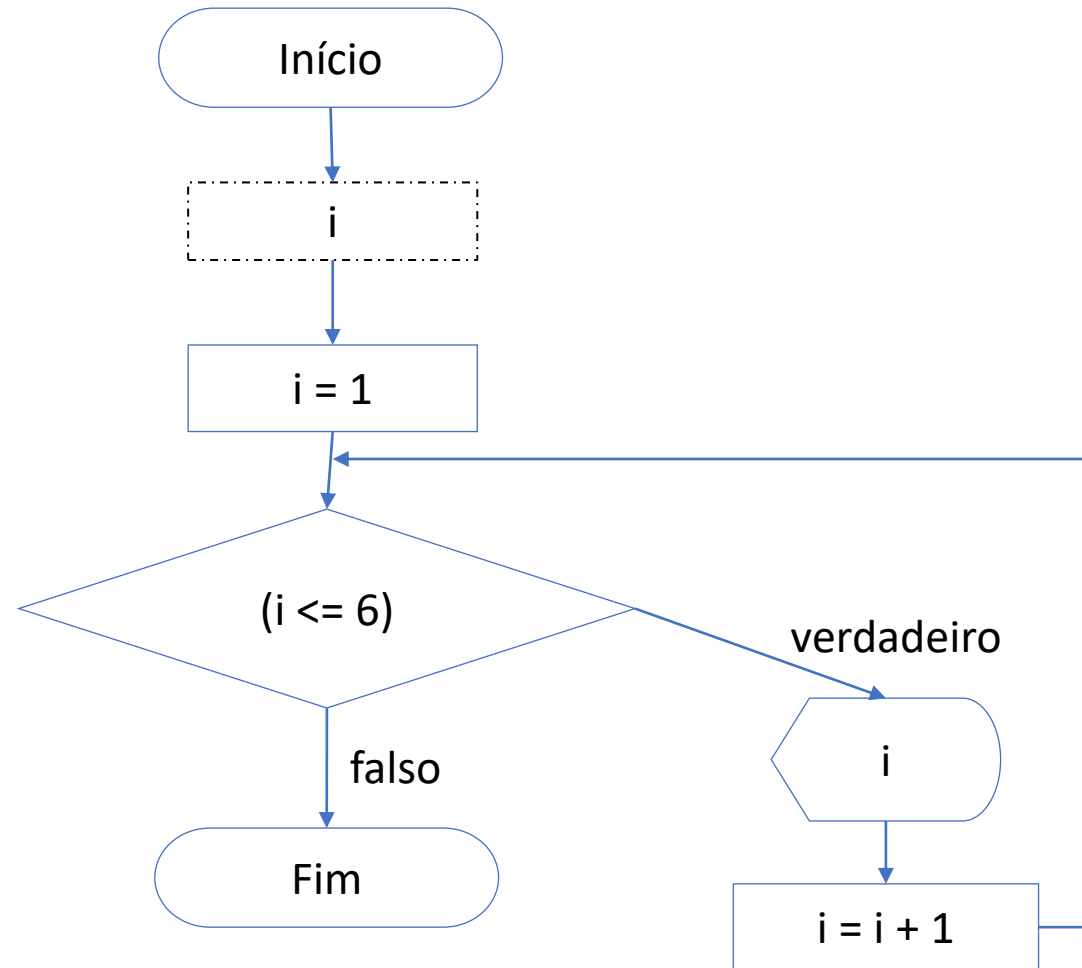
Instrução iterativa – *for*

```
...  
for (<variável de controle> in <conjunto>)  
    <bloco de instruções>  
...
```

```
for (i in 1:6) {  
    print (i)  
}
```



```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5  
[1] 6
```

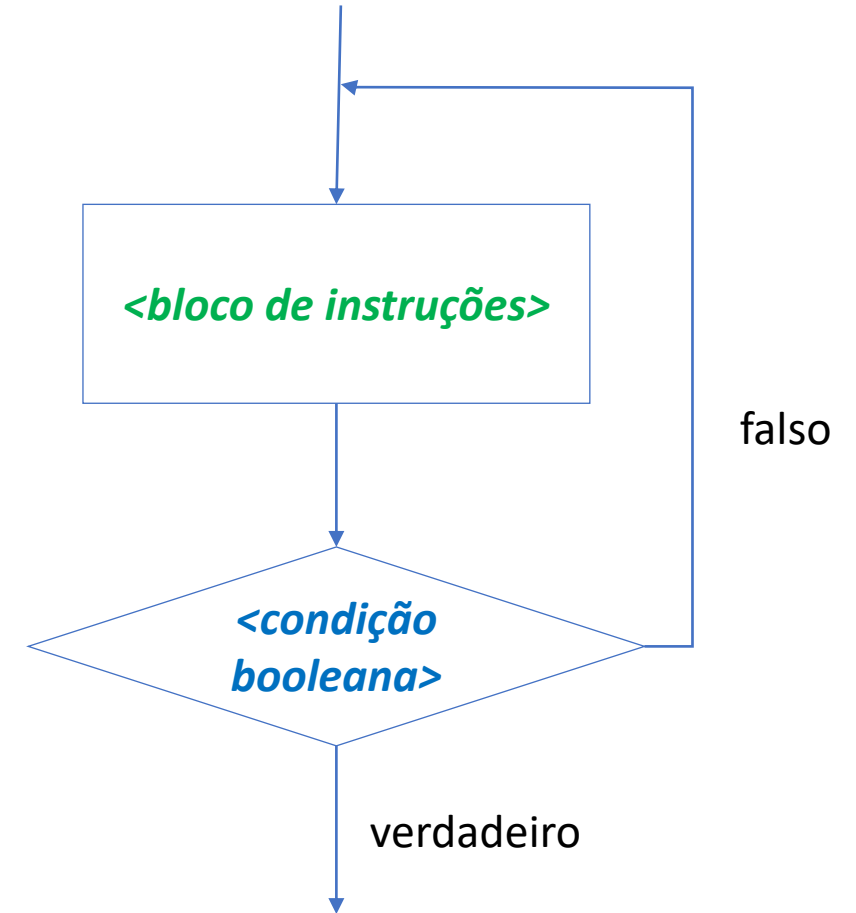


Instrução iterativa – *repeat*

- A instrução **repeat** permite **executar** um **bloco de instruções uma ou mais vezes**.

```
....  
repeat  
  <bloco de instruções>  
...
```

- A instrução **break** dentro do loop faz o R **terminar a repetição** da execução do **bloco de instruções**. Ela também pode ser utilizada com **while** e **for**.
- Um erro **relativamente frequente** consiste em escrever instruções iterativas que levam a **loops (ciclos) que nunca terminam**.

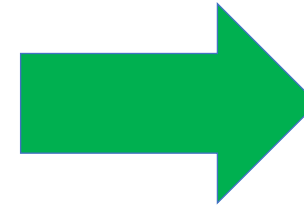


Instrução iterativa – *repeat*

- A instrução **repeat** permite **executar** um **bloco de instruções** uma ou mais **vezes**.

```
....  
repeat  
  <bloco de instruções>  
...
```

```
x <- 0  
repeat {  
  x <- x + 1  
  print (x)  
  if (x > 5) {  
    break  
  }  
}
```



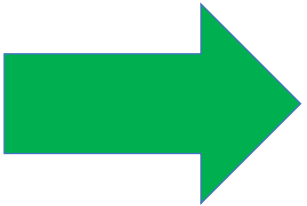
```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5  
[1] 6
```

- A instrução **break** dentro do loop faz o R **terminar a repetição** da execução do **bloco de instruções**. Ela também pode ser utilizada com **while** e **for**.
- Um erro **relativamente frequente** consiste em escrever instruções iterativas que levam a **loops (ciclos) que nunca terminam**.

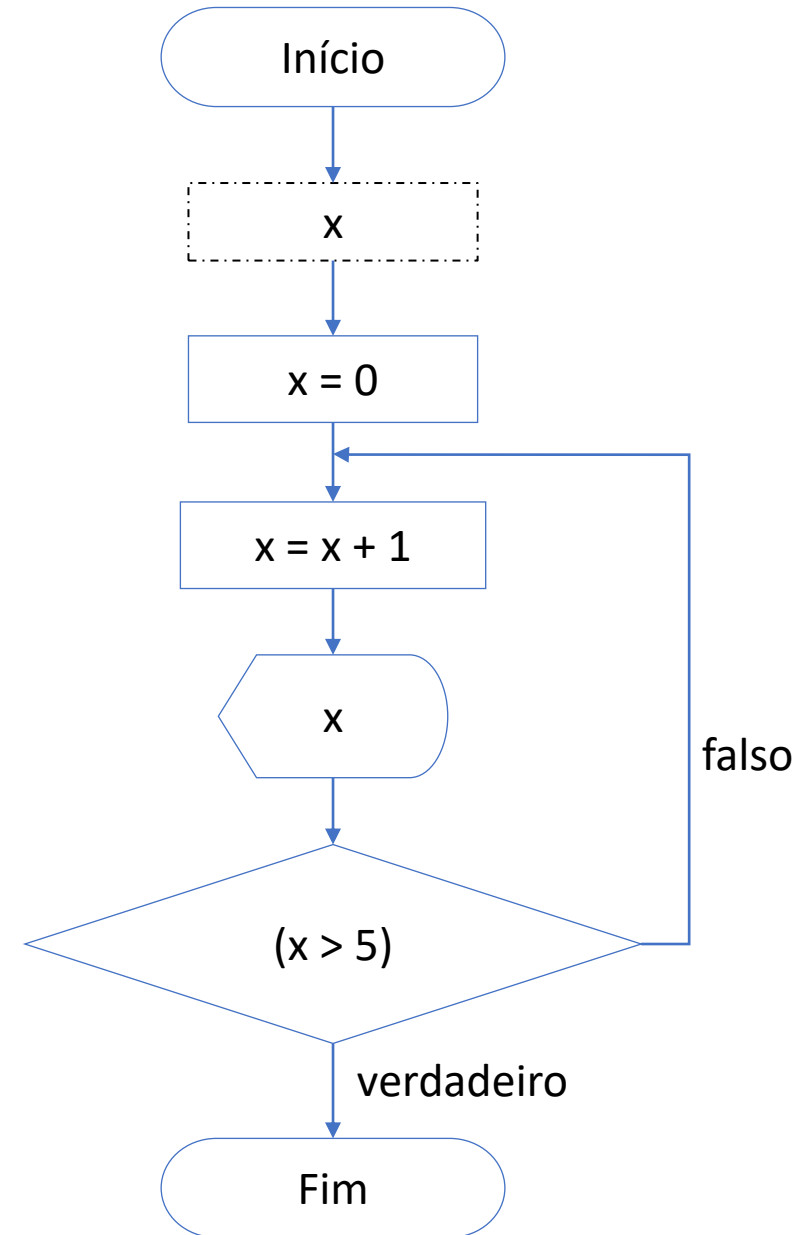
Instrução iterativa – *repeat*

```
....  
repeat  
  <bloco de instruções>  
...
```

```
x <- 0  
repeat {  
  x <- x + 1  
  print (x)  
  if (x > 5) {  
    break  
  }  
}
```



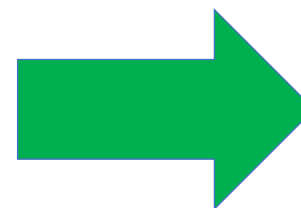
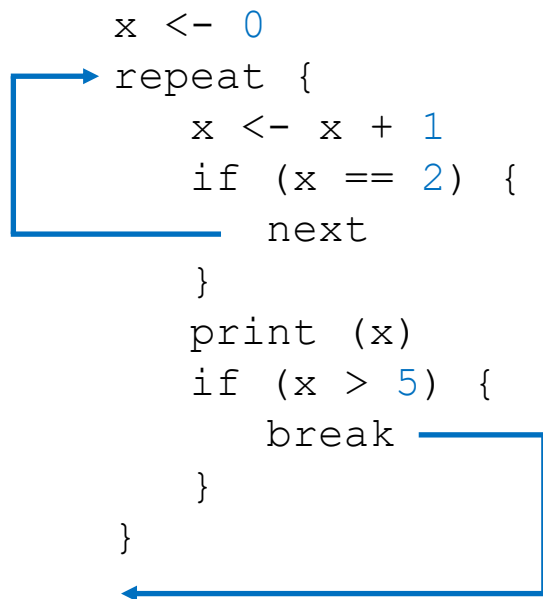
```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5  
[1] 6
```



Instrução iterativa – *repeat*

```
....  
repeat  
  <bloco de instruções>  
....
```

```
x <- 0  
repeat {  
  x <- x + 1  
  if (x == 2) {  
    next  
  }  
  print (x)  
  if (x > 5) {  
    break  
  }  
}
```



```
[1] 1  
[1] 3  
[1] 4  
[1] 5  
[1] 6
```

- A instrução **next** dentro do loop faz o R **saltar para o início** da execução do **bloco de instruções**.

Valores reservados do R

- Existem valores reservados para representar dados faltantes, infinitos e indefinições matemáticas.
- **NA** (Not Available) significa **dado faltante/indisponível**. É o **null** de outras linguagens. O **NA** tem uma classe, ou seja, podemos ter **NA** numeric, **NA** character, etc.
- **NaN** (Not a Number) representa **indefinições matemáticas**, como $0/0$ e $\log(-1)$. Um **NaN** é um **NA**, mas a recíproca não é verdadeira.
- **Inf** (Infinito) é um **número muito grande** ou o **limite matemático**, por exemplo, $1/0$ e 10^{310} . Aceita sinal negativo **-Inf**.
- **NULL** representa a ausência de informação. Conceitualmente, a diferença entre **NA** e **NULL** é sutil, mas, no R, o **NA** está mais alinhado com os conceitos de estatística (ou como gostaríamos que os dados faltantes se comportassem em análise de dados), e o **NULL** está em sintonia com comportamentos de lógica de programação.
- Use as funções **is.na()**, **is.nan()**, **is.infinite()** e **is.null()** para testar se um objeto possui algum desses valores.

Classes básicas de objetos no R

- Existem cinco classes básicas de objetos no R:
 - character (“aula”)
 - numeric (1.)
 - integer (3)
 - logical (TRUE or FALSE)
 - complex : números complexos no formato $a + bi$
- Os objetos pertencentes a estas classes são chamados de **objetos atômicos**.
- A função interna **class()** é utilizada para se identificar a classe de um objeto.
- No R encontramos outras classes de objetos que podem ser agrupadas em dois grupos:
 - Contém apenas objetos de uma única classe básica de objetos: vector, matrix, array, factor, ts.
 - Podem conter objetos de todos os tipos de classe: data.frame, list.

Vetores

- Vetores no R são os objetos **mais simples** que podem armazenar **objetos atômicos**.
- Um vetor **tem** sempre a **mesma classe dos objetos** que **armazena**.
- A função **length()** retorna o **número** de elementos do vetor.

```
> sequencia <- 2:5
> sequencia
[1] 2 3 4 5
> length(sequencia)
[1] 4
> class(sequencia)
[1] "integer"
>
> cor <- c("verde", "azul")
> cor
[1] "verde" "azul"
> length(cor)
[1] 2
> class(cor)
[1] "character"
```

Criação de vetores

Instrução	Descrição	Exemplo
:	Cria sequências numéricas.	<code>v <- 1:5</code> <code>v <- -3:3</code>
scan	Lê valores digitados.	<code>v <- scan()</code>
c	Combina valores.	<code>v <- c (3, 4, 8, 9)</code> <code>v <- c ("verde", "amarelo")</code>
rep	Repete valores.	<code>v <- rep (c (3, 4), 2)</code>
seq	Cria sequências numéricas.	<code>v <- seq (-1, 1, 0.5)</code>

Instrução :

- A instrução : cria sequências numéricas, podendo ter prioridade sobre outras instruções.

```
> v <- 1:5
```

```
> v
```

```
[1] 1 2 3 4 5
```

```
>
```

```
> v <- -3:3
```

```
> v
```

```
[1] -3 -2 -1 0 1 2 3
```

```
>
```

```
> v <- 2*(1:5)
```

```
> v
```

```
[1] 2 4 6 8 10
```

```
>
```

```
> v <- rep(c(3, 4), 2)
```

```
> v
```

```
[1] 3 4 3 4
```

Instrução **scan**

- A instrução **scan** lê valores digitados.

```
> v <- scan()
1: 2.3 4.7 6.8 9.1
5:
Read 4 items
> v
[1] 2.3 4.7 6.8 9.1
>
> v <- scan(what = " ", sep = ",")
1: verde,amarelo,azul turquesa
4:
Read 3 items
> v
[1] "verde"          "amarelo"         "azul turquesa"
```

Instrução **c**

- A instrução **c** combina valores.

```
> v <- c(3, 4, 8, 9)
```

```
> v
```

```
[1] 3 4 8 9
```

```
>
```

```
> v <- c("verde", "azul")
```

```
> v
```

```
[1] "verde" "azul"
```

Instrução **rep**

- A instrução **rep** retorna o primeiro argumento, repetido o número de vezes indicado pelo segundo argumento.

```
> v <- rep(2, 5)
> v
[1] 2 2 2 2 2
>
> v <- rep(c(3, 4), 2)
> v
[1] 3 4 3 4
>
> v <- rep("A", 3)
> v
[1] "A" "A" "A"
```

Instrução **seq**

- A instrução **seq** cria sequências numéricas tendo como argumentos o início, o fim e os passos da sequência.

```
> v <- seq(-1, 1, 0.5)
> v
[1] -1.0 -0.5  0.0  0.5  1.0
>
> v <- seq(-1, 1, 0.3)
> v
[1] -1.0 -0.7 -0.4 -0.1  0.2  0.5  0.8
>
> v <- seq(-1, 1, length = 6)
> v
[1] -1.0 -0.6 -0.2  0.2  0.6  1.0
>
> v <- seq(-1, by = 0.4, length = 8)
> v
[1] -1.0 -0.6 -0.2  0.2  0.6  1.0  1.4  1.8
```

Concatenação de vetores

- Podemos concatenar vetores de uma mesma classe com a instrução `c`.

```
> v1 <- 1:5
> v1
[1] 1 2 3 4 5
> class(v1)
[1] "integer"
>
> v2 <- c(6L, 7L, 8L)
> v2
[1] 6 7 8
> class(v2)
[1] "integer"
>
> v <- c(v1, v2)
> v
[1] 1 2 3 4 5 6 7 8
> class(v)
[1] "integer"
```

A concatenação também funcionaria se `v2 <- c(6, 7, 8)`. Neste caso, os vetores `v1` e `v` seriam da classe "numeric".

```
> v1 <- c("verde", "azul")
> v1
[1] "verde" "azul"
> class(v1)
[1] "character"
>
> v2 <- c("amarelo", "branco")
> v2
[1] "amarelo" "branco"
> class(v2)
[1] "character"
>
> v <- c(v1, v2)
> v
[1] "verde"      "azul"      "amarelo" "branco"
> class(v)
[1] "character"
```

Operações aritméticas com vetores

- De forma bastante intuitiva, você pode fazer **operações** com vetores.

```
> v <- 1:5
> v
[1] 1 2 3 4 5
> v <- v - 1
> v
[1] 0 1 2 3 4
```

```
> v <- v * 2
> v
[1] 0 2 4 6 8
```

- Caso os vetores tenham a **mesma dimensão**, as operações são feitas elemento a elemento. Vamos calcular o índice de massa corporal de pessoas a partir dos vetores peso e altura. ($\text{imc} = \text{peso} / ((\text{altura})^2)$)

```
> peso <- c(57, 63, 68, 78, 96)
> altura <- c(1.65, 1.90, 1.85, 1.67, 1.73)
> imc <- peso / (altura^2)
> imc
[1] 20.93664 17.45152 19.86852 27.96802 32.07591
```

Outras funções aplicadas a vetores

```
> altura <- c(1.65, 1.90, 1.85, 1.67, 1.73)
> min(altura)
[1] 1.65
> max(altura)
[1] 1.9
> range(altura)
[1] 1.65 1.90
> mean(altura)
[1] 1.76
> var(altura)
[1] 0.0122
> sum(altura)
[1] 8.8
> length(altura)
[1] 5
> sum(altura) / length(altura)
[1] 1.76
```

Instruções comparativas

Instrução	Descrição	Exemplo
<	Menor que.	<pre>v <- 1:5 v < 2</pre>
>	Maior que.	<pre>v <- 1:5 v > 3</pre>
<=	Menor ou igual a.	<pre>v <- c(3, 4, 8, 9) v <= 4</pre>
>=	Maior ou igual a.	<pre>v <- rep(c(3, 4), 2) v >= 3</pre>
==	Igual a.	<pre>v <- seq(-1, 1, 0.5) v == 1</pre>
!=	Diferente de.	<pre>v <- seq(-1, 1, 0.5) v != 1</pre>

Instruções comparativas

- As instruções comparativas verificam elemento por elemento do vetor.

```
> v <- 1:5
> v
[1] 1 2 3 4 5
> comparacao <- v < 2
> comparacao
[1] TRUE FALSE FALSE FALSE FALSE
> comparacao <- v > 3
> comparacao
[1] FALSE FALSE FALSE TRUE TRUE
> comparacao <- v <= 4
> comparacao
[1] TRUE TRUE TRUE TRUE FALSE
> comparacao <- v == 1
> comparacao
[1] TRUE FALSE FALSE FALSE FALSE
> comparacao <- v != 1
> comparacao
[1] FALSE TRUE TRUE TRUE TRUE
```

Instruções lógicas

Instrução	Descrição	Exemplo
&	E.	<code>v <- 1:5</code> <code>(v > 2) & (v < 4)</code>
	Ou.	<code>v <- c(3, 4, 8, 9)</code> <code>(v <= 4) (v == 9)</code>
xor	Ou exclusivo.	<code>v <- seq(-1, 1, 0.5)</code> <code>xor((v == 1), (v > 0))</code>

Instruções lógicas

- As instruções lógicas verificam elemento por elemento do vetor.

```
> v <- 1:5
> v
[1] 1 2 3 4 5
> comparacao <- ((v > 2) & (v < 4))
> comparacao
[1] FALSE FALSE TRUE FALSE FALSE
> v <- c(3, 4, 8, 9)
> v
[1] 3 4 8 9
> comparacao <- ((v <= 4) | (v == 5))
> comparacao
[1] TRUE TRUE FALSE FALSE
> v <- seq(-1, 1, 0.5)
> v
[1] -1.0 -0.5 0.0 0.5 1.0
> xor((v == 1), (v > 0))
[1] FALSE FALSE FALSE TRUE FALSE
```

x	y	&		xor(x,y)
FALSE	FALSE	FALSE	FALSE	FALSE
TRUE	FALSE	FALSE	TRUE	TRUE
FALSE	TRUE	FALSE	TRUE	TRUE
TRUE	TRUE	TRUE	TRUE	FALSE

Subconjuntos de vetores

- Podemos montar **subconjuntos** de um vetor usando: os índices do vetor (número inteiro que indica a posição do elemento no vetor); funções comparativas e lógicas; etc..

```
> v <- c(2, 5, 7, 8, 9, 10, 12, 3, 7, 9)
> v
[1] 2 5 7 8 9 10 12 3 7 9
> a <- v[6]
> a
[1] 10
> b <- v[3:5]
> b
[1] 7 8 9
> c <- v[c(2, 4, 10)]
> c
[1] 5 8 9
```

```
> d <- v[v > 6]
> d
[1] 7 8 9 10 12 7 9
> e <- v[(v > 6) & (v <= 9)]
> e
[1] 7 8 9 7 9
```

Classes diferentes em um mesmo vetor

- Se colocarmos duas ou mais **classes diferentes** dentro de um **mesmo vetor**, o R vai **forçar** que todos os elementos **passem** a pertencer à **mesma classe**.
- A ordem de **precedência** é: character > complex > numeric > integer > logical.

```
> v <- c(0, 1.8, "verde")
> v
[1] "0"      "1.8"    "verde"
> v <- c(TRUE, FALSE, 2)
> v
[1] 1 0 2
> v <- c(TRUE, FALSE, "verde")
> v
[1] "TRUE"   "FALSE"  "verde"
```

Classes diferentes em um mesmo vetor

- Pode-se **coagir** um objeto a ser de uma **classe específica** com as funções **as.character()**, **as.numeric()**, **as.integer()** e **as.logical()**. Isso equivale a algumas funções de conversão de tipo de dados presente em outras linguagens.

```
> v <- 0:5
> v
[1] 0 1 2 3 4 5
> class(v)
[1] "integer"
>
> a <- as.numeric(v)
> a
[1] 0 1 2 3 4 5
> class(a)
[1] "numeric"
```

```
> b <- as.logical(v)
> b
[1] FALSE  TRUE  TRUE  TRUE  TRUE  TRUE
> class(b)
[1] "logical"
>
> c <- as.character(v)
> c
[1] "0" "1" "2" "3" "4" "5"
> class(c)
[1] "character"
```

Classes diferentes em um mesmo vetor

- Se o R não entender como **coagir** uma **classe na outra**, ele levantará um ***warning*** informando que colocou **NA** no lugar.

```
> v <- c(0, 0.5, 1, 2, FALSE, TRUE)
> v
[1] 0.0 0.5 1.0 2.0 0.0 1.0
> a <- as.logical(v)
> a
[1] FALSE TRUE TRUE TRUE FALSE TRUE
>
```

```
> v <- c(1, 3.2, TRUE, "c")
> v
[1] "1"      "3.2"    "TRUE"   "c"
> b <- as.numeric(v)
Warning message:
NAs introduced by coercion
> b
[1] 1.0 3.2 NA NA
>
```

Criação de matrizes

Instrução	Descrição	Exemplo
:	Cria sequências numéricas.	<pre>v <- 1:12 mat <- matrix (v, ncol = 3)</pre>
scan	Lê valores digitados.	<pre>v <- scan() mat <- matrix (v, ncol = 2)</pre>
c	Combina valores.	<pre>v <- c (3, 4, 8, 9) mat <- matrix (v, nrow = 2)</pre>
rep	Repete valores.	<pre>v <- rep (c (3, 4, 5), 2) mat <- matrix (v, nrow = 3)</pre>
seq	Cria sequências numéricas.	<pre>v <- seq (-1, 1.5, 0.5) mat <- matrix (v, ncol = 3)</pre>

Instrução :

- A instrução : cria sequências numéricas.

```
> v <- 1:12
> v
[1] 1 2 3 4 5 6 7 8 9 10 11 12
> mat <- matrix(v, ncol = 3)
> mat
```

	[,1]	[,2]	[,3]
[1,]	1	5	9
[2,]	2	6	10
[3,]	3	7	11
[4,]	4	8	12

A matriz terá 3
colunas.

Por padrão a matriz é
preenchida por
colunas.

byrow = TRUE (por linhas) informa que a
matriz deve ser preenchida por linhas.

```
> v <- 1:12
> v
[1] 1 2 3 4 5 6 7 8 9 10 11 12
> mat <- matrix(v, ncol = 3, byrow = TRUE)
> mat
```

	[,1]	[,2]	[,3]
[1,]	1	2	3
[2,]	4	5	6
[3,]	7	8	9
[4,]	10	11	12

Instrução scan

- A instrução **scan** lê valores digitados.

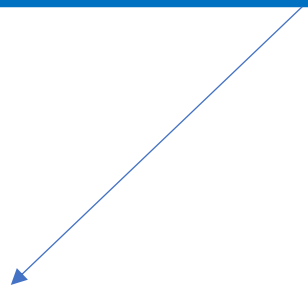
```
> v <- scan()  
1: 1 2 3 4 5 6  
7:
```

Read 6 items

```
> mat <- matrix(v, ncol = 2)  
> mat
```

	[,1]	[,2]
[1,]	1	4
[2,]	2	5
[3,]	3	6

“what” informa o tipo de dado a ser lido (logical, integer, numeric, complex, character).



```
> v <- scan(what = character(), sep = ",")  
1: Gol,Civic,Kicks,Prata,Cinza,Branco  
7:
```

Read 6 items

```
> v  
[1] "Gol"      "Civic"    "Kicks"    "Prata"  
"Cinza"    "Branco"  
> mat <- matrix(v, ncol = 2)  
> mat
```

	[,1]	[,2]
[1,]	"Gol"	"Prata"
[2,]	"Civic"	"Cinza"
[3,]	"Kicks"	"Branco"

Instrução c

- A instrução **c** combina valores.

```
> v <- c(3, 4, 8, 9)
```

```
> v
```

```
[1] 3 4 8 9
```

```
> mat <- matrix(v, nrow = 2)
```

```
> mat
```

	[,1]	[,2]
[1,]	3	8
[2,]	4	9

A matriz terá 2 linhas.

O vetor possui 8 elementos, mas a matriz somente 6. Um *warning* é levantado pelo R informando possível problema.

```
> v <- c(3, 4, 8, 9, 11, 15, 18, 33)
```

```
> v
```

```
[1] 3 4 8 9 11 15 18 33
```

```
> mat <- matrix(v, nrow = 2, ncol = 3)
```

Warning message:

In matrix(v, nrow = 2, ncol = 3) :

data length [8] is not a sub-multiple or multiple of the number of columns [3]

```
> mat
```

	[,1]	[,2]	[,3]
[1,]	3	8	11
[2,]	4	9	15

Instrução **rep**

- A instrução **rep** retorna o primeiro argumento, repetido o número de vezes indicado pelo segundo argumento.

```
> v <- rep(c(3, 4, 5), 2)
> v
[1] 3 4 5 3 4 5
> mat <- matrix(v, nrow = 3)
> mat
```

	[,1]	[,2]
[1,]	3	3
[2,]	4	4
[3,]	5	5

A função **summary()** opera em cada coluna da matriz como se fossem vetores apresentando um resumo de cada uma.

```
> summary(mat)
```

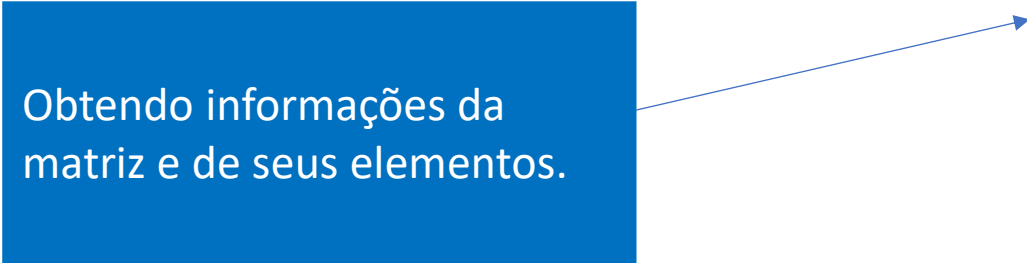
	V1	V2
Min.	:3.0	Min. :3.0
1st Qu.:	3.5	1st Qu.:3.5
Median	:4.0	Median :4.0
Mean	:4.0	Mean :4.0
3rd Qu.:	4.5	3rd Qu.:4.5
Max.	:5.0	Max. :5.0

Instrução **seq**

- A instrução **seq** cria sequências numéricas tendo como argumentos o início, o fim e os passos da seqüência.

```
> v <- seq(-1, 1.5, 0.5)
> v
[1] -1.0 -0.5  0.0  0.5  1.0  1.5
> mat <- matrix(v, ncol = 3)
> mat
      [,1] [,2] [,3]
[1,] -1.0  0.0  1.0
[2,] -0.5  0.5  1.5
```

Obtendo informações da
matriz e de seus elementos.



```
> dim(mat)
[1] 2 3
> length(mat)
[1] 6
> min(mat)
[1] -1
> max(mat)
[1] 1.5
> mean(mat)
[1] 0.25
> sd(mat)
[1] 0.9354143
```

Adicionando linhas e colunas

A função **rbind()** acrescentou uma nova linha na matriz formada pelos números 18, 20 e 35.

```
> v <- 1:12
> v
 [1]  1  2  3  4  5  6  7  8  9 10 11 12
> mat <- matrix(v, ncol = 3)
> mat
      [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
```

```
> mais.linha <- rbind(mat, c(18, 20, 35))
```

```
> mais.linha
```

```
      [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
[5,]   18   20   35
```

A função **cbind()** acrescentou uma nova coluna na matriz formada por uma sequência de 1 a 4.

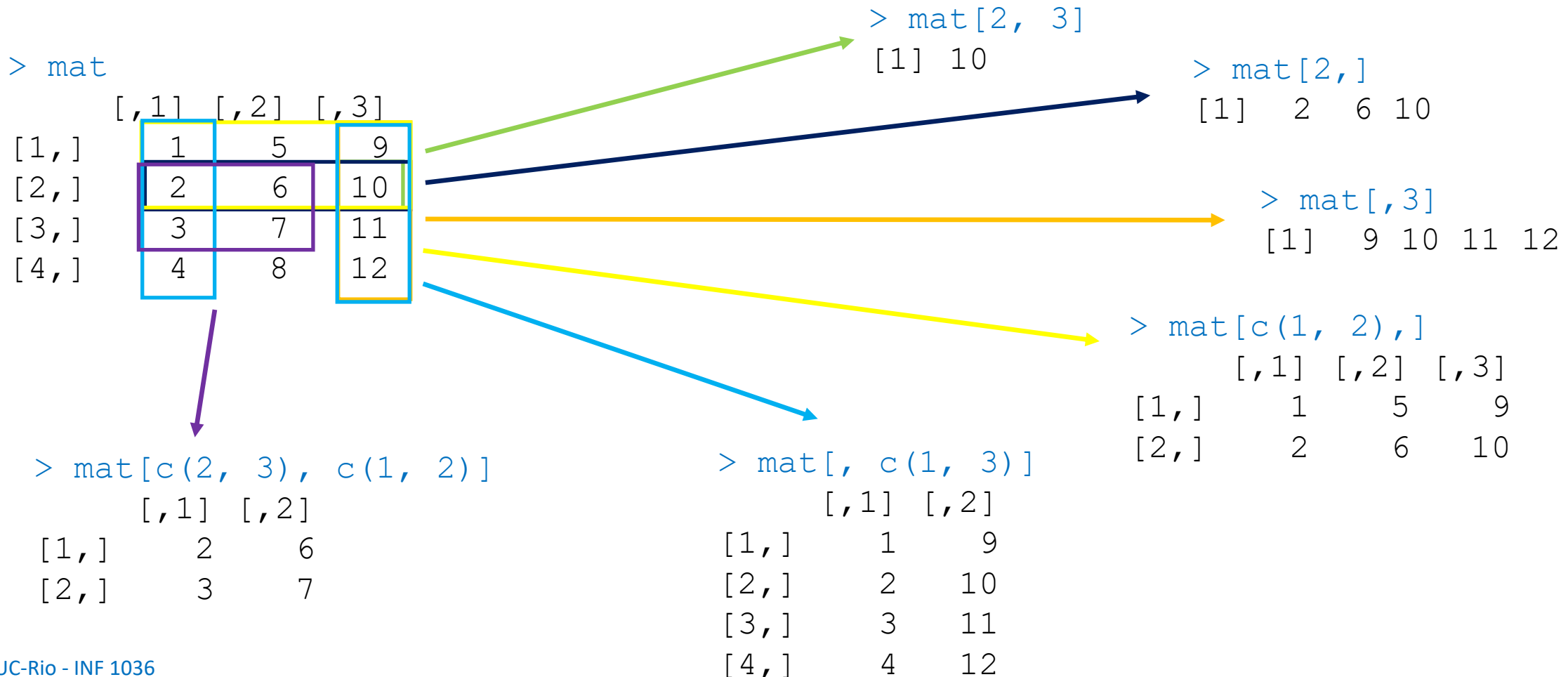
```
> mais.coluna <- cbind(mat, 1:4)
```

```
> mais.coluna
```

```
      [,1] [,2] [,3] [,4]
[1,]    1    5    9    1
[2,]    2    6   10    2
[3,]    3    7   11    3
[4,]    4    8   12    4
```

Índices de matrizes

- Como nos vetores, os **colchetes [linha, coluna]** podem ser utilizados para extrair partes de uma matriz.



Instruções comparativas

Instrução	Descrição	Exemplo
<	Menor que.	<pre>mat <- matrix (1:12, ncol = 3) mat < 5</pre>
>	Maior que.	<pre>mat <- matrix (1:12, ncol = 3) mat [, 2] > 3</pre>
<=	Menor ou igual a.	<pre>mat <- matrix (1:12, ncol = 3) mat [1, 3] <= 4</pre>
>=	Maior ou igual a.	<pre>mat <- matrix (1:12, ncol = 3) mat [1,] >= 3</pre>
==	Igual a.	<pre>mat <- matrix (1:12, ncol = 3) mat == 1</pre>
!=	Diferente de.	<pre>mat <- matrix (1:12, ncol = 3) mat != 1</pre>

Instruções comparativas

- A instruções comparativas verificam elemento por elemento da matriz.

```
> mat <- matrix(1:12, ncol = 3)
```

```
> mat
```

	[,1]	[,2]	[,3]
[1,]	1	5	9
[2,]	2	6	10
[3,]	3	7	11
[4,]	4	8	12

```
> ma <- mat < 5
```

```
> ma
```

	[,1]	[,2]	[,3]
[1,]	TRUE	FALSE	FALSE
[2,]	TRUE	FALSE	FALSE
[3,]	TRUE	FALSE	FALSE
[4,]	TRUE	FALSE	FALSE

```
> mb <- mat[, 2] > 3
```

```
> mb
```

[1]	TRUE	TRUE	TRUE	TRUE
-----	------	------	------	------

```
> mc <- mat[1, 3] <= 4
```

```
> mc
```

[1]	FALSE
-----	-------

```
> md <- mat[1, ] >= 3
```

```
> md
```

[1]	FALSE	TRUE	TRUE
-----	-------	------	------

```
> me <- mat == 1
```

```
> me
```

	[,1]	[,2]	[,3]
[1,]	TRUE	FALSE	FALSE
[2,]	FALSE	FALSE	FALSE
[3,]	FALSE	FALSE	FALSE
[4,]	FALSE	FALSE	FALSE

Instruções lógicas

Instrução	Descrição	Exemplo
&	E.	<pre>mat <- matrix (1:12, ncol = 3) (mat > 2) & (mat < 4) (mat [, 3] <= 10) & (mat [, 2] == 9)</pre>
	Ou.	<pre>mat <- matrix (1:12, ncol = 3) (mat [, 2] <= 6) (mat [, 2] == 9)</pre>
xor	Ou exclusivo.	<pre>mat <- matrix (1:12, ncol = 3) xor((mat == 1), (mat > 0))</pre>

Instruções lógicas

- As instruções lógicas verificam elemento por elemento da matriz.

```
> mat <- matrix(1:12, ncol = 3)
> mat
```

```
      [,1] [,2] [,3]
[1,]     1     5     9
[2,]     2     6    10
[3,]     3     7    11
[4,]     4     8    12
```

```
> ma <- (mat > 2)
> ma
```

```
      [,1] [,2] [,3]
[1,] FALSE TRUE TRUE
[2,] FALSE TRUE TRUE
[3,]  TRUE TRUE TRUE
[4,]  TRUE TRUE TRUE
```

```
> mb <- (mat > 2) & (mat < 4)
> mb
```

```
      [,1] [,2] [,3]
[1,] FALSE FALSE FALSE
[2,] FALSE FALSE FALSE
[3,]  TRUE FALSE FALSE
[4,] FALSE FALSE FALSE
```

```
> mc <- (mat[, 2] <= 6) | (mat[, 2] == 9)
> mc
```

```
[1]  TRUE  TRUE FALSE FALSE
```

```
> md <- (mat[, 3] <= 11) & (mat[, 3] > 9)
> md
```

```
[1] FALSE  TRUE  TRUE FALSE
```

Outras operações com matrizes

```
> mat1 <- matrix(c(1,1,1,0,3,2,0,1,0), ncol = 3)
```

```
> mat1
```

	[,1]	[,2]	[,3]
[1,]	1	0	0
[2,]	1	3	1
[3,]	1	2	0

```
> transposta <- t(mat1)
```

```
> transposta
```

	[,1]	[,2]	[,3]
[1,]	1	1	1
[2,]	0	3	2
[3,]	0	1	0

```
> inversa <- solve(mat1)
```

```
> inversa
```

	[,1]	[,2]	[,3]
[1,]	1.0	0	0.0
[2,]	-0.5	0	0.5
[3,]	0.5	1	-1.5

As operações matemáticas são aplicadas a todos os elementos da matriz.

Matriz transposta - função `t()` - e matriz inversa - função `solve()`.

Produto de matrizes (`%*%`).

```
> divisao <- mat1 / 2
```

```
> divisao
```

	[,1]	[,2]	[,3]
[1,]	0.5	0.0	0.0
[2,]	0.5	1.5	0.5
[3,]	0.5	1.0	0.0

```
> mat2 <- matrix(1:9, nrow = 3)
```

```
> mat2
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

```
> produto <- mat1 %*% mat2
```

```
> produto
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	10	25	40
[3,]	5	14	23

Listas

- **Listas** são um tipo especial de **vetor** que aceita elementos de classes diferentes.
- Uma **lista** é criada com a função **list()**, que aceita um número arbitrário de elementos, sendo que uma **lista** pode também conter outra **lista**.

```
lancamento <- list(  
  mes = "Jan",  
  valores = list(  
    pessoal = 3340.00,  
    aluguel = 256.00  
  ),  
  filial = "01"  
)
```

```
> lancamento  
$mes  
[1] "Jan"  
  
$valores  
$valores$pessoal  
[1] 3340  
  
$valores$aluguel  
[1] 256  
  
$filial  
[1] "01"
```

Listas

- **Listas** são um tipo especial de **vetor** que aceita elementos de classes diferentes.
- Uma **lista** é criada com a função **list()**, que aceita um número arbitrário de elementos, sendo que uma **lista** pode também conter outra **lista**.

```
lancamento <- list(
  mes = "Jan",
  valores = list(
    pessoal = 3340.00,
    aluguel = 256.00
  ),
  filial = "01"
)
```

```
> lancamento$mes
[1] "Jan"
> lancamento$valores
$pessoal
[1] 3340
$aluguel
[1] 256
> lancamento$valores$pessoal
[1] 3340
> lancamento$valores$aluguel
[1] 256
```

Arrays

- **Arrays** são a classe generalizada das **matrizes** e **vetores** e possuem até 3 dimensões que podem ser *numeric*, *character*, *complex* ou *logical*.
- Para acessar um elemento de um **array**, caso ele tenha 3 dimensões, a primeira coordenada representa a **linha**, a segunda coordenada representa a **coluna** e a terceira, a **página**.

```
> dados.array <- array(1:6, c(3, 2))
```

```
> dados.array  
      [,1] [,2]  
[1,]    1    4  
[2,]    2    5  
[3,]    3    6
```

```
> dados.array[1, ]
```

```
[1] 1 4
```

```
> dados.array[, 1]
```

```
[1] 1 2 3
```

```
> dados.array[3, 2]
```

```
[1] 6
```

```
> dados.array[1, ][2]
```

```
[1] 4
```

```
> dados.array[3, 2][1]
```

```
[1] 6
```

Arrays

```
> dados.array <- array(1:24, c(3, 4, 2))
```

```
> dados.array
```

```
, , 1
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	7	10
[2,]	2	5	8	11
[3,]	3	6	9	12

```
, , 2
```

	[,1]	[,2]	[,3]	[,4]
[1,]	13	16	19	22
[2,]	14	17	20	23
[3,]	15	18	21	24

```
> dados.array[1, , ]
```

	[,1]	[,2]
[1,]	1	13
[2,]	4	16
[3,]	7	19
[4,]	10	22

```
> dados.array[, 1, ]
```

	[,1]	[,2]
[1,]	1	13
[2,]	2	14
[3,]	3	15

```
> dados.array[, , 1]
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	7	10
[2,]	2	5	8	11
[3,]	3	6	9	12

```
> dados.array[2, 3, 1]
```

```
[1] 8
```

Data frames

- Um **data frame** é constituído pela concatenação de várias **colunas** assim como as **matrizes**.
- Assim como uma **matriz**, todas as **colunas** de um **data frame** têm que ter o **mesmo número de elementos**, mas, ao contrário de uma matriz, um data frame pode conter colunas de **tipos diferentes**.
- Um **data frame** é semelhante a uma **tabela** de um banco de dados relacional ou uma **planilha** do Excel, sendo objetos muito importantes na solução de diversos tipos de problemas de análise de dados.

```
> acao <- c("RADL3", "HYPE3", "USIM5", "BRAP4", "VALE3")  
> quantidade <- c(100, 20, 35, 40, 90)  
> valor <- c(77.28, 27.71, 6.98, 32.03, 54.97)  
> investimento <- data.frame(acao, quantidade, valor)
```

```
> investimento  
      acao quantidade  valor  
1 RADL3           100  77.28  
2 HYPE3            20  27.71  
3 USIM5            35   6.98  
4 BRAP4            40  32.03  
5 VALE3            90  54.97
```

Acesso aos dados

- Os elementos de um **data frame** podem ser **acessados e modificados** como em uma **matriz**.



```
> investimento
  acao quantidade valor
1 RADL3         100  77.28
2 HYPE3          20  27.71
3 USIM5          35   6.98
4 BRAP4          40  32.03
5 VALE3          90  54.97
```

```
> investimento[, 2]
[1] 100  20  35  40  90
>
> investimento[1, ]
  acao quantidade valor
1 RADL3         100  77.28
>
> investimento[2, 3]
[1] 27.71
>
> investimento[2, c("acao", "quantidade")]
  acao quantidade
2 HYPE3          20
```



```
> investimento[2, 3] <- 15.0
> investimento
  acao quantidade valor
1 RADL3         100  77.28
2 HYPE3          20  15.00
3 USIM5          35   6.98
4 BRAP4          40  32.03
5 VALE3          90  54.97
```

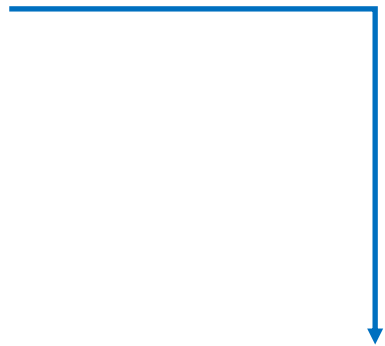
Acesso aos dados

- As colunas também podem ser selecionadas usando os símbolos \$ e [[]]. Ambos retornam um vetor como resultado.

```
> investimento
  acao quantidade valor
1 RADL3         100 77.28
2 HYPE3          20 27.71
3 USIM5          35  6.98
4 BRAP4          40 32.03
5 VALE3          90 54.97
```



```
> investimento$quantidade
[1] 100  20  35  40  90
```



```
> investimento[\"valor\"]
[1] 77.28 15.00  6.98 32.03 54.97
```

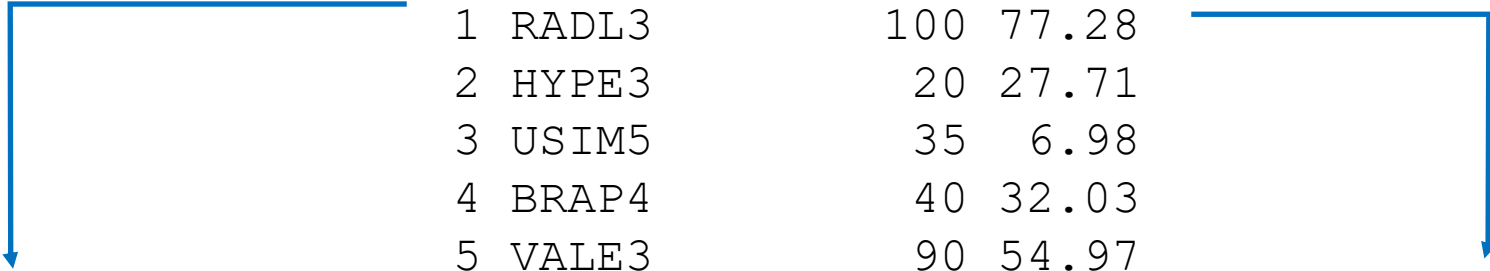
Quais são os tipos de objetos retornados por `investimento$valor`, `investimento[\"valor\"]` e `investimento[\"valor\"]` ?

Acesso aos dados

- Tanto o `$` quanto o `[[ ]]` sempre retornam um **vetor** como resultado. Se você quiser garantir que o resultado da seleção seja um **data frame**, use **drop = FALSE** ou selecione sem a vírgula.

```
> investimento
```

	acao	quantidade	valor
1	RADL3	100	77.28
2	HYPE3	20	27.71
3	USIM5	35	6.98
4	BRAP4	40	32.03
5	VALE3	90	54.97



```
> investimento[, "quantidade", drop = FALSE]
```

```
quantidade
```

1	100
2	20
3	35
4	40
5	90

```
> investimento["valor"]
```

```
valor
```

1	77.28
2	15.00
3	6.98
4	32.03
5	54.97

Adicionando linhas

```
> investimento
  acao quantidade valor
1 RADL3         100  77.28
2 HYPE3          20  27.71
3 USIM5          35   6.98
4 BRAP4          40  32.03
5 VALE3          90  54.97
```



```
> investimento <- rbind(investimento, data.frame(acao = "ITUB4", quantidade = 55, valor = 42.09))
> investimento
```

```
  acao quantidade valor
1 RADL3         100  77.28
2 HYPE3          20  27.71
3 USIM5          35   6.98
4 BRAP4          40  32.03
5 VALE3          90  54.97
6 ITUB4          55  42.09
```

A função **rbind()** acrescentou uma nova linha no data frame.

Removendo linhas

Remove as linhas 2 e 4 do data frame.

```
> investimento
  acao quantidade valor
1 RADL3         100 77.28
2 HYPE3          20 27.71
3 USIM5          35  6.98
4 BRAP4          40 32.03
5 VALE3          90 54.97
```

Remove as linhas com valor ≤ 7 (ou seleciona somente as linhas com valor > 7).

```
> investimento <- investimento[c(-2, -4), ]
```

```
> investimento
```

```
  acao quantidade valor
1 RADL3         100 77.28
3 USIM5          35  6.98
5 VALE3          90 54.97
```

```
> investimento <- investimento[valor > 7, ]
```

```
> investimento
```

```
  acao quantidade valor
1 RADL3         100 77.28
2 HYPE3          20 27.71
4 BRAP4          40 32.03
5 VALE3          90 54.97
```

Adicionando colunas

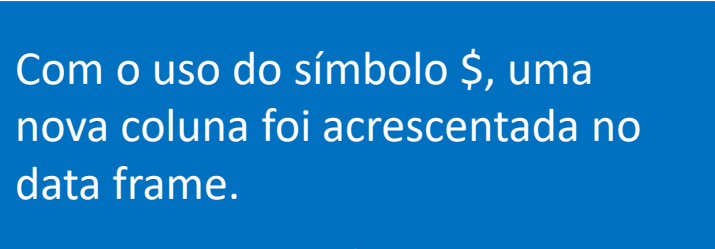
A função **cbind()** acrescentou uma nova coluna no data frame.

```
> investimento
  acao quantidade valor
1 RADL3         100  77.28
2 HYPE3          20  27.71
3 USIM5          35   6.98
4 BRAP4          40  32.03
5 VALE3          90  54.97
```

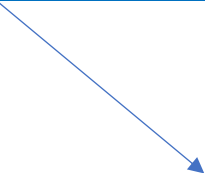
```
> investimento <- cbind(investimento, variacao = c(0.2, -0.3, 0.5, -0.8, 0.1))
> investimento
  acao quantidade valor variacao
1 RADL3         100  77.28      0.2
2 HYPE3          20  27.71     -0.3
3 USIM5          35   6.98      0.5
4 BRAP4          40  32.03     -0.8
5 VALE3          90  54.97      0.1
```

Adicionando colunas

Com o uso do símbolo \$, uma nova coluna foi acrescentada no data frame.



```
> investimento
  acao quantidade valor
1 RADL3         100 77.28
2 HYPE3          20 27.71
3 USIM5          35  6.98
4 BRAP4          40 32.03
5 VALE3          90 54.97
```



```
> investimento$variacao <- c(0.2, -0.3, 0.5, -0.8, 0.1)
> investimento
```

```
  acao quantidade valor variacao
1 RADL3         100 77.28      0.2
2 HYPE3          20 27.71     -0.3
3 USIM5          35  6.98      0.5
4 BRAP4          40 32.03     -0.8
5 VALE3          90 54.97      0.1
```

Removendo colunas

Remove a coluna valor do data frame.

```
> investimento
  acao quantidade valor
1 RADL3         100 77.28
2 HYPE3          20 27.71
3 USIM5          35  6.98
4 BRAP4          40 32.03
5 VALE3          90 54.97
```

Remove a coluna valor do data frame.

```
> investimento$valor <- NULL
```

```
> investimento
```

```
  acao quantidade
1 RADL3         100
2 HYPE3          20
3 USIM5          35
4 BRAP4          40
5 VALE3          90
```

```
> investimento <- investimento[, -3]
```

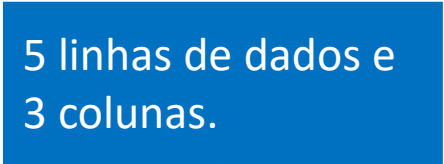
```
> investimento
```

```
  acao quantidade
1 RADL3         100
2 HYPE3          20
3 USIM5          35
4 BRAP4          40
5 VALE3          90
```

Estrutura

- A função **str** permite visualizar a **estrutura** de um **data frame**, e a função **names**, o nome das colunas.

5 linhas de dados e
3 colunas.



```
> str(investimento)
'data.frame': 5 obs. of  3 variables:
 $ acao      : chr  "RADL3" "HYPE3" "USIM5" "BRAP4" ...
 $ quantidade: num  100 20 35 40 90
 $ valor     : num  77.28 27.71 6.98 32.03 54.97
>
> names(investimento)
[1] "acao"          "quantidade"    "valor"
```

Aplicando funções no data frame

- A função **sapply** aplica uma função nas **colunas** de um **data frame**.

```
> investimento
  acao quantidade valor
1 RADL3         100  77.28
2 HYPE3          20  27.71
3 USIM5          35   6.98
4 BRAP4          40  32.03
5 VALE3          90  54.97
```

```
> minimo <- sapply(investimento["quantidade"], min)
> minimo
quantidade
          20
> minimo[[1]]
[1] 20
>
> maximo <- sapply(investimento[3], max)
> maximo
valor
77.28
>
> media <- sapply(investimento[2:3], mean)
> media
quantidade      valor
      57.000      39.794
> media[[1]]
[1] 57
> media[[2]]
[1] 39.794
```

Unindo data frames

- A função **merge** permite unir **data frames**.

```
> investimento
```

	acao	quantidade	valor
1	RADL3	100	77.28
2	HYPE3	20	27.71
3	USIM5	35	6.98
4	BRAP4	40	32.03
5	VALE3	90	54.97

```
> acao <- c("ITUB4", "PTRE2")
```

```
> quantidade <- c(10, 65)
```

```
> valor <- c(22.28, 10.71)
```

```
>
```

```
> novo <- data.frame(acao, quantidade, valor)
```

```
> novo
```

	acao	quantidade	valor
1	ITUB4	10	22.28
2	PTRE2	65	10.71

```
> investimento.final <- merge(investimento, novo, all = TRUE)
```

```
> investimento.final
```

	acao	quantidade	valor
1	BRAP4	40	32.03
2	HYPE3	20	27.71
3	ITUB4	10	22.28
4	PTRE	65	10.71
5	RADL3	100	77.28
6	USIM5	35	6.98
7	VALE3	90	54.97

O parâmetro `all = TRUE` faz com que o merge una os dois data frames. Se existirem linhas iguais nos dois data frames, somente uma será considerada.

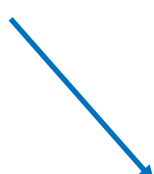
Se o parâmetro `all` não for declarado, somente as linhas que aparecem em ambos os data frames serão consideradas.

Unindo data frames

- A função **merge** permite unir **data frames**.

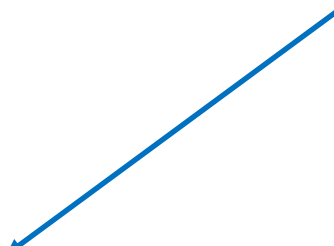
```
> codigo <- c("GFX6", "ELT6", "GLO5", "KLM1")
> compra <- c(1.01, 0.30, 1.26, 1.82)
>
> acao.compra <- data.frame(codigo, compra)
> acao.compra
```

	codigo	compra
1	GFX6	1.01
2	ELT6	0.30
3	GLO5	1.26
4	KLM1	1.82



```
> identificador <- c("ELT6", "GLO5", "GFX6", "KLM1")
> venda <- c(0.98, 0.78, 1.23, 1.95)
>
> acao.venda <- data.frame(identificador, venda)
> acao.venda
```

	identificador	venda
1	ELT6	0.98
2	GLO5	0.78
3	GFX6	1.23
4	KLM1	1.95



```
> acao <- merge(acao.compra, acao.venda, by.x = "codigo", by.y = "identificador")
> acao
```

	codigo	compra	venda
1	ELT6	0.30	0.98
2	GFX6	1.01	1.23
3	GLO5	1.26	0.78
4	KLM1	1.82	1.95

Os data frames `acao.compra` e `acao.venda` foram unidos pela coluna `codigo` e `identificador`, mas somente a coluna `codigo` foi incluída no data frame `acao`.

Os vetores `codigo` e `identificador` não precisam estar na mesma ordem.

Filtrando data frames

- Mecanismo padrão.

```
> filtrado <- investimento[investimento$quantidade >= 70, ]  
> filtrado  
  acao quantidade valor  
1 RADL3          100 77.28  
5 VALE3           90 54.97
```

- A função **filter** do pacote **dplyr** que permite **selecionar linhas** específicas dos **data frames**.

```
> library(dplyr)  
> filtrado <- filter(investimento, quantidade >= 70)  
> filtrado  
  acao quantidade valor  
1 RADL3          100 77.28  
2 VALE3           90 54.97  
> filtrado <- filter(investimento, (quantidade >= 70) & (valor < 60))  
> filtrado  
  acao quantidade valor  
1 VALE3           90 54.97
```

A função **filter()** gera um data frame como retorno.

Ordenando data frames

- Mecanismo padrão com **order**.

```
> ordenado <- investimento[order (investimento$acao), ]  
> ordenado  
  acao quantidade valor  
4 BRAP4          40 32.03  
2 HYPE3          20 27.71  
1 RADL3         100 77.28  
3 USIM5          35  6.98  
5 VALE3          90 54.97
```

Para colocar em ordem decrescente, use o argumento `decreasing = TRUE`.

- A função **arrange** do pacote **dplyr** permite **ordenar** os **data frames**.

```
> library(dplyr)  
> ordenado <- arrange(investimento, quantidade)  
> ordenado  
  acao quantidade valor  
1 HYPE3          20 27.71  
2 USIM5          35  6.98  
3 BRAP4          40 32.03  
4 VALE3          90 54.97  
5 RADL3         100 77.28
```

Para ordenar por quantidade e valor, use: `arrange(investimento, quantidade, valor)`.

Para colocar em ordem decrescente, use: `arrange(investimento, - quantidade)`.

Outras funções aplicadas a data frames

- A função **select** do pacote **dplyr** permite **selecionar colunas dos data frames**.

```
> library(dplyr)
> selecionado <- select(investimento, acao, quantidade)
> selecionado
```

	acao	quantidade
1	RADL3	100
2	HYPE3	20
3	USIM5	35
4	BRAP4	40
5	VALE3	90

Somente as colunas ação e quantidade são selecionadas.

Todas as colunas de ação a valor são selecionadas.

```
> selecionado <- select(investimento, acao:valor)
> selecionado
```

	acao	quantidade	valor
1	RADL3	100	77.28
2	HYPE3	20	27.71
3	USIM5	35	6.98
4	BRAP4	40	32.03
5	VALE3	90	54.97

```
> selecionado <- select(investimento, - quantidade)
> selecionado
```

	acao	valor
1	RADL3	77.28
2	HYPE3	27.71
3	USIM5	6.98
4	BRAP4	32.03
5	VALE3	54.97

Todas as colunas, exceto quantidade, são selecionadas.

Outras funções aplicadas a data frames

- A função **mutate** do pacote **dplyr** permite **criar** novas **colunas** nos **data frames**.

```
> library(dplyr)
> investimento <- mutate(investimento, investido = quantidade * valor)
> investimento
```

	acao	quantidade	valor	investido
1	RADL3	100	77.28	7728.0
2	HYPE3	20	27.71	554.2
3	USIM5	35	6.98	244.3
4	BRAP4	40	32.03	1281.2
5	VALE3	90	54.97	4947.3



Para incluir no data frame a coluna investido, use:
`investimento <- mutate (investimento, investido = quantidade * valor).`

Outras funções aplicadas a data frames

- A função **mutate** do pacote **dplyr** permite **criar novas colunas** nos **data frames**.

```
> library(dplyr)
> investimento <- mutate(investimento, a.investir = quantidade * valor,
investir = ifelse(quantidade < 50, "Sim", "Não"))
> investimento
```

	acao	quantidade	valor	a.investir	investir
1	RADL3	100	77.28	7728.0	Não
2	HYPE3	20	27.71	554.2	Sim
3	USIM5	35	6.98	244.3	Sim
4	BRAP4	40	32.03	1281.2	Sim
5	VALE3	90	54.97	4947.3	Não

A coluna a.investir foi adicionada ao data frame e a coluna investir foi criada a partir de uma instrução ifelse aplicada à coluna quantidade.

Importando arquivo CSV

- Arquivos **CSV** (*Comma Separated Values*) são arquivos textos sem formatação alguma (negrito, itálico, etc) que podem ser visualizados por qualquer editor de textos sendo muito utilizados por diversas ferramentas para exportar seus dados ou mesmo para importar novos dados.
- Consiste em uma sequência de linhas onde cada linha representa um registro de dados. Dentro de cada linha, as informações das colunas (campos) são separadas por “,”.

Arquivo CSV com cinco linhas e quatro colunas contendo as colunas sigla, nome, população e PIB (mil R\$) de alguns estados brasileiros.



```
AC,Acre,732793,8477
AL,Alagoas,3120922,24575
AP,Amapá,668689,8266
AM,Amazonas,3480937,59779
BA,Bahia,14021432,154340
```

Importando arquivo CSV

- Na prática, em função da vírgula apresentar diversas funções, outros padrões, como “;” e “TAB”, também são utilizados para separar as informações das colunas.
- Há muitos pacotes para leitura de arquivos **CSV** como o **readr**, **vroom**, e o **data.table**, sendo que o R também apresenta funções nativas como a **read.csv()** e a **read.csv2()**.
- O comando **help** permite obter maiores detalhes das funções **read.csv()** e **read.csv2()**.

```
read.csv(file, header = TRUE, sep = ",", quote = "\"",  
         dec = ".", fill = TRUE, comment.char = "", ...)
```

```
read.csv2(file, header = TRUE, sep = ";", quote = "\"",  
          dec = ",", fill = TRUE, comment.char = "", ...)
```

Importando arquivo CSV

- Antes de importarmos um arquivo **CSV**, é necessário conhecer algumas informações sobre o mesmo, como:
 - qual caractere é utilizado para separar as colunas;
 - se o delimitador das casas decimais é “,” ou “.”.
 - se existe algum caractere especial (aspas simples ou duplas, por exemplo) limitando os dados em cada coluna;
 - se os nomes das colunas estão na primeira linha do arquivo;
 - se o arquivo possui linhas iniciais e finais que precisam ser ignoradas.
- A função **file.head()** do pacote **descr**, que pode ser instalado pelo comando **install.packages("descr")**, permite identificar algumas dessas informações.
- O comando **read_lines()** do pacote **readr**, que pode ser instalado pelo comando **install.packages("readr")**, também pode ser utilizado.

Importando arquivo CSV

- Uso da função **file.head()** do pacote **descr**.

```
> library(descr)
> file.head("PIB2.CSV")
estado;descricao;populacao;pib
"AC";"Acre";"732793";"8477"
"AL";"Alagoas";"3120922";"24575"
"AP";"Amapá";"668689";"8266"
```

- A função nativa **read.csv2()** permite **importar** um arquivo no formato **CSV**.

```
> indicador <- read.csv2("PIB2.CSV", header = TRUE, sep = ";", quote = "'", dec = ",")
```

```
> indicador
```

	estado	descricao	populacao	pib
1	AC	Acre	732793	8477
2	AL	Alagoas	3120922	24575
3	AP	Amapá	668689	8266
4	AM	Amazonas	3480937	59779

O comando simples `read.csv2("PIB2.CSV")` também funcionaria neste caso, devido aos valores padrão dos parâmetros da função `read.csv2()`.

Importando arquivo CSV

- A função nativa **read.csv2()** retorna os dados em um objeto do tipo **data.frame** (detalhado no material 6) que, de forma simplificada, consiste em um tipo de dado que mantém os dados organizados em linhas e colunas, como uma tabela, podendo ter colunas com tipos diferentes.

```
> class(indicador)
[1] "data.frame"
```

- A função nativa **summary()** pode ser usada para conferir se a importação foi realizada com sucesso.

```
> summary(indicador)
```

estado	descricao	populacao	pib
Length:27	Length:27	Min. : 451227	Min. : 6341
Class :character	Class :character	1st Qu.: 2506152	1st Qu.: 24254
Mode :character	Mode :character	Median : 3512672	Median : 59779
		Mean : 7064174	Mean : 139633
		3rd Qu.: 8622044	3rd Qu.: 151194
		Max. :41252160	Max. :1247596

Importando arquivo CSV

- As funções nativas **head()** e o **tail()** são úteis para visualizar os primeiros e os últimos registros dos dados importados, respectivamente.
- É sempre muito importante observar os dados após a importação, pois, essa prática ajuda a identificar erros básicos.

```
> head(indicador)
```

	estado	descricao	populacao	pib
1	AC	Acre	732793	8477
2	AL	Alagoas	3120922	24575
3	AP	Amapá	668689	8266
4	AM	Amazonas	3480937	59779
5	BA	Bahia	14021432	154340
6	CE	Ceara	8448055	77865

```
> tail(indicador)
```

	estado	descricao	populacao	pib
22	RO	Rondônia	1560501	23561
23	RR	Roraima	451227	6341
24	SC	Santa Catarina	6249682	152482
25	SP	São Paulo	41252160	1247596
26	SE	Sergipe	2068031	23932
27	TO	Tocantins	1383453	17240

Importando arquivo CSV

- Para verificar a estrutura do objeto, ou seja, seus campos e tipos de dados, também pode ser utilizada a função **glimpse** do pacote **dplyr**.

```
> glimpse(indicador)
```

```
Rows: 27
```

```
Columns: 4
```

```
$ estado    <chr> "AC", "AL", "AP", "AM", "BA",...
```

```
$ descricao <chr> "Acre", "Alagoas", "Amapá", "...
```

```
$ populacao <int> 732793, 3120922, 668689, 3480...
```

```
$ pib       <int> 8477, 24575, 8266, 59779, 154...
```

Importando arquivo CSV

- O pacote **readr** possui a função **read_csv2()** que permite carregar arquivos **CSV** em uma estrutura chamada **tibble**, que é um tipo especial de **data frame** que apresenta mais recursos para organização e manipulação de dados.
- Se os **dados a serem importados** estiverem em uma planilha do Microsoft Excel (**XLS**), algumas alternativas são:
 - **abrir** o arquivo no Excel e **salvar** a planilha como arquivo do tipo **CSV**.
 - usar a função **read_xlsx()** do pacote **readxl**, integrante do pacote **Tidyverse**.

Importando arquivo com colunas de largura fixa

- São arquivos texto (**TXT**), como os arquivos **CSV**, mas sem delimitadores de coluna.
- Para sua importação, é preciso saber quantos caracteres cada coluna ocupa.
- Arquivos desse tipo não são muito comuns atualmente, mas muitas bases antigas e importantes estão disponíveis neste formato.
- Normalmente estes arquivos são acompanhados de um dicionário (tabela ou mesmo um texto) indicando a posição inicial e final de cada coluna, seus rótulos e os valores das colunas categóricas.

ACAcre	732793	8477
ALAlagoas	3120922	24575
APAmapá	668689	8266
AMAmazonas	3480937	59779
BABahia	14021432	154340

Importando arquivo com colunas de largura fixa

- Uso da função **file.head()** do pacote **descr**.

```
> library(descr)
> file.head("PIB3.TXT")
ACAcre          732793      8477
ALAlagoas       3120922    24575
APAmapá         668689     8266
AMAmazonas     3480937    59779
BABahia        14021432   154340
```

- A função nativa **read.fwf()** permite **importar** um arquivo no formato **TXT**.

```
> indicador <- read.fwf("PIB3.TXT", col.names = c("estado", "descricao",
"populacao", "pib"), widths = c(2, 19, 8, 7), colClasses = c("character",
"character", rep("numeric", 2)))
> indicador
```

	estado	descricao	populacao	pib
1	AC Acre		732793	8477
2	AL Alagoas		3120922	24575
3	AP Amapá		668689	8266

Importando arquivo com colunas de largura fixa

- A função interna **read.fwf()** retorna os dados em um objeto do tipo **data.frame** e, desta forma, as funções **summary()**, **head()**, **tail()**, **glimpse()** também se aplicam.
- O pacote **readr** possui a função **read_fwf()** que permite carregar arquivos **TXT** em estruturas **tibble**.

Importando arquivo JSON

- **JSON** (JavaScript Object Notation - Notação de Objetos JavaScript) é uma formatação leve de troca de dados. Para seres humanos, é fácil de ler e escrever e, para computadores, é fácil de interpretar e gerar.
- **JSON** lembra o padrão **XML** (é texto simples, é autodescritivo, é hierárquico - valores dentro de valores) mas é diferente da **XML** (é mais curto e simples, é mais rápido para processar, não possui palavras reservadas, etc).
- As principais regras de sintaxe **JSON** são:
 - dados **JSON** estão definidos aos pares no formato: **nome : valor**.
 - os dados são separados por vírgulas (,).
 - as chaves **{}** contêm objetos.
 - os colchetes **[]** expressam **matrizes/vetores**.

Importando arquivo JSON

```
{
  "estados":
    [
      { "sigla":"RJ" , "descricao":"Rio de Janeiro" },
      { "sigla":"SP" , "descricao":"São Paulo" },
      { "sigla":"ES" , "descricao":"Espírito Santo" }
    ]
}
```

```
> library(RJSONIO)
> lista <- fromJSON("estados.json")
> lista
$estados
$estados[[1]]
  sigla descricao
  "RJ" "Rio de Janeiro"
$estados[[2]]
  sigla descricao
  "SP" "São Paulo"
$estados[[3]]
  sigla descricao
  "ES" "Espírito Santo"
```

```
> lista$estados
[[1]]
  sigla descricao
  "RJ" "Rio de Janeiro"
[[2]]
  sigla descricao
  "SP" "São Paulo"
[[3]]
  sigla descricao
  "ES" "Espírito Santo"
> lista$estados[[1]]
  sigla descricao
  "RJ" "Rio de Janeiro"
> lista$estados[[1]][['sigla']]
[1] "RJ"
```

Importando arquivo JSON

```
{
  "investidor":
    [
      "Paulo Silva", "Cris Pinheiro", "Thomas Filho"
    ],
  "acao":
    [
      { "sigla": "PTR04", "quantidade": 77 },
      { "sigla": "IBU05", "quantidade": 25 }
    ],
  "carteira": "GOLD"
}
```

```
> library(RJSONIO)
> carteira <- fromJSON("carteira.json")
> carteira$investidor
[1] "Paulo Silva" "Cris Pinheiro" "Thomas Filho"
```

```
> carteira$acao
[[1]] [[1]]$sigla
[1] "PTR04"

[[1]]$quantidade
[1] 77

[[2]]
[[2]]$sigla
[1] "IBU05"

[[2]]$quantidade
[1] 25
> carteira$acao[[1]]$sigla
[1] "PTR04"
>
> carteira$carteira
[1] "GOLD"
```

Importando arquivo JSON

- Dependendo da estruturação do arquivo **JSON** a função **fromJSON** pode retornar objetos que sejam de tipos de dados distintos do R, como vetores e listas e, desta forma, as funções **summary()**, **head()**, **tail()**, **glimpse()** também podem se utilizadas uma vez que são funções genéricas.

Importando tabelas de Banco de Dados Relacional

dbConnect cria uma conexão (con) com o banco de dados.

```
> library(DBI)
> con <- dbConnect(RSQLite::SQLite(), dbname="c:\\temp\\Indicadores.db")
> alltables = dbListTables(con)
> alltables
[1] "Estados" "Municipios"
```

dbListTables recupera o nome de todas as tabelas do banco de dados.

Importando tabelas de Banco de Dados Relacional

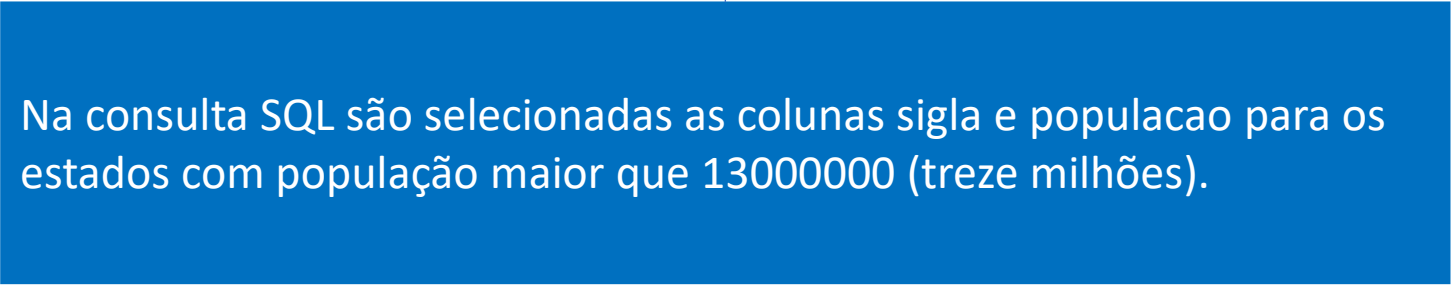
```
> library (RSQLite)
> estados <- dbGetQuery (con, "select * from estados;")
> estados
```

	sigla	descricao	populacao
1	AC	Acre	732793
2	AL	Alagoas	3120922
3	AP	Amapá	668689
4	AM	Amazonas	3480937
5	BA	Bahia	14021432
		...	

- dbGetQuery executa uma consulta SQL que neste caso retorna todas as colunas e todos os registros da tabela estados.
- Os dados retornados são armazenados em um data frame, onde as colunas selecionadas no SQL viram as colunas do data frame.

Importando tabelas de Banco de Dados Relacional

```
> library(RSQLite)
> estados <- dbGetQuery (con, "select sigla, populacao from estados where populacao
> 13000000;")
> estados
  sigla  populacao
1   BA    14021432
2   MG    19595309
3   RJ    15993583
4   SP    41252160
>
> estados$sigla
[1] "BA" "MG" "RJ" "SP"
```



Na consulta SQL são selecionadas as colunas sigla e populacao para os estados com população maior que 13000000 (treze milhões).

Importando tabelas de Banco de Dados Relacional

```
> library(RSQLite)
> estados <- dbGetQuery (con, "select sigla, populacao from estados where sigla in
('RJ', 'SP');")
> estados
```

	sigla	populacao
1	RJ	15993583
2	SP	41252160



Na consulta SQL são selecionadas as colunas sigla e populacao para os estados com sigla 'RJ' ou 'SP'.

Pacote dplyr

- O pacote **dplyr** é hoje um dos pacotes mais utilizados para **manipulação** de **data frames**.
- Ele disponibiliza diversas funções, muitas “equivalentes” às funções básicas do R, mas com melhorias que nos **poupam tempo** e deixam o código **mais fácil de interpretar**.
- Em aulas anteriores já exploramos algumas funções do pacote como:
 - **select**: permite **selecionar colunas** dos **data frames**;
 - **filter**: permite **selecionar linhas** específicas dos **data frames**;
 - **arrange**: permite **ordenar** os **data frames**;
 - **mutate**: permite **criar novas colunas** nos **data frames**.
- Iremos explorar agora outras funções do pacote **dplyr**.

Pacote dplyr

- Considere o banco de dados Financeiro.db que apresenta as tabelas Cliente e NotaFiscal.
- Tabela Cliente.

Codigo	Nome	Estado
123	Transportes Tico	RJ
201	Hotel Aloft	SP
426	Gerson construções	MG
451	Distribuidora Antunes	RJ
678	Mercado Ganha Pouco	SP
890	Colégio 11 de Julho	ES

Pacote dplyr

- Tabela NotaFiscal.

NumeroNota	Serie	CodigoCliente	DataEmissao	Valor	Imposto	DataCancelamento
138092	A1	123	05/01/2019	3450	345	NULL
236781	B1	201	10/01/2019	3000	200	20/01/2019
387209	C2	426	03/05/2019	3800	190	NULL
674240	B1	201	06/07/2019	6480	432	NULL
764901	C1	NULL	10/07/2019	8540	427	NULL
834092	A2	123	11/08/2019	31560	789	NULL
834519	A1	426	25/08/2019	200	20	NULL
842854	C1	NULL	30/09/2019	3050	305	NULL
934891	B1	201	01/10/2019	4800	320	NULL

Pacote dplyr

- Carregando os data frames cliente e nota.

```
library (DBI)
library (RSQLite)
library (dplyr)

con <- dbConnect (RSQLite::SQLite(), dbname = "d:\\Temp\\Financeiro.db")

cliente <- dbGetQuery (con, "SELECT * from Cliente;")

nota <- dbGetQuery (con, "SELECT * from NotaFiscal;")

dbDisconnect (con)
```

Pacote dplyr

- A função **distinct** retorna somente os valores únicos removendo conteúdo duplicado.

```
estado.distinto <- distinct (cliente, Estado)
```

estado.distinto contém todos os estados presentes na coluna Estado mas sem repetição.

```
> estado.distinto
```

	Estado
1	RJ
2	SP
3	MG
4	ES

```
estado.all.distinto <- distinct (cliente, Estado, .keep_all = T)
```

.keep_all = T modifica o comando trazendo todas as colunas do data frame cliente de um estado selecionado.

```
> estado.all.distinto
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	201	Hotel Aloft	SP
3	426	Gerson construções	MG
4	890	Colégio 11 de Julho	ES

Pacote dplyr

- A função **slice** extrai linhas por posição.

```
linhas.selecionadas.cliente <- slice(cliente, 2:4)
```

```
> cliente
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	201	Hotel Aloft	SP
3	426	Gerson construções	MG
4	451	Distribuidora Antunes	RJ
5	678	Mercado Ganha Pouco	SP
6	890	Colégio 11 de Julho	ES

```
> linhas.selecionadas.cliente
```

	Codigo	Nome	Estado
1	201	Hotel Aloft	SP
2	426	Gerson construções	MG
3	451	Distribuidora Antunes	RJ

linhas.selecionadas.cliente contém as linhas 2, 3 e 4 do data frame cliente.

Pacote dplyr

- A função **top_n** seleciona as **n** primeiras ou as **n** últimas linhas por valor de uma ou mais colunas.

```
top.maior.valor.nota <- top_n(nota, -3, Imposto)
```

```
> top.menor.valor.nota
```

	NumeroNota	Serie	CodigoCliente	DataEmissao	Valor	Imposto	DataCancelamento
1	236781	B1	201	2019-10-01	3000	200	2019-20-01
2	387209	C2	426	2019-03-05	3800	190	<NA>
3	834519	A1	426	2019-25-08	200	20	<NA>

top.menor.valor.nota contém as três últimas linhas (número -3 no comando) do data frame nota que possuem menor valor para a coluna Imposto.

Pacote dplyr

- A função **top_n** seleciona as **n** primeiras ou as **n** últimas linhas por valor de uma ou mais colunas.

```
top.maior.valor.nota <- top_n(nota, 2, Imposto)
```

```
> top.maior.valor.nota
```

	NumeroNota	Serie	CodigoCliente	DataEmissao	Valor	Imposto	DataCancelamento
1	674240	B1	201	2019-06-07	6480	432	<NA>
2	834092	A2	123	2019-11-08	31560	789	<NA>

top.maior.valor.nota contém as duas primeiras linhas (número 2 no comando) do data frame nota que possuem maior valor para a coluna Imposto.

Pacote dplyr

- A função **top_frac** seleciona, percentualmente, as primeiras ou as últimas linhas por valor de uma ou mais colunas.

```
top.maior.valor.nota.frac <- top_frac(nota, 0.35, Valor - Imposto)
```

```
> top.maior.valor.nota.frac
```

	NumeroNota	Serie	CodigoCliente	DataEmissao	Valor	Imposto	DataCancelamento
1	674240	B1	201	2019-06-07	6480	432	<NA>
2	764901	C1	NA	2019-10-07	8540	427	<NA>
3	834092	A2	123	2019-11-08	31560	789	<NA>

top.maior.valor.nota.frac contém as três primeiras linhas (35% do total de linhas) do data frame nota que possuem maior valor para a diferença entre as colunas Valor e Imposto.

Pacote dplyr

- A função **top_frac** seleciona, percentualmente, as primeiras ou as últimas linhas por valor de uma ou mais colunas.

```
top.menor.valor.nota.frac <- top_frac(nota, -0.60, Valor - Imposto)
```

```
> top.menor.valor.nota.frac
```

	NumeroNota	Serie	CodigoCliente	DataEmissao	Valor	Imposto	DataCancelamento
1	138092	A1	123	2019-05-01	3450	345	<NA>
2	236781	B1	201	2019-10-01	3000	200	2019-20-01
3	387209	C2	426	2019-03-05	3800	190	<NA>
4	834519	A1	426	2019-25-08	200	20	<NA>
5	842854	C1	NA	2019-30-09	3050	305	<NA>

top.menor.valor.nota.frac contém as cinco últimas linhas (60% do total de linhas e negativo) do data frame nota que possuem menor valor para a diferença entre as colunas Valor e Imposto.

Pacote dplyr

- A função **count** retorna o número de linhas com a combinação única do conteúdo das variáveis indicadas.

```
conta.cliente.estado <- count(cliente, Estado)
```

conta.cliente.estado contém os quatro estados presentes na coluna Estado do data frame cliente e as respectivas quantidades de clientes.

```
> conta.cliente.estado
```

	Estado	n
1	ES	1
2	MG	1
3	RJ	2
4	SP	2

```
conta.nota.cliente.serie <- count(nota,CodigoCliente, Serie)
```

conta.nota.cliente.serie contém as seis combinações existentes das colunas CodigoCliente e Serie no data frame nota e as respectivas quantidades de linhas.

```
> conta.nota.cliente.serie
```

	CodigoCliente	Serie	n
1	123	A1	1
2	123	A2	1
3	201	B1	3
4	426	A1	1
5	426	C2	1
6	NA	C1	2

Pacote dplyr

- A função **summarise** permite criar um resumo do data frame, através do cálculo de estatísticas de suas colunas.

```
summarise1.nota <- summarise(nota, quantidade = n(), maximo = max(Valor))
```

summarise1.nota contém duas colunas, uma com a quantidade de linhas (função `n()`) do data frame `nota` e outra com o valor máximo (função `max()`) da coluna `Valor`.

```
> summarise1.nota
  quantidade maximo
1           9  31560
```

```
summarise2.nota <- summarise(nota, qtd = n(), serie = n_distinct(Serie), minimo = min(Valor))
```

summarise2.nota contém três colunas, uma com a quantidade de linhas (função `n()`) do data frame `nota`, outra com a quantidade distinta (função `n_distinct()`) de séries de nota fiscal e uma terceira com o valor mínimo (função `min()`) da coluna `Valor`.

```
> summarise2.nota
  qtd serie minimo
1   9     5    200
```

Pacote dplyr

- A função **group_by** permite separar linhas em grupos.

```
grupo.cliente <- group_by(cliente, Estado)
sumario.cliente <- summarise(grupo.cliente, n())
```

```
> grupo.cliente
# A tibble: 6 x 3
# Groups:   Estado [4]
  Codigo Nome                Estado
  <int> <chr>                <chr>
1     123 Transportes Tico      RJ
2     201 Hotel Aloft          SP
3     426 Gerson construções   MG
4     451 Distribuidora Antunes RJ
5     678 Mercado Ganha Pouco  SP
6     890 Colégio 11 de Julho  ES
```

```
> sumario.cliente
# A tibble: 4 x 2
  Estado `n()`
  <chr>   <int>
1 ES             1
2 MG             1
3 RJ             2
4 SP             2
```

grupo.cliente contém quatro grupos definidos pela coluna Estado e sumario.cliente contém os quatro grupos, definidos pelos estados, e a quantidade de linhas em cada grupo.

Pacote dplyr

- A função **inner_join(a, b, by)** retorna um data frame com as linhas do data frame **a** que possuem correspondente no **b**. O data frame retornado possui todas as colunas de **a** e **b**. No parâmetro **by** é especificada a relação de correspondência.

```
inner.join.cliente.nota <- inner_join(cliente, nota, by = c("Codigo" = "CodigoCliente"))
```

```
> inner.join.cliente.nota
```

	Codigo	Nome	Estado	NumeroNota	Serie	DataEmissao	Valor	Imposto	DataCancelamento
1	123	Transportes Tico	RJ	138092	A1	2019-05-01	3450	345	<NA>
2	123	Transportes Tico	RJ	834092	A2	2019-11-08	31560	789	<NA>
3	201	Hotel Aloft	SP	236781	B1	2019-10-01	3000	200	2019-20-01
4	201	Hotel Aloft	SP	674240	B1	2019-06-07	6480	432	<NA>
5	201	Hotel Aloft	SP	934891	B1	2019-01-10	4800	320	<NA>
6	426	Gerson construções	MG	387209	C2	2019-03-05	3800	190	<NA>
7	426	Gerson construções	MG	834519	A1	2019-25-08	200	20	<NA>

inner.join.cliente.nota contém as linhas dos data frames cliente e nota que possuem correspondência. O data frame possui todas as colunas de cliente e nota. A função inner_join é equivalente ao Inner Join do SQL

Pacote dplyr

- A função **inner_join (a, b, by)** retorna um data frame com as linhas do data frame **a** que possuem correspondente no **b**. O data frame retornado possui todas as colunas de **a** e **b**. No parâmetro **by** é especificada a relação de correspondência.

Também é possível e, muitas vezes necessário, utilizar mais de uma variável para vincular dois data frames. Nesse caso, a função ficaria da seguinte forma: `inner_join (dataframe1, dataframe2, by = c("variavel_comum_1", "variavel_comum_2"))`.

Pacote dplyr

- A função **left_join(a, b, by)** retorna todas as linhas que estão no data frame **a** (mesmo sem correspondência em **b**) e as linhas de **b** que são comuns ao **a**. O data frame retornado possui todas as colunas de **a** e **b**. No parâmetro **by** é especificada a relação de correspondência.

```
left.join.nota.cliente <- left_join(nota, cliente, by = c("CodigoCliente" = "Codigo"))
```

```
> left.join.nota.cliente
```

	NumeroNota	Serie	CodigoCliente	DataEmissao	Valor	Imposto	DataCancelamento	Nome	Estado
1	138092	A1	123	2019-05-01	3450	345	<NA>	Transportes Tico	RJ
2	236781	B1	201	2019-10-01	3000	200	2019-20-01	Hotel Aloft	SP
3	387209	C2	426	2019-03-05	3800	190	<NA>	Gerson construções	MG
4	674240	B1	201	2019-06-07	6480	432	<NA>	Hotel Aloft	SP
5	764901	C1	NA	2019-10-07	8540	427	<NA>	<NA>	<NA>
6	834092	A2	123	2019-11-08	31560	789	<NA>	Transportes Tico	RJ
7	834519	A1	426	2019-25-08	200	20	<NA>	Gerson construções	MG
8	842854	C1	NA	2019-30-09	3050	305	<NA>	<NA>	<NA>
9	934891	B1	201	2019-01-10	4800	320	<NA>	Hotel Aloft	SP

left.join.nota.cliente contém todas as linhas do data frame nota que possuem ou não cliente e as linhas de cliente que possuem nota. É equivalente ao Left Join do SQL.

Pacote dplyr

- A função **right_join(a, b, by)** retorna um data frame com todas as linhas do data frame **b** (mesmo sem correspondência em **a**) e as linhas de **a** que são comuns a **b**. o data frame retornado possui todas as colunas de **a** e **b**. No parâmetro **by** é especificada a relação de correspondência.

```
right.join.nota.cliente <- right_join(nota, cliente, by = c("CodigoCliente" = "Codigo"))
> right.join.nota.cliente
```

	NumeroNota	Serie	CodigoCliente	DataEmissao	Valor	Imposto	DataCancelamento	Nome	Estado
1	138092	A1	123	2019-05-01	3450	345	<NA>	Transportes Tico	RJ
2	236781	B1	201	2019-10-01	3000	200	2019-20-01	Hotel Aloft	SP
3	387209	C2	426	2019-03-05	3800	190	<NA>	Gerson construções	MG
4	674240	B1	201	2019-06-07	6480	432	<NA>	Hotel Aloft	SP
5	834092	A2	123	2019-11-08	31560	789	<NA>	Transportes Tico	RJ
6	834519	A1	426	2019-25-08	200	20	<NA>	Gerson construções	MG
7	934891	B1	201	2019-01-10	4800	320	<NA>	Hotel Aloft	SP
8	<NA>	<NA>	451	<NA>	NA	NA	<NA>	Distribuidora Antunes	RJ
9	<NA>	<NA>	678	<NA>	NA	NA	<NA>	Mercado Ganha Pouco	SP
10	<NA>	<NA>	890	<NA>	NA	NA	<NA>	Colégio 11 de Julho	ES

right.join.nota.cliente contém todas as linhas do data frame cliente que possuem ou não nota e as linhas de nota que possuem cliente. É equivalente ao Right Join do SQL

Pacote dplyr

- A função **full_join(a, b, by)** retorna um data frame com todas as linhas do data frame **a** (mesmo sem correspondência em **b**) e todas as linhas de **b** (mesmo sem correspondência em **a**).

```
full.join.cliente.nota <- full_join(cliente, nota, by = c("Codigo" = "CodigoCliente"))
```

```
> full.join.cliente.nota
```

	Codigo	Nome	Estado	NumeroNota	Serie	DataEmissao	Valor	Imposto	DataCancelamento
1	123	Transportes Tico	RJ	138092	A1	2019-05-01	3450	345	<NA>
2	123	Transportes Tico	RJ	834092	A2	2019-11-08	31560	789	<NA>
3	201	Hotel Aloft	SP	236781	B1	2019-10-01	3000	200	2019-20-01
4	201	Hotel Aloft	SP	674240	B1	2019-06-07	6480	432	<NA>
5	201	Hotel Aloft	SP	934891	B1	2019-01-10	4800	320	<NA>
6	426	Gerson construções	MG	387209	C2	2019-03-05	3800	190	<NA>
7	426	Gerson construções	MG	834519	A1	2019-25-08	200	20	<NA>
8	451	Distribuidora Antunes	RJ	<NA>	<NA>	<NA>	NA	NA	<NA>
9	678	Mercado Ganha Pouco	SP	<NA>	<NA>	<NA>	NA	NA	<NA>
10	890	Colégio 11 de Julho	ES	<NA>	<NA>	<NA>	NA	NA	<NA>
11	NA	<NA>	<NA>	764901	C1	2019-10-07	8540	427	<NA>
12	NA	<NA>	<NA>	842854	C1	2019-30-09	3050	305	<NA>

`full.join.cliente.nota` contém todas as linhas do data frame cliente que possuem ou não nota e todas as linhas de nota que possuem ou não cliente. É equivalente ao Full Outer Join do SQL.

Pacote dplyr

- A função **semi_join(a, b, by)** retorna um data frame com todas as linhas do data frame **a** que possuem correspondência em **b**. O data frame retornado possui somente as colunas de **a**. No parâmetro **by** é especificada a relação de correspondência.

```
semi.join.cliente.nota <- semi_join(cliente, nota, by = c("Codigo" = "CodigoCliente"))
```

```
> semi.join.cliente.nota
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	201	Hotel Aloft	SP
3	426	Gerson construções	MG

semi.join.cliente.nota contém todas as linhas do data frame cliente que possuem nota. Em SQL pode ser implementado usando Inner Joins, Exists ou In.

Pacote dplyr

- A função **anti_join(a, b, by)** retorna um data frame com todas as linhas do data frame **a** que **não** possuem correspondência em **b**. O data frame retornado possui somente as colunas de **a**. No parâmetro **by** é especificada a relação de correspondência.

```
anti.join.cliente.nota <- anti_join(cliente, nota, by = c("Codigo" = "CodigoCliente"))
```

```
> anti.join.cliente.nota
```

	Codigo	Nome	Estado
1	451	Distribuidora Antunes	RJ
2	678	Mercado Ganha Pouco	SP
3	890	Colégio 11 de Julho	ES

anti.join.cliente.nota contém todas as linhas do data frame cliente que não possuem nota. Em SQL pode ser implementado usando Inner Joins, Exists ou In.

Pacote dplyr

- A função **bind_rows(a, b)** retorna um data frame com todas as linhas do data frame **b** adicionadas ao data frame **a**.

```
cliente.RJ <- filter(cliente, Estado == "RJ")
novo.cliente.RJ <- data.frame(Codigo = 111, Nome = "Bike for all", Estado = "RJ")
cliente.RJ <- bind_rows(cliente.RJ, novo.cliente.RJ)
```

```
> cliente.RJ
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	451	Distribuidora Antunes	RJ

```
> cliente.RJ
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	451	Distribuidora Antunes	RJ
3	111	Bike for all	RJ

cliente.RJ contém ao final da sequência de comandos os dois clientes do estado “RJ” filtrados do data frame cliente mais o novo cliente (novo.cliente.RJ) adicionado pelo comando bind_rows.

Pacote dplyr

- A função **intersect(a, b)** retorna um data frame com todas as linhas comuns aos data frames **a** e **b**. Compara todas as colunas.

```
intersecao.cliente <- intersect(cliente, cliente.RJ)
```

```
> cliente
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	201	Hotel Aloft	SP
3	426	Gerson construções	MG
4	451	Distribuidora Antunes	RJ
5	678	Mercado Ganha Pouco	SP
6	890	Colégio 11 de Julho	ES

```
> cliente.RJ
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	451	Distribuidora Antunes	RJ
3	111	Bike for all	RJ

```
> intersecao.cliente
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	451	Distribuidora Antunes	RJ

intersecao.cliente contém somente os dois clientes comuns aos data frames cliente e cliente.RJ.

Pacote dplyr

- A função **union(a, b)** retorna um data frame com todas as linhas dos data frames **a** e **b** sem duplicidade. Compara todas as colunas.

```
uniao.cliente <- union(cliente, cliente.RJ)
```

```
> cliente
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	201	Hotel Aloft	SP
3	426	Gerson construções	MG
4	451	Distribuidora Antunes	RJ
5	678	Mercado Ganha Pouco	SP
6	890	Colégio 11 de Julho	ES

```
> cliente.RJ
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	451	Distribuidora Antunes	RJ
3	111	Bike for all	RJ

```
> uniao.cliente
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	201	Hotel Aloft	SP
3	426	Gerson construções	MG
4	451	Distribuidora Antunes	RJ
5	678	Mercado Ganha Pouco	SP
6	890	Colégio 11 de Julho	ES
7	111	Bike for all	RJ

uniao.cliente contém todos os clientes dos data frames cliente e cliente.RJ sem duplicar os clientes comuns.

Pacote dplyr

- A função **union_all(a, b)** retorna um data frame com todas as linhas dos data frames **a** e **b**.

```
uniao.all.cliente <- union_all(cliente, cliente.RJ)
```

```
> cliente
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	201	Hotel Aloft	SP
3	426	Gerson construções	MG
4	451	Distribuidora Antunes	RJ
5	678	Mercado Ganha Pouco	SP
6	890	Colégio 11 de Julho	ES

```
> cliente.RJ
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	451	Distribuidora Antunes	RJ
3	111	Bike for all	RJ

```
> uniao.all.cliente
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	201	Hotel Aloft	SP
3	426	Gerson construções	MG
4	451	Distribuidora Antunes	RJ
5	678	Mercado Ganha Pouco	SP
6	890	Colégio 11 de Julho	ES
7	123	Transportes Tico	RJ
8	451	Distribuidora Antunes	RJ
9	111	Bike for all	RJ

uniao.all.cliente contém todos os clientes dos data frames cliente e cliente.RJ duplicando os clientes comuns.

Pacote dplyr

- A função **setdiff(a, b)** retorna um data frame com todas as linhas do data frame **a** que não estão no **b**. Compara todas as colunas.

```
diferenca.cliente <- setdiff(cliente, cliente.RJ)
```

```
> cliente
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	201	Hotel Aloft	SP
3	426	Gerson construções	MG
4	451	Distribuidora Antunes	RJ
5	678	Mercado Ganha Pouco	SP
6	890	Colégio 11 de Julho	ES

```
> cliente.RJ
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	451	Distribuidora Antunes	RJ
3	111	Bike for all	RJ

```
> diferenca.cliente
```

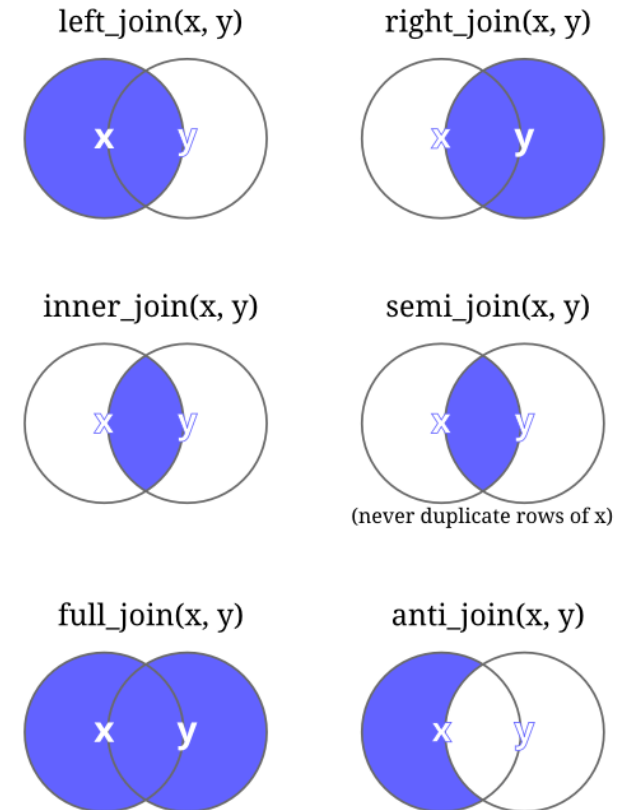
	Codigo	Nome	Estado
1	201	Hotel Aloft	SP
2	426	Gerson construções	MG
3	678	Mercado Ganha Pouco	SP
4	890	Colégio 11 de Julho	ES

uniao.all.cliente contém todos os clientes do data frame cliente que não estão no data frame cliente.RJ.

Pacote dplyr resumo JOINS

- **left_join** (x , y , **by**) retorna todas as linhas que estão no data frame x (mesmo sem correspondência em y) e as linhas de y que são comuns ao x . O data frame retornado possui todas as colunas de x e y .
- **right_join** (x , y , **by**) retorna um data frame com todas as linhas do data frame y (mesmo sem correspondência em x) e as linhas de x que são comuns a y . o data frame retornado possui todas as colunas de x e y .
- **inner_join** (x , y , **by**) retorna um data frame com as linhas do data frame x que possuem correspondente no y . O data frame retornado possui todas as colunas de x e y .
- **semi_join** (x , y , **by**) retorna um data frame com todas as linhas do data frame x que possuem correspondência em y . O data frame retornado possui somente as colunas de x .
- **full_join** (x , y , **by**) retorna um data frame com todas as linhas do data frame x (mesmo sem correspondência em y) e todas as linhas de y (mesmo sem correspondência em x).
- **anti_join** (x , y , **by**) retorna um data frame com todas as linhas do data frame x que não possuem correspondência em y . O data frame retornado possui somente as colunas de x .
- No parâmetro **by** é especificada a relação de correspondência das funções.

dplyr joins



Operador %>% (pipe)

- O operador funciona da seguinte forma:

```
dataset %>% função()
```

- Isso significa que o objeto do lado esquerdo do operador (dataset) será inserido na função ao lado direito do operador no primeiro argumento da função.
- O operador %>% pode ser usado conjuntamente com inúmeras funções.
- Exemplo 1, de sequências equivalentes:

```
ordenado.cliente <- arrange(cliente,Codigo)  
ordenado.cliente
```

```
ordenado.cliente <- cliente %>% arrange(Codigo)  
ordenado.cliente
```

Operador %>% (pipe)

- Exemplo 2, o data frame cliente é submentido como parâmetro à função **filter** e o data frame resultante é submetido como parâmetro à função **arrange** resultando um data frame filtrado e ordenado:

```
filtrado.ordenado.cliente <- cliente %>% filter(Estado == "RJ") %>% arrange(Codigo)
```

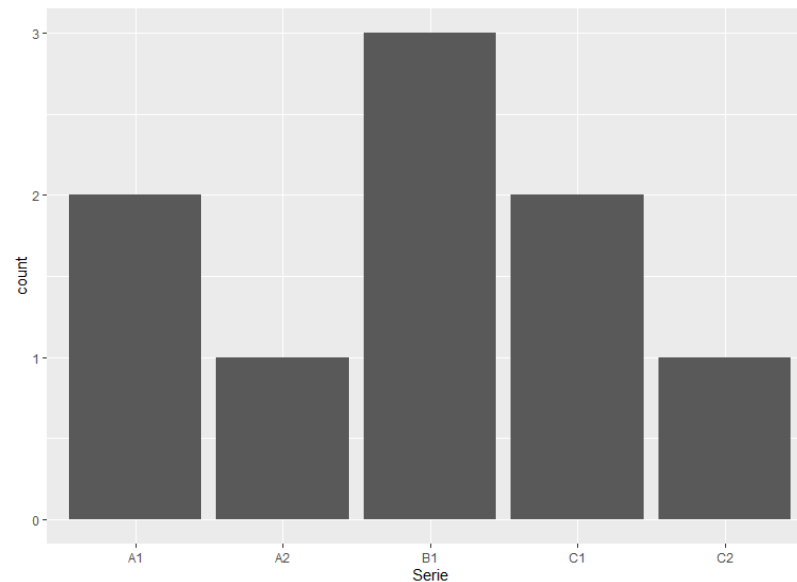
```
> filtrado.ordenado.cliente
```

	Codigo	Nome	Estado
1	123	Transportes Tico	RJ
2	451	Distribuidora Antunes	RJ

Operador %>% (pipe)

- Exemplo 3, o data frame **nota** é submentido como parâmetro à função **ggplot** que define o **dataset** como sendo **nota** e mapeia para **x** a coluna **Serie**. O gráfico apresenta no eixo **x** as séries e no eixo **y** a quantidade de notas para cada série:

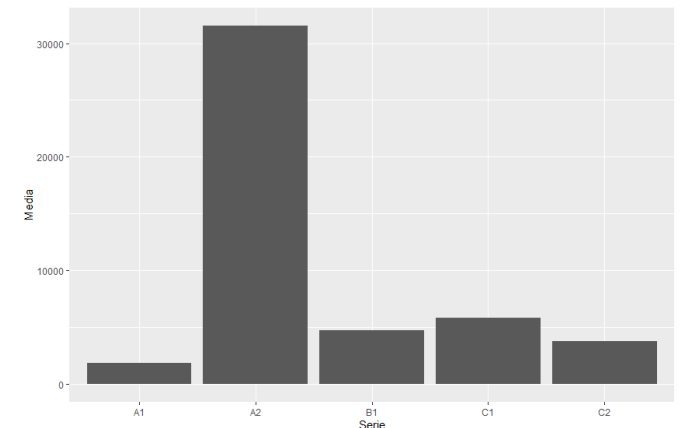
```
nota %>% ggplot(mapping = aes(x = Serie)) + geom_bar()
```



Operador %>% (pipe)

- Exemplo 4, o data frame **nota** é submentido como parâmetro à função **group_by** (agrupa **nota** por **Serie**), o resultado é submetido à função **summarise** (calcula a média do valor das notas por série) e o resultado da sumarização é submetido à função **ggplot** que define o **dataset** como o resultado da sumarização e mapeia para **x** a coluna **Serie** e para **y** a coluna **Media**. O gráfico apresenta no eixo **x** as séries e no eixo **y** a média do valor das notas para cada série:

```
nota %>% group_by(Serie) %>% summarise(Media = mean(Valor)) %>% ggplot(mapping =  
aes(x = Serie, y = Media)) + geom_bar(stat = "identity")
```



Manipulação de hora e data

- Existem vários formatos para uma variável data e hora, como por exemplo:
 - 16/02/2020 14:11:43 - dia, mês, ano com 4 dígitos, horas, minutos e segundos;
 - 02/18/18 13:10 - mês, dia, ano com 2 dígitos, horas e minutos;
 - 12 Novembro 2020 - dia, mês por extenso e ano com 4 dígitos.
- Cada fonte de dados pode definir um padrão de hora e data diferente, porém, o R tem um formato padrão para manipulação dos dados.

Manipulação de hora e data

- O formato tradicional usado no R para data e hora é:
 - 2020-11-28 16:32:42 - ano com 4 dígitos, mês, dia, hora, minutos e segundos.
- Ou, somente a data:
 - 2020-11-28 - ano com 4 dígitos, mês, dia.
- Assim, quando os dados que for processar apresentarem uma variável de hora e data, é aconselhável transformá-la para o formato padrão do R.
- Os formatos de data e hora são definidos através de letras e para saber o que cada letra representa na definição do formato da variável de data, basta digitar no R o comando:
 - `?strptime`

Manipulação de hora e data

- Para transformar dados originais de data e hora para o formato padrão do R, deve ser informado qual é o padrão original.

```
data1 <- c("14-02-2020")  
data1 <- strptime(data1, format = "%d-%m-%Y")  
data1
```

- Ao informar que o formato original dos dados é “%d-%m-%Y”, estamos avisando ao R que originalmente a variável de data está no formato dia (%d), mês (%m) e ano com 4 dígitos (%Y).

```
> data1  
[1] "2020-02-14 -02"
```

- O -02 ao final da data1 convertida se refere à diferença de fuso horário em relação ao horário UTC (Tempo Universal Coordenado).

Manipulação de hora e data

- Para transformar dados originais de data e hora para o formato padrão do R, deve ser informado qual é o padrão original.

```
data2 <- c("15 Novembro 2018")  
data2 <- strptime(data2, format = "%d %B %Y")  
data2
```

- Ao informar que o formato original dos dados é “%d %B %Y”, estamos falando que primeiro é o dia (%d), depois o mês por extenso (%B) e depois o ano com 4 dígitos (%Y).

```
> data2  
[1] "2018-11-15 -02"
```

- O -02 ao final da data2 convertida se refere à diferença de fuso horário em relação ao horário UTC (Tempo Universal Coordenado).

Manipulação de hora e data

- Para transformar dados originais de data e hora para o formato padrão do R, deve ser informado qual é o padrão original.

```
data.hora<-c("25 Fevereiro 18, 16:14:49")  
data.hora <- strptime(data.hora, "%d %B %y, %H:%M:%S")  
data.hora
```

- Ao informar que o formato original dos dados é “%d %B %y, %H:%M:%S”, estamos falando que primeiro é o dia (%d), depois o mês por extenso (%B), depois o ano com 2 dígitos (%y), depois as horas com 2 dígitos (%H), depois os minutos com 2 dígitos (%M) e depois os segundos com 2 dígitos (%S).

```
> data.hora  
[1] "2018-02-25 16:14:49 -03"
```

- O -03 ao final da data.hora convertida se refere à diferença de fuso horário em relação ao horário UTC (Tempo Universal Coordenado).

Manipulação de hora e data

- Para obter o ano de uma data pode ser utilizado o comando:

```
data.hora<-c("25 Fevereiro 18, 16:14:49")
data.hora <- strptime(data.hora, "%d %B %y, %H:%M:%S")
data.hora
```

```
ano <- format(data.hora,"%Y")
ano
```

```
> ano
[1] "2018"
```

- Para obter o mês basta trocar %Y por %m no comando acima. Para obter o dia deve ser usado %d, as horas %H, os minutos %M e os segundos %S.
- Para obter as horas e minutos deve ser usado %H:%M.

Manipulação de hora e data

- A função **difftime** permite calcular a diferença entre duas datas:

```
data.hora1 <-c("25 Fevereiro 18, 16:14:49")
data.hora1 <- strptime(data.hora1, "%d %B %y, %H:%M:%S")

data.hora2 <-c("23 Fevereiro 18, 15:14:49")
data.hora2 <- strptime(data.hora2, "%d %B %y, %H:%M:%S")

diferenca.horas <- difftime(data.hora1, data.hora2, units='hours')
diferenca.horas

> diferenca.horas
Time difference of 49 hours
```

- A diferença também pode ser obtida em dias (days), minutos (mins), segundos (secs), etc.

Manipulação de hora e data

- Também é possível somar ou subtrair valores de uma data e hora, sendo o segundo a menor unidade:

```
data.hora1 <- c("25 Fevereiro 18, 16:14:49")
data.hora1 <- strptime(data.hora1, "%d %B %y, %H:%M:%S")
data.hora1 <- data.hora1 + 59
data.hora1
```

- À data.hora1 foi somado 59 segundos.

```
> data.hora1
[1] "2018-02-25 16:15:48 -03"
```

- Para reduzir 2 minutos pode ser usado:

```
data.hora1 <- c("25 Fevereiro 18, 16:14:49")
data.hora1 <- strptime(data.hora1, "%d %B %y, %H:%M:%S")
data.hora1 <- data.hora1 - 2*60
data.hora1
```

```
> data.hora1
[1] "2018-02-25 16:12:49 -03"
```

Pacote tidyverse

- O pacote **tidyverse** é um pacote **guarda-chuva** que consolida uma série de ferramentas que fazem parte o ciclo da ciência de dados.
- Oferece uma reimplementação e extensão das funcionalidades padrões do R para **manipulação** e **visualização** de dados.
- Os pacotes (abaixo) que fazem parte do **tidyverse** foram planejados e construídos para trabalhar em conjunto.

```
"broom"  
"dbplyr"  
"hms"  
"magrittr"  
"readxl"  
"rvest"  
"xml2"
```

```
"cli"  
"forcats"  
"httr"  
"modelr"  
"reprex"  
"stringr"  
"tidyverse"
```

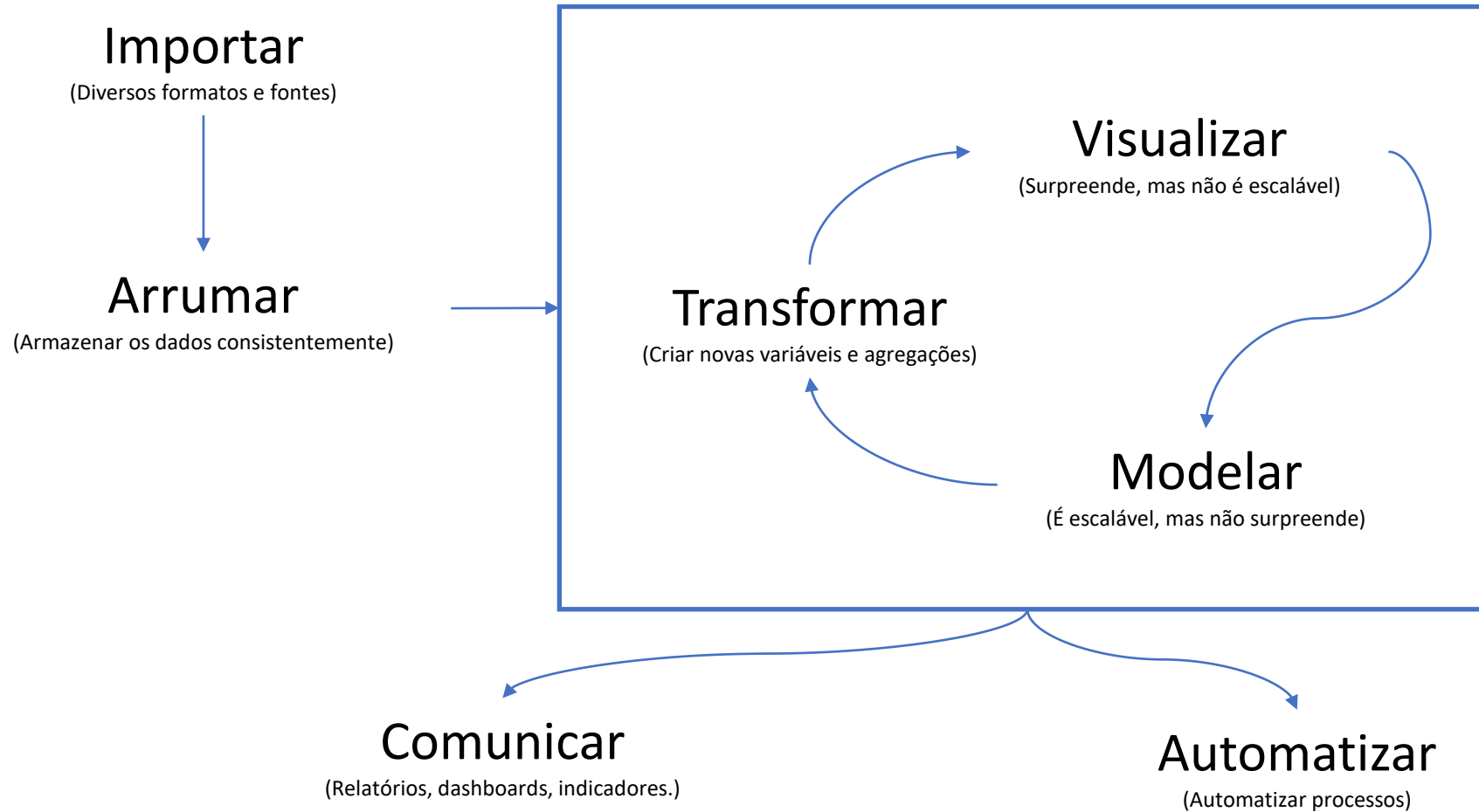
```
"crayon"  
"ggplot2"  
"jsonlite"  
"purrr"  
"rlang"  
"tibble"
```

```
"dplyr"  
"haven"  
"lubridate"  
"readr"  
"rstudioapi"  
"tidyr"
```

Pacote tidyverse

- Alguns dos princípios fundamentais do uso do **tidyverse** consiste em organizar funções simples usando o **pipe**.
- O pacote **stringr** apresenta uma série de recursos para manipulação de **strings**.
- O pacote **purrr** inclui funções semelhantes a **sapply** mas eles interagem melhor com outras funções **tidyverse**.
- O data frame é a estrutura nativa do R para representar tabelas de dados, o **tidyverse** agrega ao ambiente R o **tibble** que é uma reimplementação da estrutura do data frame com melhorias:
 - Método print (visualização) mais enxuto e informativo.
 - Mais consistente para seleção e modificação de conteúdo.
 - Colunas/cédulas podem representar objetos mais complexos.

Pacote tidyverse



Operações típicas com tidyverse

- Importar e/ou acessar dados.
- Ordenar registros.
- Selecionar e fatiar dados por linhas ou colunas.
- Filtrar registros.
- Renomear colunas.
- Reorganizar a disposição do conteúdo.
- Modificar e/ou transformar dados.
- Aplicar funções e calcular resumos.
- Agregar dados por categorias.
- Concatenar, juntar e/ou conciliar tabelas.

Seguindo, serão apresentados exemplos de algumas operações típicas com tidyverse em dados no formato tibble.

Pacote tidyverse (tibble)

```
> cliente
  Id      Nome      DDD Telefone Classificacao
1  42450854525 Júlio Luiz Fausto Fernandes 11 360021983 L
2  23712872453 Alex wulff Moraes 11 762087607 L
3  38614430159 Casemiro Paulo Cormack 12 458393440 V
4  82559918404 Maria Cristina Garcia Vargas 12 336841205 L
5  33421388318 Thainá Diniz Frotté 12 699167650 N
6  14768110401 José Ximenes Vargas 11 334940412 L
...
97 22841875954 Adilson de Padua Pedroso 11 962112712 L
98 40721885225 Telmo Knupp Filho 11 920013264 A
99 26071793432 Ivan Felizardo Sarmento 11 903649616 L
100 12150713373 Benício Arlia Duarte 11 947685566 A
> |
```

Data frame cliente à esquerda e o mesmo data frame transformado em tibble abaixo.

Print mais organizado.

Por padrão, apenas as dez primeiras linhas são apresentadas. Também são apresentadas a dimensão (100x5) e as classes de cada coluna.

```
> as_tibble(cliente)
# A tibble: 100 x 5
  Id      Nome      DDD Telefone Classificacao
  <chr>   <chr>   <chr> <chr>   <chr>
1 42450854525 Júlio Luiz Fausto Fernandes 11 360021983 L
2 23712872453 Alex wulff Moraes 11 762087607 L
3 38614430159 Casemiro Paulo Cormack 12 458393440 V
4 82559918404 Maria Cristina Garcia Vargas 12 336841205 L
5 33421388318 Thainá Diniz Frotté 12 699167650 N
6 14768110401 José Ximenes Vargas 11 334940412 L
7 84823241282 Raul Vitorino Antunes 11 321840034 A
8 28371521294 Rosali Baesso Marques 11 978342023 A
9 93227565005 Margareth Sampaio Cruz 17 955388502 N
10 23205940270 Vânia Cordeiro Queiroz 17 913404039 L
# ... with 90 more rows
```

Pacote readr

- A função **read_csv2** permite carregar um arquivo **CSV** em uma **tibble**.

```
clientes <- read_csv2("NovosClientes.csv", col_types = cols(col_character(),
col_character(), col_character(), col_character(), col_integer()))
```

```
> clientes
```

```
# A tibble: 100 x 6
```

	CPF	Nome	DDD	Telefone	Classificacao	Divida
	<chr>	<chr>	<chr>	<chr>	<chr>	<int>
1	10244521310	Margarida Palmas Stutz	12	959128774	L	NA
2	11125248785	Cláudio Peixoto Barelllos	17	945271192	NA	NA
3	12150713373	Benício Arlia Duarte	11	947685566	A	NA
4	14436831590	Tereza Pires Barbosa	22	994053970	A	NA
5	14768110401	José Ximenes Vargas	11	334940412	NA	NA
6	15271527409	Alba Knupp Barrada	22	931442914	NA	NA
7	15445645207	Marisa de Padua da Sousa	21	911021168	L	NA
8	15796045440	Marlene Silva Meyer	19	991341440	L	NA
9	18112077231	Júlio Rosa Viana	22	917547560	L	NA
10	19167287409	Lina de Lima Uchôa	13	925186816	L	NA

```
# ... with 90 more rows
```

Pacote dplyr

- A função **arrange** permite ordenar por uma ou mais colunas.

```
cli <- clientes %>% arrange(DDD, Nome)
```

```
> cli
# A tibble: 100 x 6
   CPF          Nome          DDD  Telefone  Classificacao Divida
   <chr>        <chr>        <chr> <chr>      <chr>         <int>
1 22841875954 Adilson de Padua Pedroso 11    962112712 L          NA
2 23712872453 Alex Wulff Moraes      11    762087607 L          NA
3 12150713373 Benício Arlia Duarte    11    947685566 A          NA
4 36672030395 Clemilda Neves Figueiredo 11    906497335 V          NA
5 32615743481 Glauber Carvalho Muchão 11    929201474 L          NA
6 45484376300 Greycy Sales Alfradique 11    949096195 L          NA
7 68781251386 Iára Figueiro Chaves    11    960514681 V          NA
8 26071793432 Ivan Felizardo Sarmento 11    903649616 L          NA
9 14768110401 José Ximenes Vargas     11    334940412 NA         NA
10 42450854525 Júlio Luiz Fausto Fernandes 11    360021983 L          NA
# ... with 90 more rows
```

Pacote dplyr

- A função **slice** permite extrair linhas por posição.

```
cli <- clientes %>% arrange(DDD, Nome) %>% slice(2:5)
```

```
> cli
# A tibble: 4 x 6
  CPF          Nome          DDD  Telefone  Classificacao Divida
<chr>      <chr>      <chr> <chr>      <chr>          <int>
1 23712872453 Alex Wulff Moraes  11    762087607 L             NA
2 12150713373 Benício Arlia Duarte  11    947685566 A             NA
3 36672030395 Clemilda Neves Figueiredo 11    906497335 V             NA
4 32615743481 Glauber Carvalho Muchão  11    929201474 L             NA
```

Pacote dplyr

- A função **filter** permite filtrar por uma ou mais colunas.

```
cli <- clientes %>% filter(DDD %in% c("14", "13"))
```

```
> cli
# A tibble: 3 x 6
  CPF      Nome      DDD  Telefone  Classificacao Divida
<chr>    <chr>    <chr> <chr>    <chr>         <int>
1 19167287409 Lina de Lima Uchôa 13 925186816 L           NA
2 19377803411 Bryan Louzano Schuenck 13 979111679 A           NA
3 72869674287 Marilena Cristo Monnerat 14 983594857 A           NA
```

Pacote dplyr

- A função **sample_n** permite extrair uma amostra de tamanho **size** com ou sem reposição.

```
cli <- clientes %>% sample_n(size = 5, replace = FALSE)
```

```
> cli
# A tibble: 5 x 6
  CPF      Nome      DDD  Telefone  Classificacao Divida
<chr>    <chr>    <chr> <chr>    <chr>         <int>
1 84872296702 Dayvid Cretella Giacomini 19 927940075 N NA
2 65297312531 Casemiro Palmas Billé 22 999427645 L NA
3 11125248785 Cláudio Peixoto Barellos 17 945271192 NA NA
4 87488541154 Simone Inácio Azevedo 15 926087052 A NA
5 28371521294 Rosali Baesso Marques 11 978342023 A NA
```

Pacote dplyr

- A função **sample_frac** permite extrair uma amostra percentual **size** com ou sem reposição.

```
cli <- clientes %>% sample_frac(size = 0.1, replace = FALSE)
```

```
> cli
# A tibble: 10 x 6
   CPF      Nome      DDD  Telefone Classificacao Divida
  <chr>   <chr>   <chr> <chr>      <chr>         <int>
1 84823241282 Raul Vitorino Antunes 11 321840034 A NA
2 48224841901 Lucimberto Valansuela de Barros 27 915960939 A NA
3 66552658001 Jayne Leite Temperini 21 936222787 L NA
4 67164673276 Marilene Corrêa Linhares 21 940270580 L NA
5 27387358435 Renata Bilé Saldanha 21 987407812 L NA
6 63721743598 Valentim Nespoli Figueiras 28 943177423 L NA
7 12150713373 Benício Arlia Duarte 11 947685566 A NA
8 59004304991 Amanda Lopes Neto 21 931167881 A NA
9 11125248785 Cláudio Peixoto Barellos 17 945271192 NA NA
10 28235543435 Carmen da Sousa Linhares 22 909637937 L NA
```

Pacote dplyr

- A função **mutate_if** permite alterar dados de colunas. No exemplo, os nomes são colocados em maiúsculo pela função **str_to_upper** do pacote **stringr**.

```
cli <- clientes %>% mutate_if(is.character, str_to_upper)
```

```
> cli
# A tibble: 100 x 6
   CPF      Nome      DDD  Telefone Classificacao Divida
  <chr>    <chr>    <chr> <chr>      <chr>         <int>
1 10244521310 MARGARIDA PALMAS STUTZ 12    959128774 L             NA
2 11125248785 CLÁUDIO PEIXOTO BARELLOS 17    945271192 NA            NA
3 12150713373 BENÍCIO ARLIA DUARTE 11    947685566 A             NA
4 14436831590 TEREZA PIRES BARBOSA 22    994053970 A             NA
5 14768110401 JOSÉ XIMENES VARGAS 11    334940412 NA            NA
6 15271527409 ALBA KNUPP BARRADA 22    931442914 NA            NA
7 15445645207 MARISA DE PADUA DA SOUSA 21    911021168 L             NA
8 15796045440 MARLENE SILVA MEYER 19    991341440 L             NA
9 18112077231 JÚLIO ROSA VIANA 22    917547560 L             NA
10 19167287409 LINA DE LIMA UCHÔA 13    925186816 L             NA
# ... with 90 more rows
```



Pacote dplyr

- A função **mutate_if** permite alterar dados de colunas. No exemplo, se o tipo da coluna for **integer** ela é convertida para **double**.

```
cli <- clientes %>% mutate_if(is.integer, as.double)
```

```
> cli
# A tibble: 100 x 6
   CPF      Nome      DDD  Telefone Classificacao Divida
<chr>    <chr>    <chr> <chr>      <chr>      <dbl>
1 10244521310 Margarida Palmas Stutz 12 959128774 L NA
2 11125248785 Cláudio Peixoto Barellos 17 945271192 NA NA
3 12150713373 Benício Arlia Duarte 11 947685566 A NA
4 14436831590 Tereza Pires Barbosa 22 994053970 A NA
5 14768110401 José Ximenes Vargas 11 334940412 NA NA
6 15271527409 Alba Knupp Barrada 22 931442914 NA NA
7 15445645207 Marisa de Padua da Sousa 21 911021168 L NA
8 15796045440 Marlene Silva Meyer 19 991341440 L NA
9 18112077231 Júlio Rosa Viana 22 917547560 L NA
10 19167287409 Lina de Lima Uchôa 13 925186816 L NA
# ... with 90 more rows
```

Pacote dplyr

- A função **replace_na** permite alterar dados de colunas. No exemplo, se o valor do campo Classificacao for **NA** ele assume o valor 'N' e se o valor do campo Divida for **NA** ele assume o valor 0.

```
cli <- clientes %>% replace_na(replace = list(Classificacao = 'N', Divida = 0))
```

```
> cli
# A tibble: 100 x 6
   CPF      Nome      DDD Telefone Classificacao Divida
<chr>    <chr>    <chr> <chr>      <chr>      <dbl>
1 10244521310 Margarida Palmas Stutz 12 959128774 L 0
2 11125248785 Cláudio Peixoto Barellos 17 945271192 N 0
3 12150713373 Benício Arlia Duarte 11 947685566 A 0
4 14436831590 Tereza Pires Barbosa 22 994053970 A 0
5 14768110401 José Ximenes Vargas 11 334940412 N 0
6 15271527409 Alba Knupp Barrada 22 931442914 N 0
7 15445645207 Marisa de Padua da Sousa 21 911021168 L 0
8 15796045440 Marlene Silva Meyer 19 991341440 L 0
9 18112077231 Júlio Rosa Viana 22 917547560 L 0
10 19167287409 Lina de Lima Uchôa 13 925186816 L 0
# ... with 90 more rows
```

Gráficos

- O R já vem com funções básicas que fazem gráficos estatísticos de todas as naturezas.
- Essas funções são rápidas e simples de usar.
- Além das funções do R, vamos utilizar as funções do pacote **ggplot2**.
- Outros pacotes que também podem ser utilizados são o **lattice** e o **ggvis**.

Gráfico de dispersão

- Para construir um **gráfico de dispersão**, utilizamos a função **plot()**, cujos principais parâmetros são:
 - *x*, *y* - vetores com os dados dos eixos *x* e *y*, respectivamente.
 - *type* - tipo de gráfico, podendo ser pontos, linhas, escada, entre outros.

```
n <- 200  
x <- 1:n  
y <- 10 + 3 * x  
plot(x, y)
```

Uma reta formada por 200 pares (x, y).

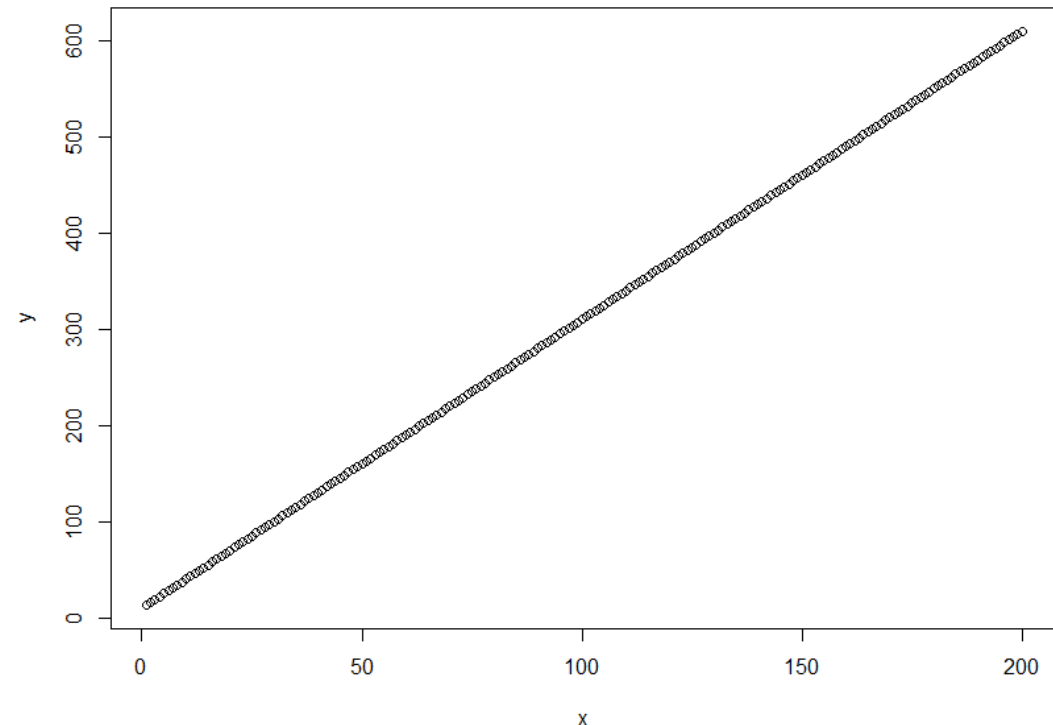


Gráfico de dispersão

- Para construir um **gráfico de dispersão**, utilizamos a função **plot()**, cujos principais parâmetros são:
 - *x*, *y* - vetores com os dados dos eixos *x* e *y*, respectivamente.
 - *type* - tipo de gráfico, podendo ser pontos, linhas, escada, entre outros.

```
n <- 200  
x <- 1:n  
y <- 10 + 3 * x  
y <- y + rnorm(n, sd = 50)  
plot(x, y)
```

rnorm é uma função que simula variáveis normais.

Sua forma de uso é: `x <- rnorm(n, mean, sd)`

- *n* é o tamanho da amostra;
- *mean* é a média (opcional com default 0);
- *sd* é o desvio-padrão (opcional com default 1);

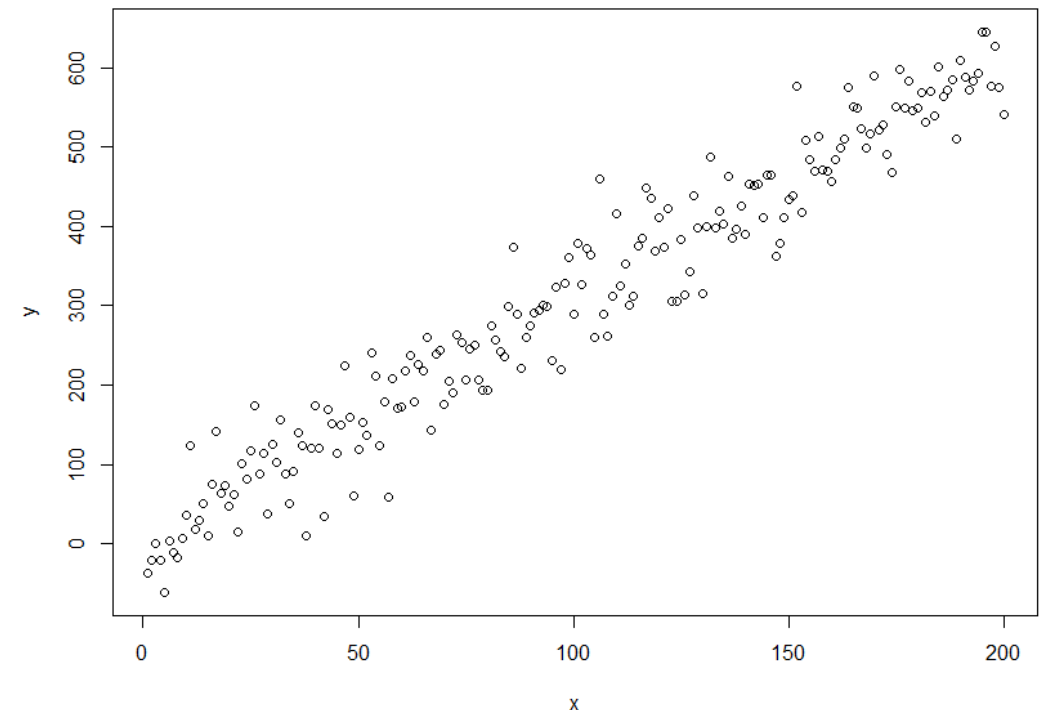
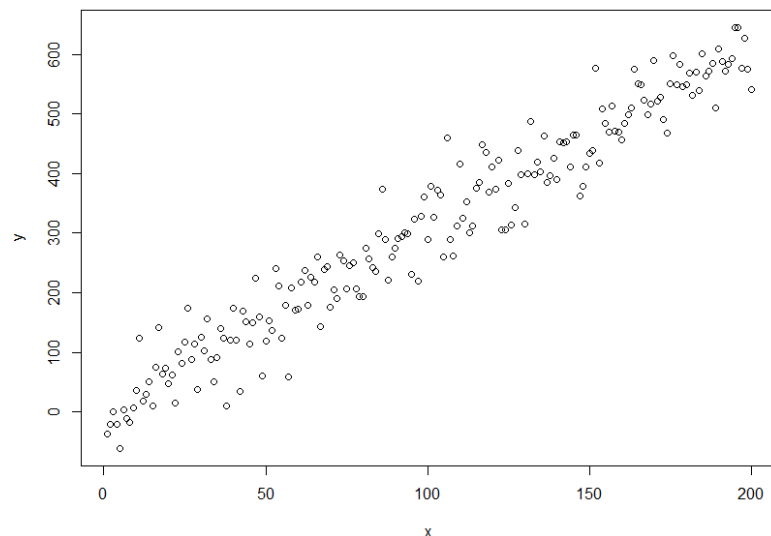


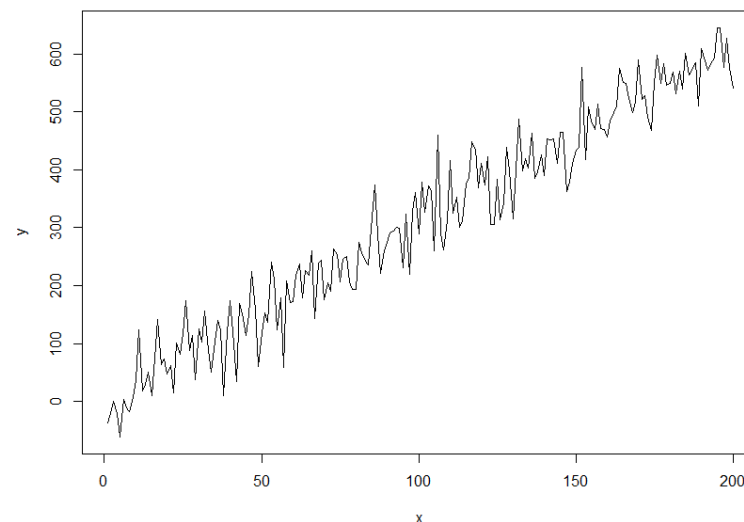
Gráfico de dispersão

- Para construir um **gráfico de dispersão**, utilizamos a função **plot()**, cujos principais parâmetros são:
 - *x, y* - vetores com os dados dos eixos *x* e *y*, respectivamente.
 - *type* - tipo de gráfico, podendo ser pontos, linhas, escada, entre outros.

```
n <- 200
x <- 1:n
y <- 10 + 3 * x + rnorm(n, sd = 50)
plot(x, y)
```



```
n <- 200
x <- 1:n
y <- 10 + 3 * x + rnorm(n, sd = 50)
plot(x, y, type = "l")
```

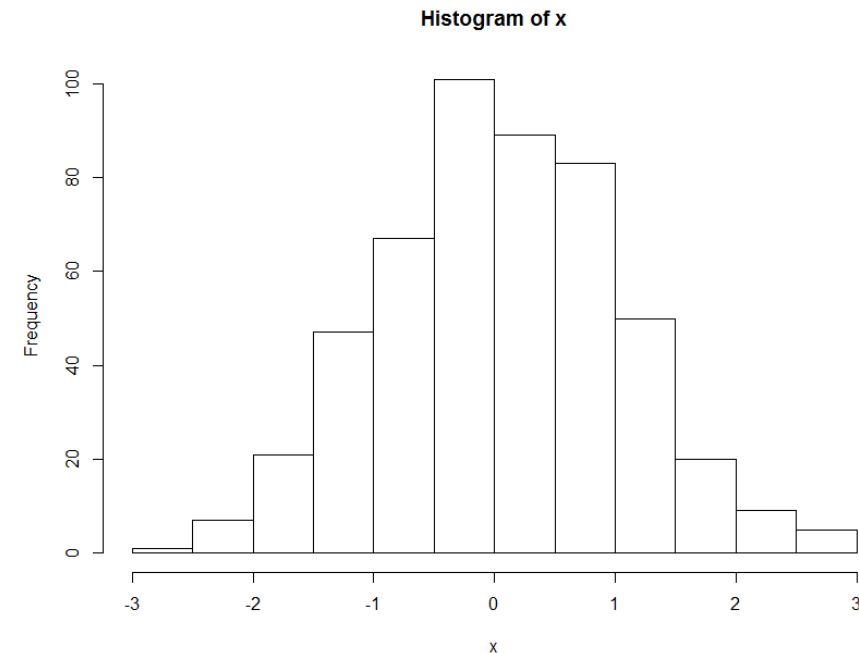


Histograma

- Para construir **histogramas**, utilizamos a função **hist()**, cujos principais parâmetros são:
 - *x* - o vetor numérico para o qual o histograma será construído.
 - *breaks* - o número aproximado de barras.

```
x <- rnorm(500)  
hist(x)
```

Média 0 e desvio padrão 1.

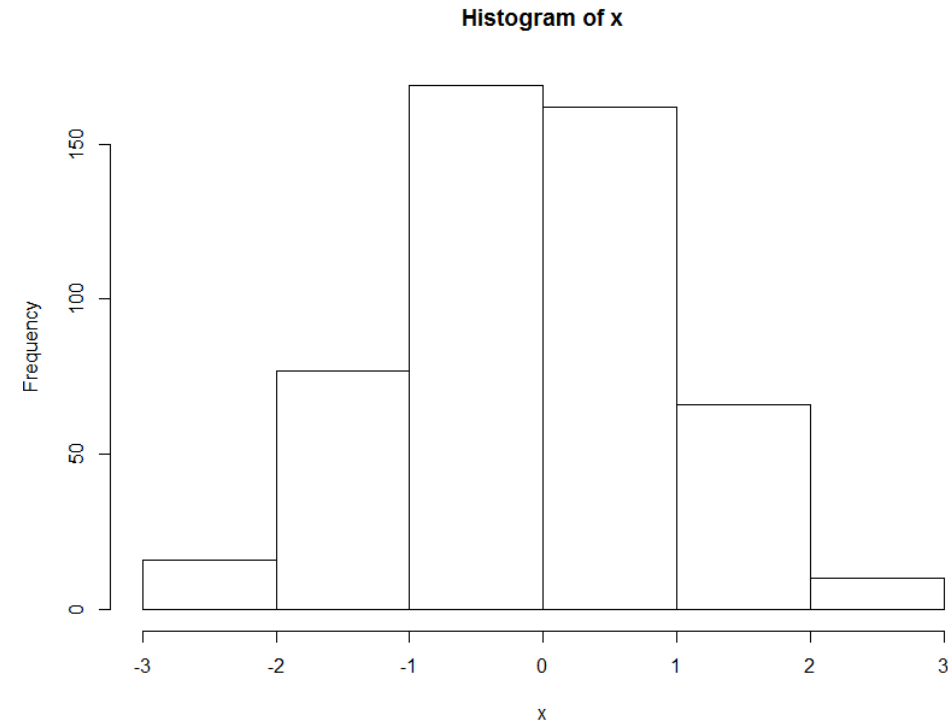


Histograma

- Para construir **histogramas**, utilizamos a função **hist()**, cujos principais parâmetros são:
 - *x* - o vetor numérico para o qual o histograma será construído.
 - *breaks* - o número aproximado de barras.

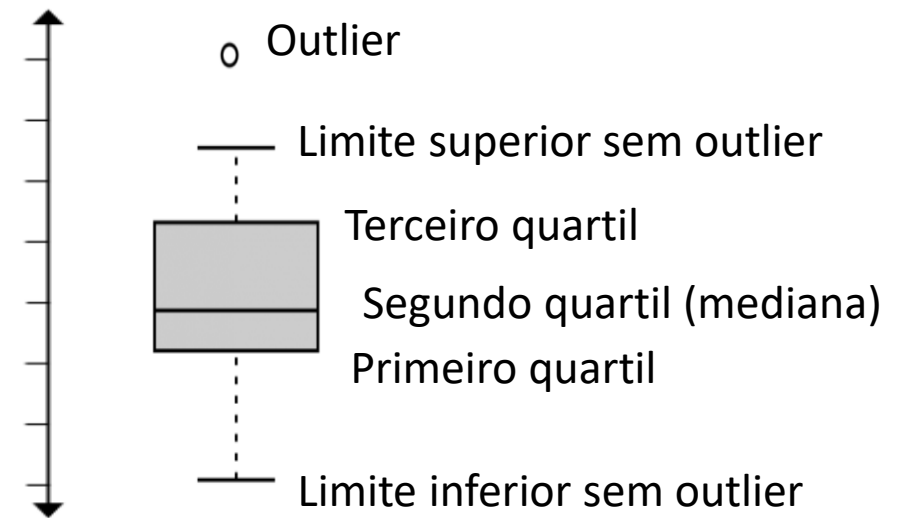
```
x <- rnorm(500)  
hist(x, breaks = 6)
```

Histograma com 6 barras.



Boxplot

- O **boxplot** é um gráfico utilizado para estudar o comportamento de variáveis, sendo composto por:
 - Uma caixa (*box*) que representa a região entre o primeiro e o terceiro quartis (quantis 25% e 75%), ou seja, 50% dos dados estão dentro da caixa.
 - Uma linha dentro da caixa que representa a posição da mediana (segundo quartil ou quantil 50%).
 - Linhas que se prolongam a partir da caixa até no máximo 1,5 vezes a distância interquartil (diferença entre o 1º e 3º quartis).
 - As observações que passarem essa distância são representadas individualmente por pontos.



Boxplot

- Para construir **boxplots**, utilizamos a função **boxplot()**, cujos principais parâmetros são:
 - x - o vetor numérico para o qual o **boxplot** será construído.

```
idade <- c(8, 9, 12, 20, 22, 18, 21, 18, 35, 23, 10, 12, 17)
```

```
summary(idade)
```

```
Min. 1st Qu. Median Mean 3rd Qu. Max.
8.00 12.00 18.00 17.31 21.00 35.00
```

```
boxplot(idade, main="Boxplot: Idade", col="blue")
```

Suponhamos um vetor contendo uma amostra da idade de pessoas registradas em um determinado plano de saúde.

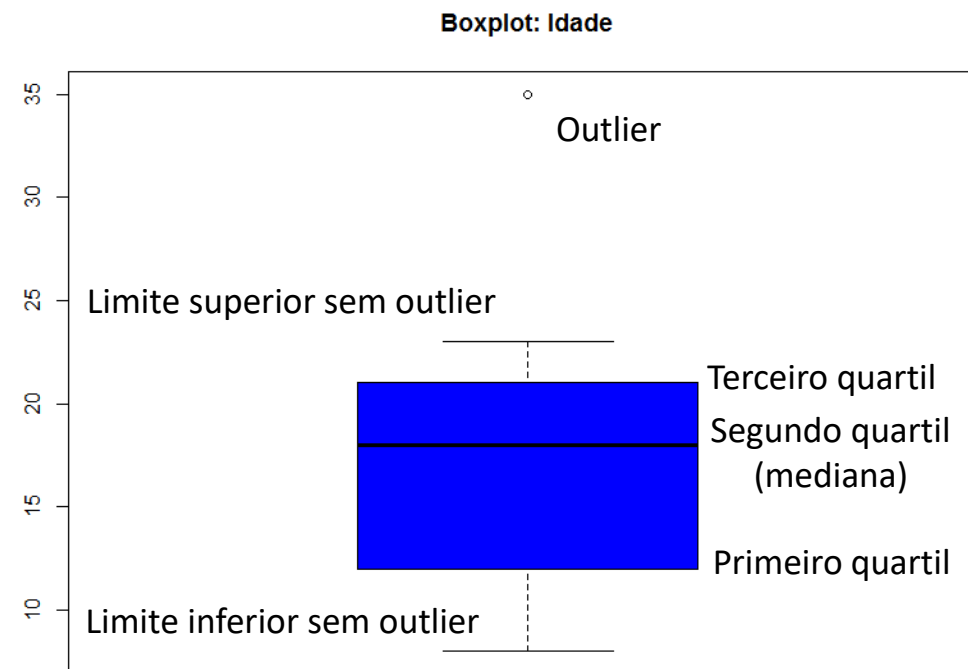


Gráfico de barras

- Uma forma simples de descrever quantitativamente observações é agrupá-las em categorias e contar quantas observações pertencem a cada categoria.
- No R a forma mais simples e direta de obter contagens (frequências) é através da função **table()**.
- Para construir **gráficos de barras**, utilizamos as funções **table()** e **barplot()** combinadas.

Gráfico de barras

- Para construir **gráficos de barras**, utilizamos as funções **table()** e **barplot()** combinadas.

```
filhos <- c(0, 2, 1, 3, 2, 0, 1, 3, 1, 1, 1, 0, 0)
tabela <- table(filhos)
barplot(tabela, main = "Paternidade",
        xlab = "Quantidade de filhos", ylab = "Quantidade de pessoas")
```

Suponhamos um vetor contendo uma amostra do número de filhos de pessoas registradas em um determinado plano de saúde.

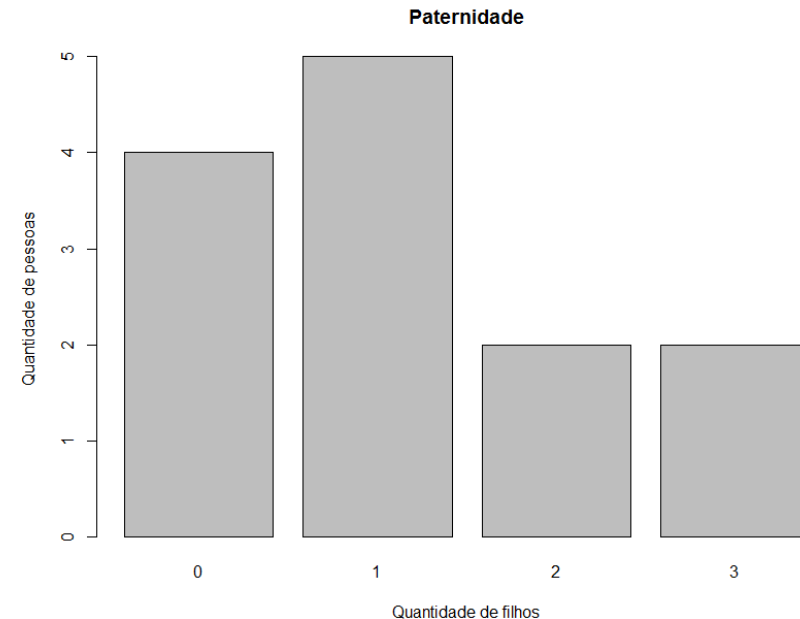
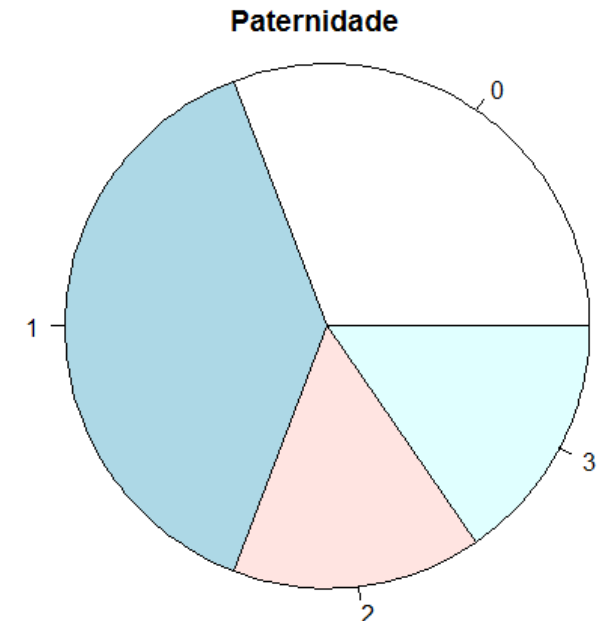


Gráfico de setores

- **Gráficos de setores (pizza)** são normalmente utilizados quando se quer demonstrar a representatividade de cada categoria de uma determinada variável na constituição do total.
- Para construir **setores**, utilizamos a função **pie()**.

```
filhos <- c(0, 2, 1, 3, 2, 0, 1, 3, 1, 1, 1, 0, 0)
tabela <- table(filhos)
pie(tabela, main = "Paternidade")
```

Suponhamos um vetor contendo uma amostra do número de filhos de pessoas registradas em um determinado plano de saúde.



Gráficos com ggplot2

- É um pacote do R voltado para a criação de gráficos estatísticos baseado na **Gramática dos Gráficos** criada por Leland Wilkinson.
- Essa gramática diz que um **gráfico estatístico** é um mapeamento dos dados a partir de **atributos estéticos** (cores, formas, tamanho) de **formas geométricas** (pontos, linhas, barras).
- Seguindo essa gramática, os gráficos no **ggplot2** são criados em camadas, construídas uma a uma.
- Para a criação dos gráficos serão utilizadas duas bases:
 - mtcars – contém dados como consumo, cilindradas, número de carburadores, etc. sobre 32 automóveis.
 - ais - contém dados como sangue e peso de atletas australianos. Para utilizar esta base, deve ser instalado o pacote DAAG.

Bancos ais e mtcars

- Antes de seguir, vamos visualizar as primeiras linhas dos bancos **ais** e **mtcars**, utilizando os comandos:

```
> library(DAAG)
```

```
Carregando pacotes exigidos: lattice
```

```
> head(ais)
```

	rcc	wcc	hc	hg	ferr	bmi	ssf	pcBfat	lbm	ht	wt	sex	sport
1	3.96	7.5	37.5	12.3	60	20.56	109.1	19.75	63.32	195.9	78.9	f	B_Ball
2	4.41	8.3	38.2	12.7	68	20.67	102.8	21.30	58.55	189.7	74.4	f	B_Ball
3	4.14	5.0	36.4	11.6	21	21.86	104.6	19.88	55.36	177.8	69.1	f	B_Ball
4	4.11	5.3	37.3	12.6	69	21.88	126.4	23.66	57.18	185.0	74.9	f	B_Ball
5	4.45	6.8	41.5	14.0	29	18.96	80.3	17.64	53.20	184.6	64.6	f	B_Ball
6	4.10	4.4	37.4	12.5	42	21.04	75.2	15.58	53.77	174.0	63.7	f	B_Ball

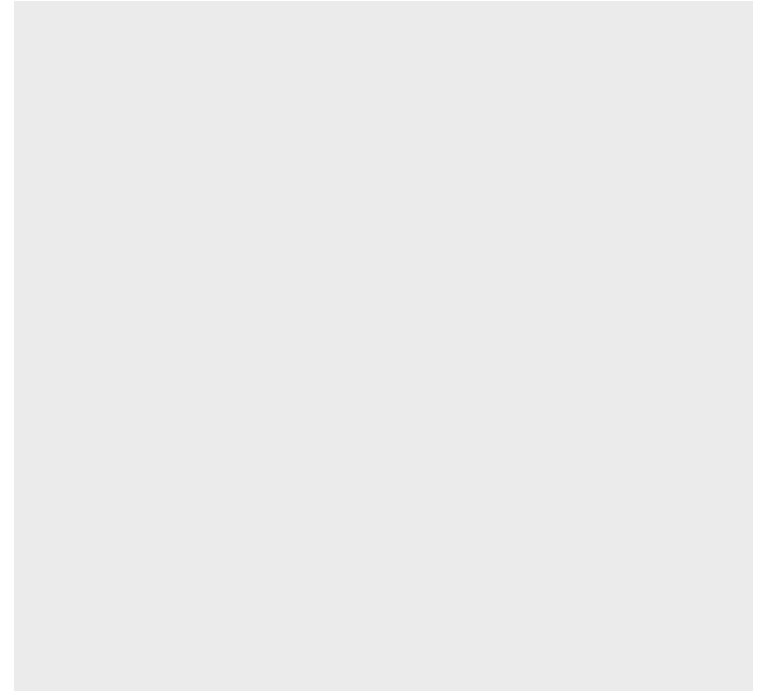
```
> head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

Primeira camada

- A primeira camada é criada pela função **ggplot()** do pacote **ggplot2**, que deve receber como parâmetro um data frame.
- A primeira camada é um painel em branco.

```
library(ggplot2)  
ggplot(data = mtcars)
```

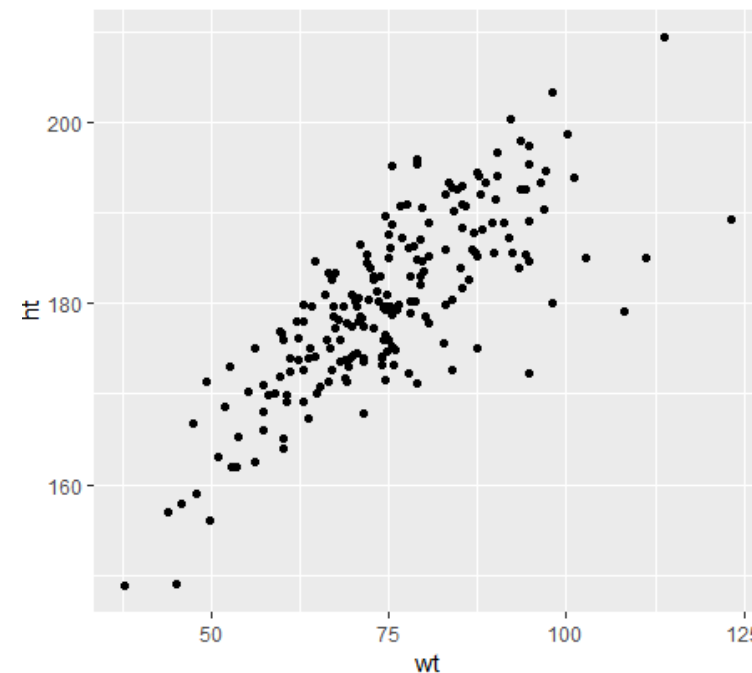


Camada por camada

- Cada camada representa um tipo de mapeamento ou personalização do gráfico.
- A partir da primeira camada, podemos, então, construir as demais.

```
library(DAAG)
library(ggplot2)
ggplot(data = ais) + geom_point(mapping = aes(x = wt, y = ht))
```

Gráfico de dispersão envolvendo os dados peso (eixo x) e altura (eixo y).



Segunda camada

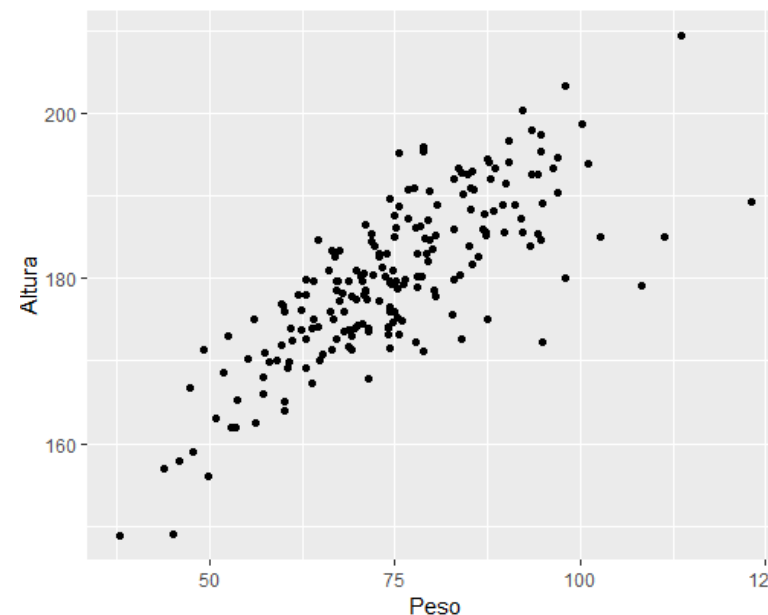
- A primeira camada foi criada pela função **ggplot()**, que recebeu um data frame.
- A segunda camada foi criada pela função **geom_point()** que especificou a **forma geométrica** (gráfico dispersão transformando pares (x, y) em pontos, no caso) utilizada no mapeamento das observações.
- A função **aes()** foi usada para indicar qual variável seria representada no eixo **x** e qual seria representada no eixo **y**.
- As duas camadas foram unidas pelo operador “+”.
- A função **aes()** pode ser utilizada, por exemplo, para:
 - definir a estética (***aesthetics***) das variáveis.
 - indicar a relação entre os dados, no exemplo, par (x, y).
 - definir a cor e o tamanho dos componentes geométricos.

Terceira camada

- A primeira camada foi criada pela função **ggplot()**, que recebeu um data frame.
- A segunda camada foi criada pela função **geom_point()** que especificou a **forma geométrica** (gráfico dispersão, no caso) utilizada no mapeamento das observações.
- As funções **labs()**, **xlab()** e **ylab()** podem ser utilizadas para acrescentar rótulos aos eixos **x** e **y**.

```
library(DAAG)
library(ggplot2)

ggplot(data = ais) +
  geom_point(mapping = aes(x = wt, y = ht)) +
  labs(x = "Peso", y = "Altura")
```

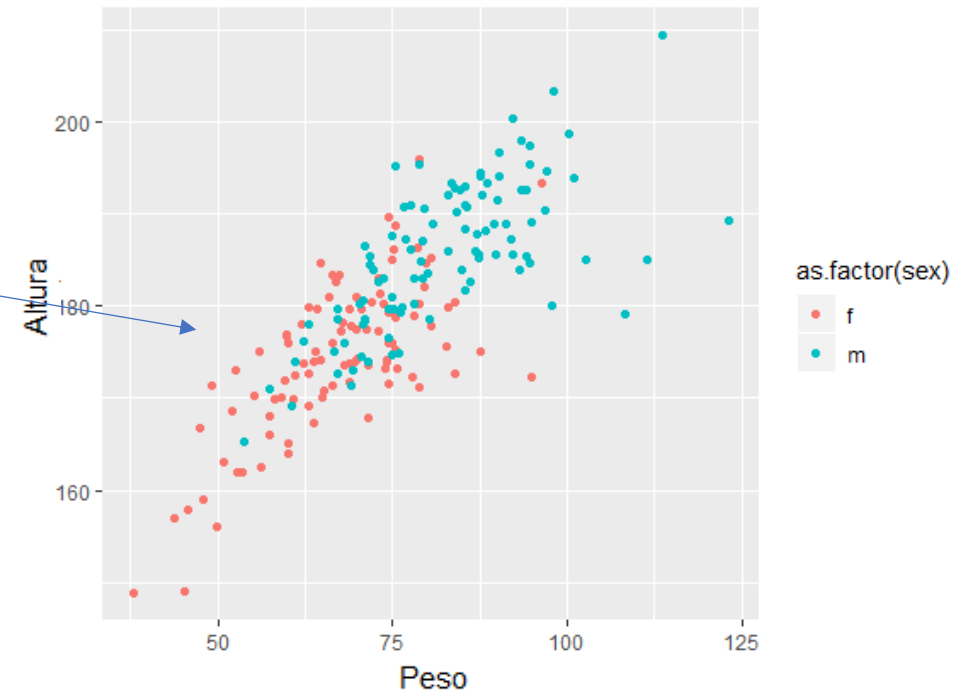


Estética

- Outro aspecto que pode ser mapeado nesse gráfico é a cor dos pontos.

```
library(DAAG)
library(ggplot2)
ggplot(data = ais) +
  geom_point(mapping = aes(x = wt, y = ht, color = as.factor(sex))) +
  labs(x = "Peso", y = "Altura")
```

A variável categórica sex (f ou m) do data frame foi utilizada como fator para definir as cores dos pontos.

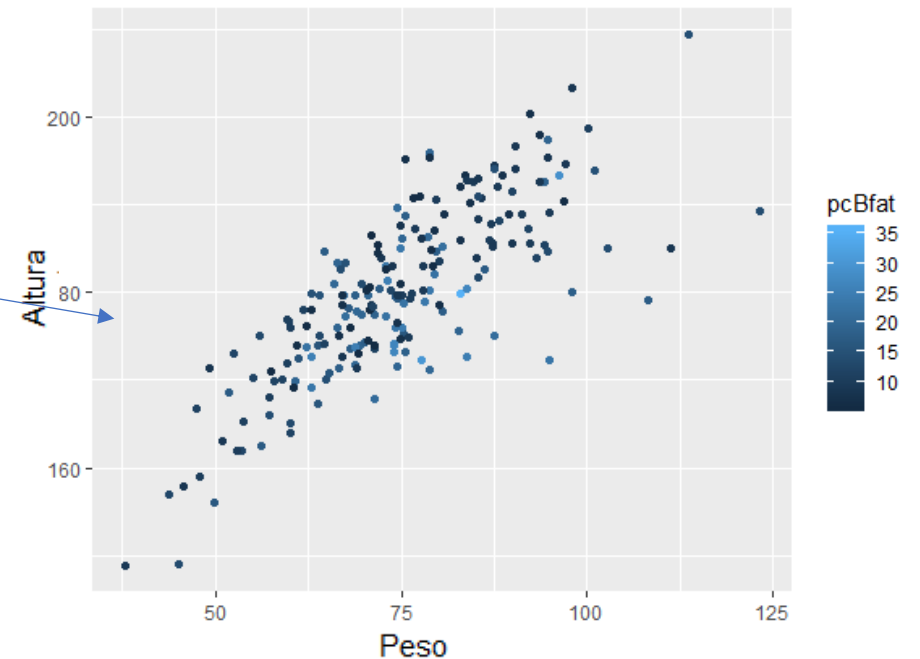


Estética

- Outro aspecto que pode ser mapeado nesse gráfico é a cor dos pontos.

```
library(DAAG)
library(ggplot2)
ggplot(data = ais) +
  geom_point(mapping = aes(x = wt, y = ht, color = pcBfat)) +
  labs(x = "Peso", y = "Altura")
```

A variável contínua pcBfat do data frame foi utilizada para definir as cores dos pontos.



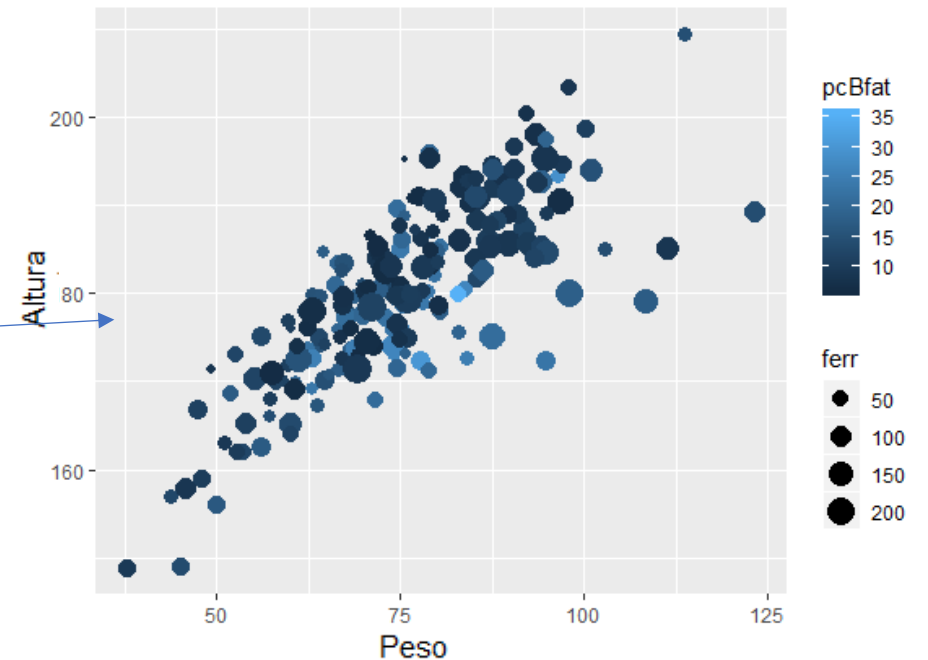
Estética

- Outro aspecto que pode ser mapeado nesse gráfico é a cor dos pontos.

```
library(DAAG)
library(ggplot2)
ggplot(data = ais) +
  geom_point(mapping = aes(x = wt, y = ht, color = pcBfat, size = ferr)) +
  labs(x = "Peso", y = "Altura")
```

A variável contínua pcBfat do data frame foi utilizada para definir as cores dos pontos.

A variável contínua ferr do data frame foi utilizada para mapear o tamanho dos pontos

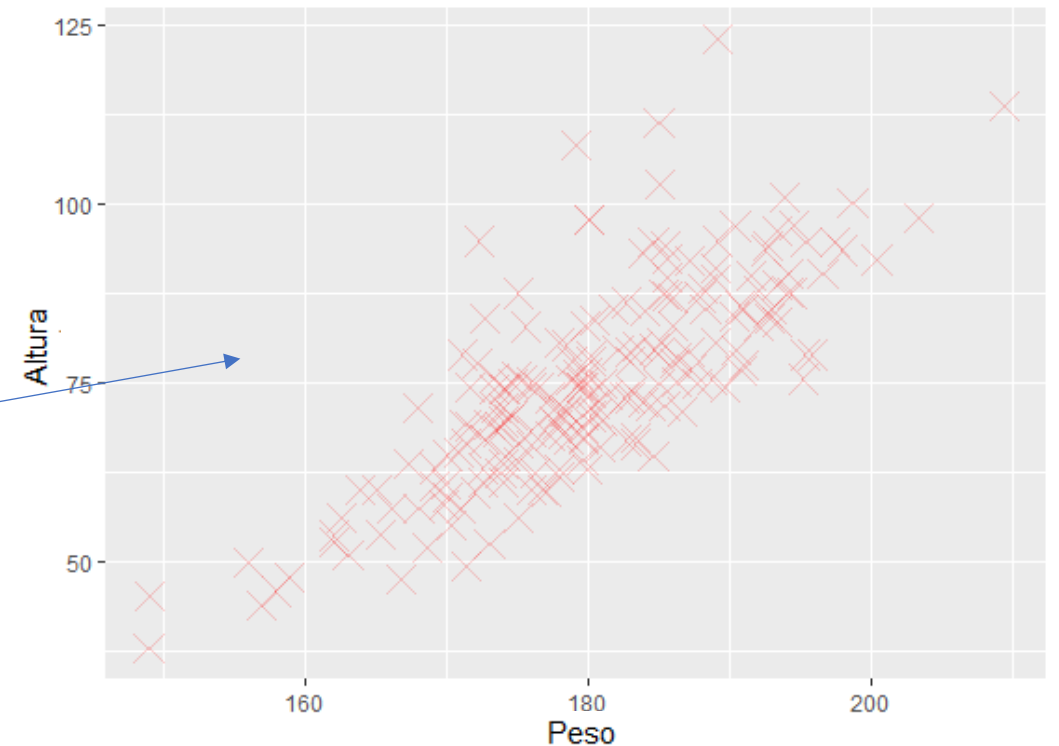


Estética

- Outros aspectos também podem ser definidos.

```
library(DAAG)
library(ggplot2)
ggplot(ais, aes(y = wt, x = ht)) +
  geom_point(color = "red", size = 5, shape = 4, alpha = 0.2) +
  labs(x = "Peso", y = "Altura")
```

Em `geom_point` `color` define a cor do ponto, `size`, o tamanho do ponto, `shape`, o formato do ponto e `alpha`, a transparência (entre 1 e 0).



Geoms_XXXX

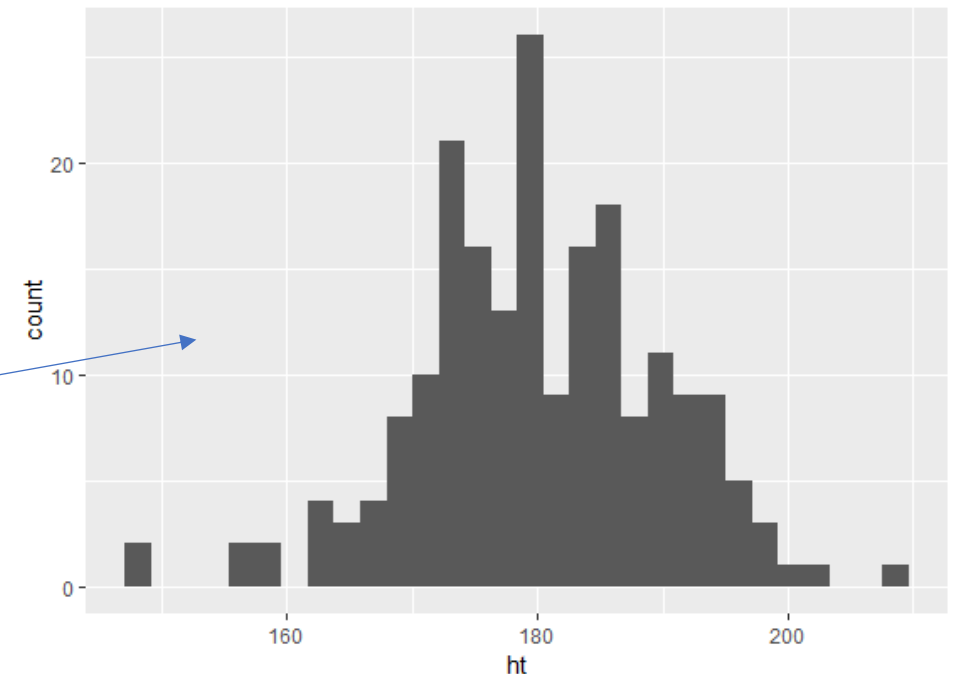
- Exemplos de outros geoms:
 - `geom_point` – para gráficos de dispersão.
 - `geom_line` - para linhas definidas por pares (x,y).
 - `geom_abline` - para retas definidas por um intercepto e uma inclinação.
 - `geom_hline` - para retas horizontais.
 - `geom_bar` - para barras.
 - `geom_histogram` - para histogramas.
 - `geom_boxplot` - para boxplots.
 - `geom_density` - para densidades.
 - `geom_area` - para áreas.
- Para fazer um gráfico usando `ggplot2` e a gramática de gráficos, seguimos o padrão:
 - `ggplot(data = DATA) + GEOM_FUNCTION(mapping = aes(MAPPINGS))`

Geoms_histogram

- Para fazer um histograma, precisamos passar uma variável x, sendo que y é a frequência calculada.

```
library(DAAG)
library(ggplot2)
ggplot(ais) + geom_histogram(aes(x = ht), bins = 30)
```

A variável ht é usada como base para gerar o histograma.

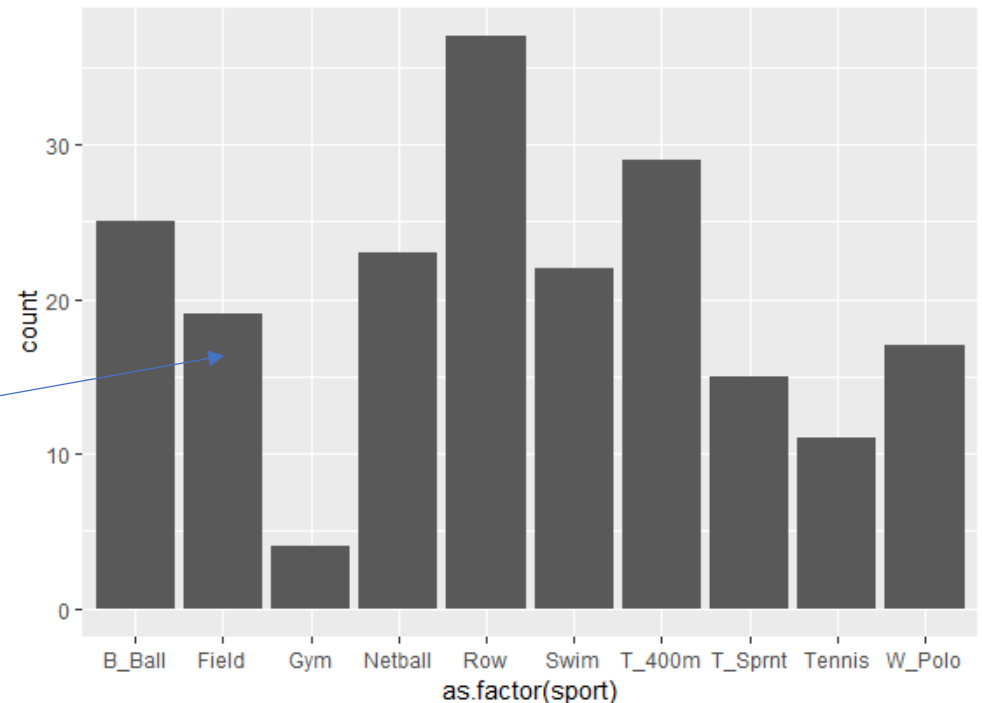


Geoms_bar

- Para fazer um histograma, precisamos passar uma variável x, sendo que y é a frequência calculada.

```
library(DAAG)
library(ggplot2)
ggplot(ais) + geom_bar(aes(x = as.factor(sport)))
```

A variável sport é usada como base para gerar o gráfico de barras.

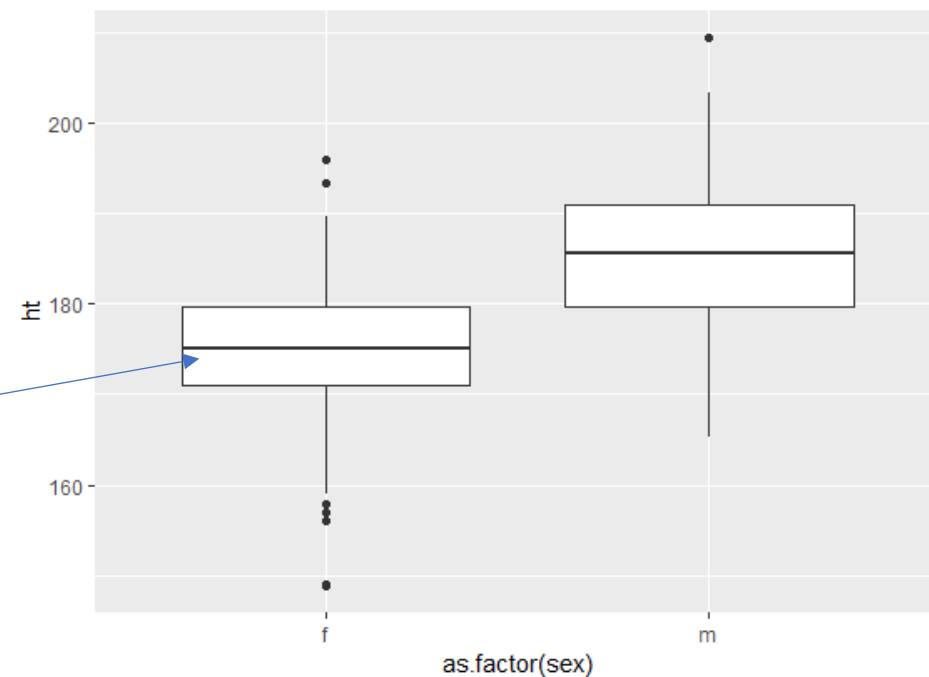


Geoms_boxplot

- Para fazer um boxplot para cada grupo, precisamos passar um fator para o aspecto x do gráfico.

```
library(DAAG)
library(ggplot2)
ggplot(ais) + geom_boxplot(aes(x = as.factor(sex), y = ht))
```

A variável sex é usada como fator para construção do boxplot considerando a variável altura.



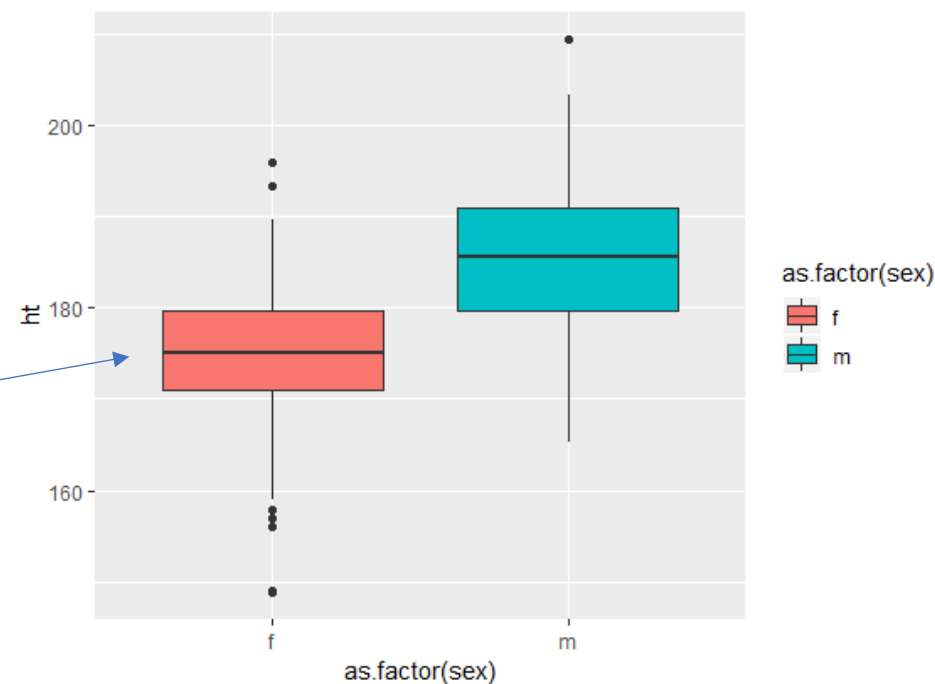
Geoms_boxplot

- Para fazer um boxplot para cada grupo, precisamos passar um fator para o aspecto x do gráfico.

```
library(DAAG)
library(ggplot2)
ggplot(ais) + geom_boxplot(aes(x = as.factor(sex), y = ht, fill = as.factor(sex)))
```

Legenda.

A variável sex é usada como fator para construção do boxplot considerando a variável altura.



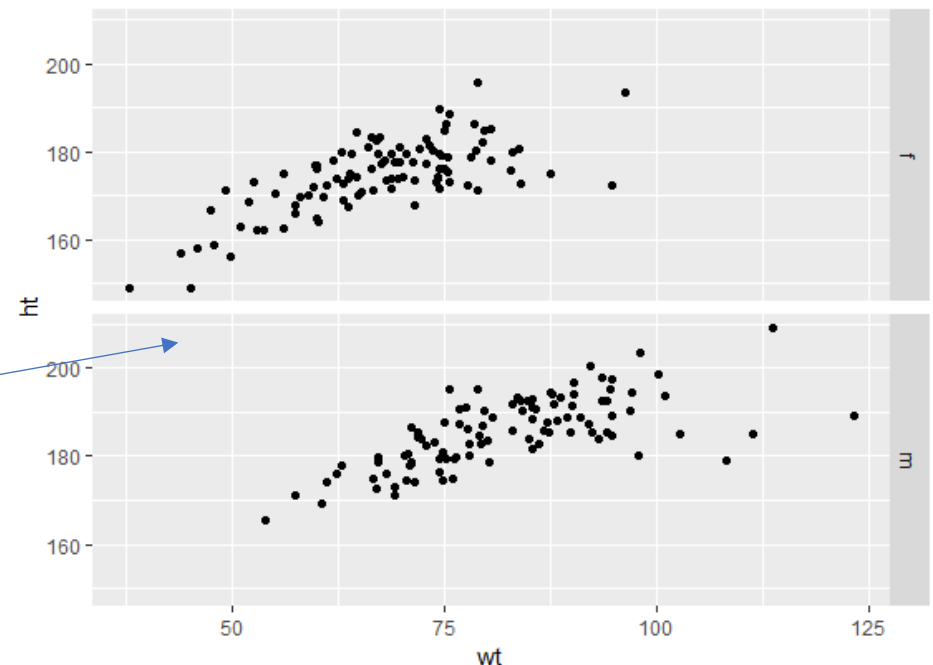
Facets

- Auxilia na visualização de diferentes subconjuntos dos dados em gráficos separados.

```
library(DAAG)
library(ggplot2)
ggplot(data = ais) + geom_point(mapping = aes(x = wt, y = ht)) + facet_grid(sex~.)
```

Conjuntos na horizontal.

A variável sex é usada na função facet_grid para dividir o conjunto de dados em masculino e feminino.



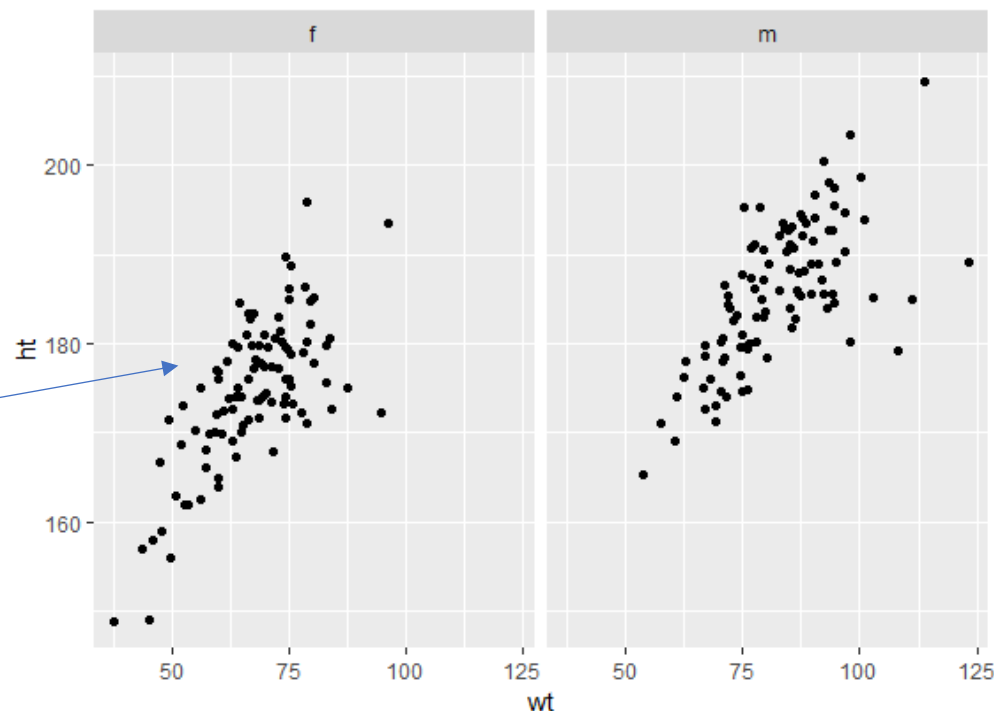
Facets

- Auxilia na visualização de diferentes subconjuntos dos dados em gráficos separados.

```
library(DAAG)
library(ggplot2)
ggplot(data = ais) + geom_point(mapping = aes(x = wt, y = ht)) + facet_grid(.~sex)
```

Conjuntos na vertical.

A variável sex é usada na função facet_grid para dividir o conjunto de dados em masculino e feminino.



Exemplo

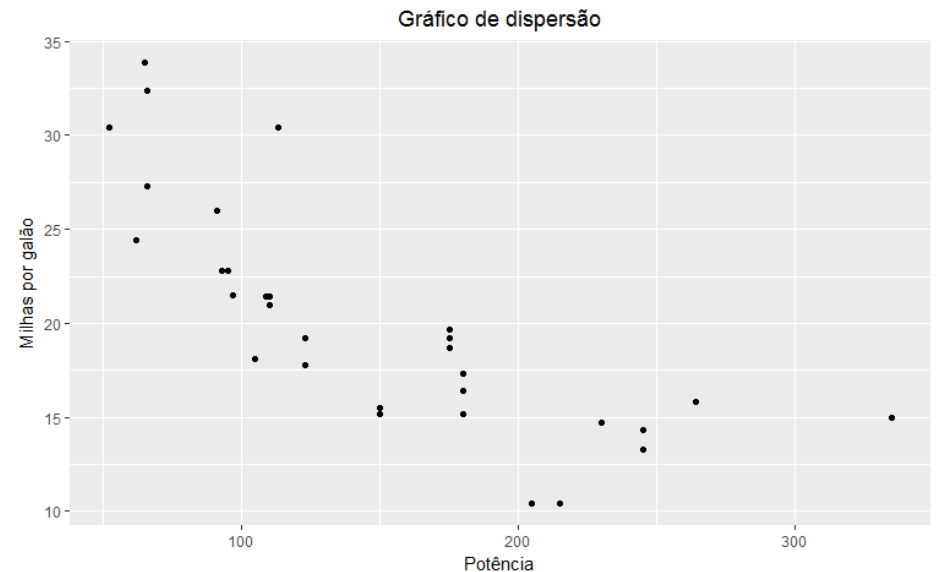
- Usando o banco **mtcars** e o pacote **ggplot2**, elabore um gráfico de dispersão, um gráfico barras, um histograma e um boxplot. Todos os gráficos deverão ter título e rótulos nos eixos x e y quando aplicável. Para o gráfico de dispersão criado, elabore um novo utilizando **facets**.

Exemplo - Resolução

- Usando o banco **mtcars** e o pacote **ggplot2**, elabore um gráfico de dispersão, um gráfico barras, um histograma e um boxplot. Todos os gráficos deverão ter título e rótulos nos eixos x e y quando aplicável. Para o gráfico de dispersão criado, elabore um novo utilizando **facets**.

```
library(ggplot2)
ggplot(data = mtcars) +
  geom_point(mapping = aes(x = hp, y = mpg)) +
  labs(x = "Potência", y = "Milhas por galão") +
  ggtitle("Gráfico de dispersão") +
  theme(plot.title = element_text(hjust = 0.50))
```

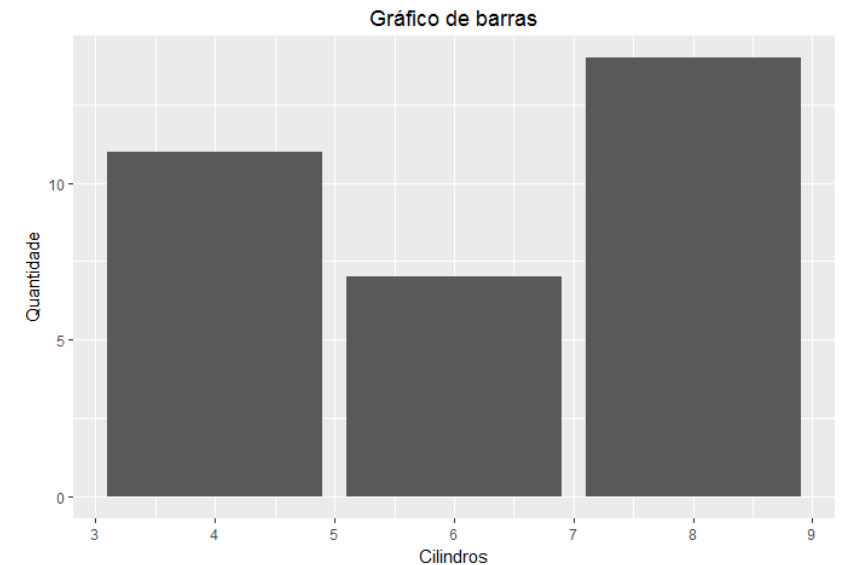
Por padrão o título é alinhado à esquerda. Use **theme** conforme descrito acima para centralizar o título.



Exemplo - Resolução

- Usando o banco **mtcars** e o pacote **ggplot2**, elabore um gráfico de dispersão, um gráfico barras, um histograma e um boxplot. Todos os gráficos deverão ter título e rótulos nos eixos x e y quando aplicável. Para o gráfico de dispersão criado, elabore um novo utilizando **facets**.

```
library(ggplot2)
ggplot(data = mtcars) +
  geom_bar(mapping = aes(x = cyl)) +
  labs(x = "Cilindros", y = "Quantidade") +
  ggtitle("Gráfico de barras") +
  theme(plot.title = element_text(hjust = 0.50))
```

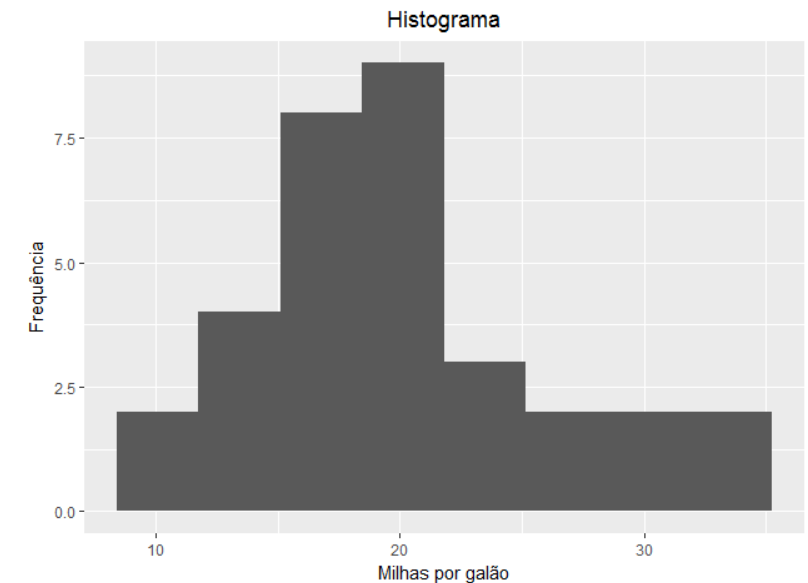


Exemplo - Resolução

- Usando o banco **mtcars** e o pacote **ggplot2**, elabore um gráfico de dispersão, um gráfico barras, um histograma e um boxplot. Todos os gráficos deverão ter título e rótulos nos eixos x e y quando aplicável. Para o gráfico de dispersão criado, elabore um novo utilizando **facets**.

```
library(ggplot2)
ggplot(data = mtcars) +
  geom_histogram(mapping = aes(mtcars$mpg), bins = 8) +
  labs(x = "Milhas por galão", y = "Frequência") +
  ggtitle("Histograma") +
  theme(plot.title = element_text(hjust = 0.50))
```

O parâmetro bins define o número de divisões do histograma.

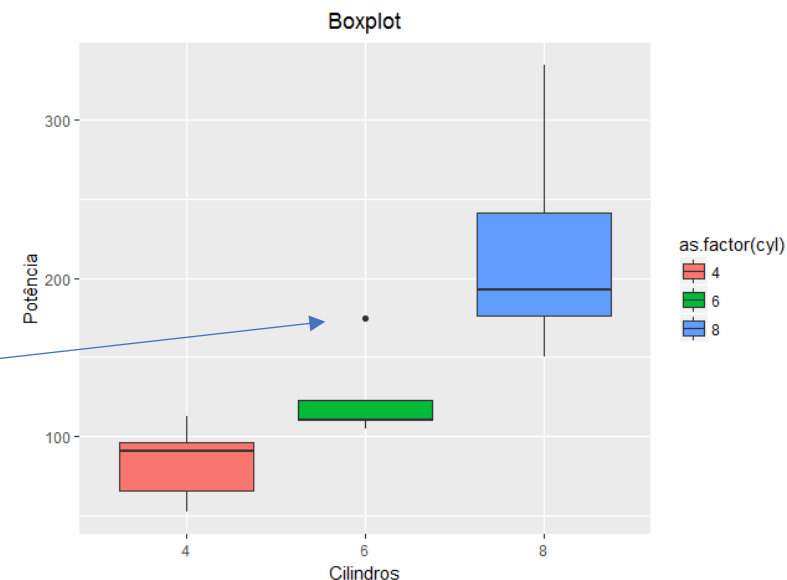


Exemplo - Resolução

- Usando o banco **mtcars** e o pacote **ggplot2**, elabore um gráfico de dispersão, um gráfico barras, um histograma e um boxplot. Todos os gráficos deverão ter título e rótulos nos eixos x e y quando aplicável. Para o gráfico de dispersão criado, elabore um novo utilizando **facets**.

```
library(ggplot2)
ggplot(data = mtcars) +
  geom_boxplot(mapping = aes(x = as.factor(cyl), y = hp, fill = as.factor(cyl))) +
  labs(x = "Cilindros", y = "Potência") +
  ggtitle("Boxplot") +
  theme(plot.title = element_text(hjust = 0.50))
```

Exemplo de um outlier.

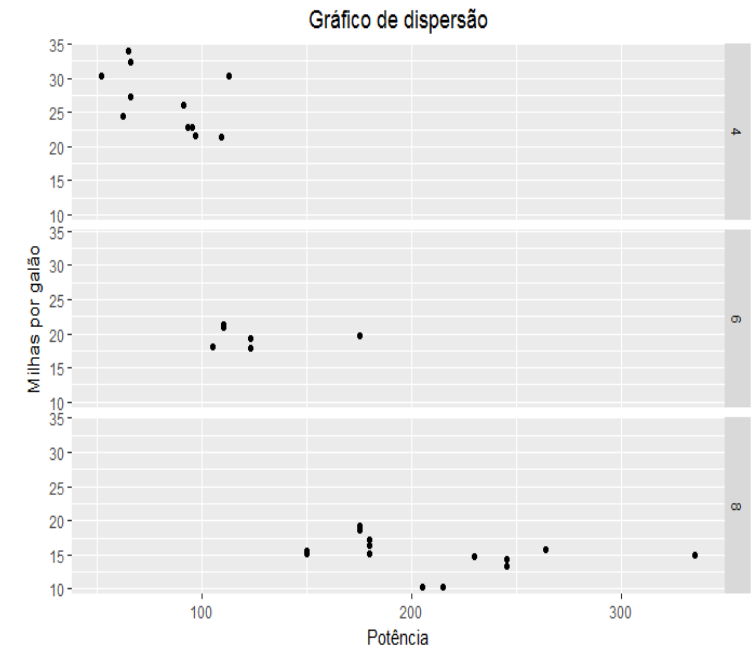


Exemplo - Resolução

- Usando o banco **mtcars** e o pacote **ggplot2**, elabore um gráfico de dispersão, um gráfico barras, um histograma e um boxplot. Todos os gráficos deverão ter título e rótulos nos eixos x e y quando aplicável. Para o gráfico de dispersão criado, elabore um novo utilizando **facets**.

```
library(ggplot2)
ggplot(data = mtcars) +
  geom_point(mapping = aes(x = hp, y = mpg)) +
  facet_grid (cyl~.) +
  labs(x = "Potência", y = "Milhas por galão") +
  ggtitle("Gráfico de dispersão") +
  theme(plot.title = element_text(hjust = 0.50))
```

Uso do recurso **facets**.



Gerando mapas

- Existem diferentes bibliotecas para a elaboração de mapas no R. No nosso exemplo, foram utilizadas as funções **st_read()** (pacote **sf**) e **plot** nativa do R.
- Um dos recursos utilizados no R para a plotagem de mapas é o *shapefile*, que é um formato popular (formado por diferentes arquivos) contendo dados geoespaciais em forma de vetor usado por Sistemas de Informações Geográficas (SIG).
- Os *shapefiles* normalmente possuem não só as informações para o desenho do mapa propriamente dito, como também informações como código e nome de estados e municípios, entre outras.
- Muitos *shapefiles* podem ser encontrados nos próprios pacotes de mapas e também em fontes como o IBGE (<https://www.ibge.gov.br/geociencias/downloads-geociencias.html>).
- No nosso exemplo foi necessário realizar o "download" do arquivo `rj_municipios.zip` seguindo o caminho (o arquivo já está disponível no EAD):
`https:\\geoftp.ibge.gov.br\\organizaç o_do_territ rio\\malhas_territoriais\\Malhas_municipais\\Munic pio_2018\\Ufs\\RJ.`

Gerando mapas

- Inicialmente precisamos importar para o R o *shapefile* do mapa a ser trabalhado. Será utilizado o *shapefile* **33MUE250GC_SIR.SHP** dos municípios do RJ que está no arquivo **rj_municípios.zip**.
- A carga do *shapefile* pode ser feita através da função **st_read()**.
- Observe no *workspace* do RStudio (canto superior direito) a estrutura do objeto Rio (é um data frame), onde temos dados espaciais e não espaciais.

```
library(sf)
Rio <- st_read("C:\\Temp\\33MUE250GC_SIR.shp")
str(Rio)
n <- length(Rio$CD_GEOCMU) # ou length(Rio$NM_MUNICIP)
```

No exemplo, o *shapefile* **33MUE250GC_SIR.SHP**, extraído do arquivo **rj_municípios.zip**, possui a informação geoespacial (**geometry**) da divisão do estado do Rio em municípios bem os códigos IBGE (**CD_GEOCMU**) e os nomes (**NM_MUNICIP**) dos municípios.

O valor de **n** é **92**, ou seja, são **92** municípios.

Gerando mapas

- A seguir, deve ser escolhido o conjunto de dados (população, número de escolas, índice de analfabetismo, entre outros) utilizado como base para colorir as regiões (no caso os municípios) do mapa. Os dados podem ser obtidos de fontes como IBGE, FGV, entre outros ou mesmo gerados por você.
- Será utilizado o arquivo **DadosRJ.CSV**: possui cabeçalho (codigo; populacao; pibpercapita); dados separados por “;”; separador de casas decimais “,”. Para arquivos que possuem texto é boa prática informar o parâmetro **encoding** (UTF-8, ISO-8859-1, Shift-JIS, etc) que identifica como o texto deve ser reconstruído, em caracteres válidos, ao se ler o arquivo.
- O arquivo **DadosRJ.CSV** foi gerado a partir de dados obtidos no IBGE e possui 3 colunas: **codigo**, contendo o código IBGE do município, ou seja, o mesmo usado no arquivo de mapas; **populacao**, contendo a população residente do município em 2010; **pibpercapita**, contendo o PIB per capita do município no ano de 2010.

```
library(descr)
file.head("C:\\Temp\\DadosRJ.CSV")
dados <- read.csv("C:\\Temp\\DadosRJ.CSV", header=T, sep=";", dec=",")
```

Gerando mapas

- Já temos os dados geoespaciais e os atributos básicos dos municípios (NM_MUNICIP e CD_GEOCMU) na variável **Rio** e o conjunto de dados escolhido na variável **dados**.
- No passo seguinte, deve ser feita a junção das duas variáveis em uma única variável utilizando, por exemplo, o comando **merge**.

```
mapa_com_dados <- merge(Rio, dados, by.x = "CD_GEOCMU", by.y = "codigo")  
str(mapa_com_dados)
```

- **by.x** e **by.y** indicam, respectivamente, qual coluna do primeiro data frame (**Rio**) e do segundo data frame (**dados**) devem ser utilizadas como índice para realizar a junção.
- Observe que a coluna **codigo** do data.frame **dados** (segundo data.frame no comando **merge**) não é copiada para o data.frame resultante **mapa_com_dados** pois seria redundante em termos de informações.

Gerando mapas

- Finalmente podemos realizar a plotagem do mapa através do comando **plot** utilizando uma das colunas de dados escolhidos como referência.

```
library(sf)

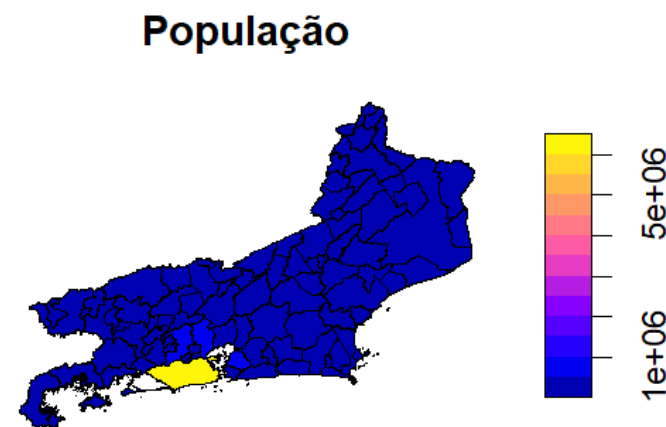
Rio <- st_read("C:\\Tempo\\33MUE250GC_SIR.shp")

dados <- read.csv("C:\\Tempo\\DadosRJ.CSV", header=T, sep=";", dec=",")

mapa_com_dados <- merge(Rio, dados, by.x = "CD_GEOCMU", by.y = "codigo")

plot(mapa_com_dados["populacao"], main = "População")
```

Use o comando **file.head** do pacote **descr** para explorar o arquivo **DadosRJ.CSV**.



Gerando mapas

- Finalmente podemos realizar a plotagem do mapa através do comando **plot** utilizando uma das colunas de dados escolhidos como referência.

```
library(sf)

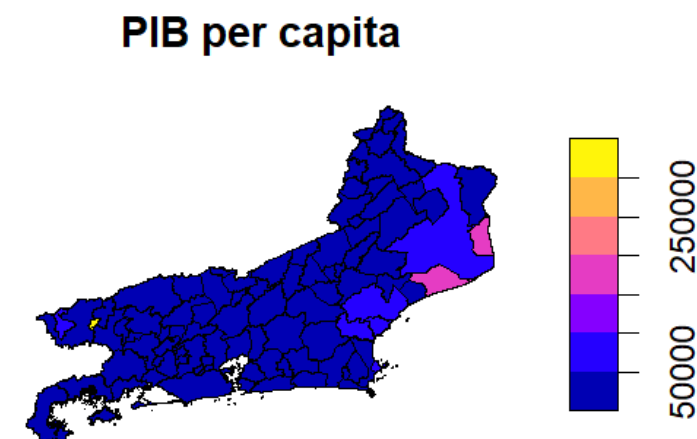
Rio <- st_read("C:\\Tempo\\33MUE250GC_SIR.shp")

dados <- read.csv("C:\\Tempo\\DadosRJ.CSV", header=T, sep=";", dec=",")

mapa_com_dados <- merge(Rio, dados, by.x = "CD_GEOCMU", by.y = "codigo")

plot(mapa_com_dados["pibpercapita"], main = "PIB per capita")
```

Use o comando **file.head** do pacote **descr** para explorar o arquivo **DadosRJ.CSV**.



Gerando mapas

- O pacote **geobr** carrega automaticamente *shapefiles* de conjuntos de dados espaciais oficiais do Brasil. O pacote **sf**, por sua vez, é um dos principais pacotes para trabalhar com dados vetoriais no R.

```
# Pacotes
library(dplyr)
library(geobr)
library(ggplot2)
library(sf)

# Função read_municipality do pacote geobr faz o download automático do shapefile
# dos municípios do RJ para o ano de 2018.
municipios <- read_municipality("RJ", 2018)

# Realiza a leitura dos dados do PIB.
dados <- read.csv("C:\\Temp\\DadosRJ.CSV", header = T, sep=";", dec=",")
```

Gerando mapas

- O pacote **geobr** carrega automaticamente *shapefiles* de conjuntos de dados espaciais oficiais do Brasil. O pacote **sf**, por sua vez, é um dos principais pacotes para trabalhar com dados vetoriais no R.

```
# O merge é utilizado para juntar os dados espaciais e não espaciais,  
# sendo code_muni a coluna do objeto municipios que contém os códigos do municípios  
# e codigo a coluna do data frame dados que contém os códigos dos municípios.  
# Desta forma, mapa_com_dados contém o data frame final.  
mapa_com_dados <- merge(municipios, dados, by.x = "code_muni", by.y = "codigo")  
  
# geom_sf é utilizado em conjunto com o ggplot para plotar o gráfico utilizando a  
# coluna pibpercapita para definir a escala de cores.  
ggplot() +  
  geom_sf(data = mapa_com_dados, aes(fill = pibpercapita)) +  
  labs(fill="PIB per Capita", title="PIB per Capita dos Municípios do RJ") +  
  scale_fill_distiller (palette = "Greens")
```

Gerando mapas

- O pacote **geobr** carrega automaticamente *shapefiles* de conjuntos de dados espaciais oficiais do Brasil. O pacote **sf**, por sua vez, é um dos principais pacotes para trabalhar com dados vetoriais no R.

```
library(dplyr); library(geobr); library(ggplot2); library(sf)

municipios <- read_municipality("RJ", 2018)

dados <- read.csv("C:\\Temp\\DadosRJ.CSV", header = T, sep=";", dec=",")

mapa_com_dados <- merge(municipios, dados, by.x = "code_muni", by.y = "codigo")

ggplot() +
  geom_sf(data = mapa_com_dados, aes(fill = pibpercapita)) +
  labs(fill="PIB per Capita", title="PIB per Capita dos Municípios do RJ") +
  scale_fill_distiller (palette = "Greens")
```

Gerando mapas

- O pacote **geobr** carrega automaticamente *shapefiles* de conjuntos de dados espaciais oficiais do Brasil. O pacote **sf**, por sua vez, é um dos principais pacotes para trabalhar com dados vetoriais no R.

